THE DEVELOPER'S FIELD GUIDE · COMMUNITY EDITION 1.4.2

Plane + GitHub

From your first work item to your next merged pull request.

A detailed, illustrated handbook for developers using Plane for tasks, GitHub for code, and git-sync to keep the two connected.

Meet the tools. Find your work. Link your code. Understand what happens next.

Website team · Fictional teaching examples
Verified against the running CE interface and this repository's code
15 October 2026

Contents

Each chapter link opens its starting page. Use your PDF viewer's bookmarks to jump to individual sections.

01. Welcome to the team 3

02. Getting in and making Plane yours 7

03. Finding your way around Plane 15

04. Work items in depth 24

05. Seeing work your way 41

06. Planning with cycles, modules, pages and intake 51

07. Notifications and staying informed 62

08 · Connect your personal GitHub account 66

09 · Link branches, commits, and pull requests 73

10 · Magic words and Smart Commits 81

11 · What syncs, in which direction, and when 90

12. A day in the life 101

13. Team conventions and good habits 116

14. Troubleshooting 120

15. FAQ and glossary 130

16. Printable cheat sheet 139

> **Start here.** Read chapters 1–3 on your first morning. Use chapters 4–7 while working in Plane. Chapters 8–12 connect that work to GitHub. Keep the last page beside your terminal.

01. Welcome to the team

Your team keeps its plans in Plane and its code in GitHub. This handbook helps you use both without losing the connection between a request and the change that delivers it. You do not need to install the server. You need an invitation, access to the right project, and your usual development environment.

Plane is a shared place to describe work. A **work item** is one record of an outcome the team wants: fix a bug, add a feature, investigate a problem, or write a document. Each record has a short identifier, a description and properties that help the team organize it. You will see `WEB-42` throughout this book. `WEB` identifies the Website project; `42` is that item's sequence number. The title can improve over time while the identifier remains the reference people use in conversation and code.

Git is the version control tool on your computer. It records changes in commits and lets you work on a branch without immediately changing the team's main branch. GitHub hosts the shared repository and provides a place to discuss a proposed change. That discussion happens in a **pull request**, often shortened to PR. Merging a PR puts its accepted changes into the selected base branch.

git-sync is this repository's integration service. It recognizes Plane identifiers in supported GitHub events, adds development links, applies configured workflow rules and can mirror GitHub issues with Plane work items. It is not a replacement for reading the task, testing the change or requesting review. It handles the bookkeeping that would otherwise require someone to copy information between tools.

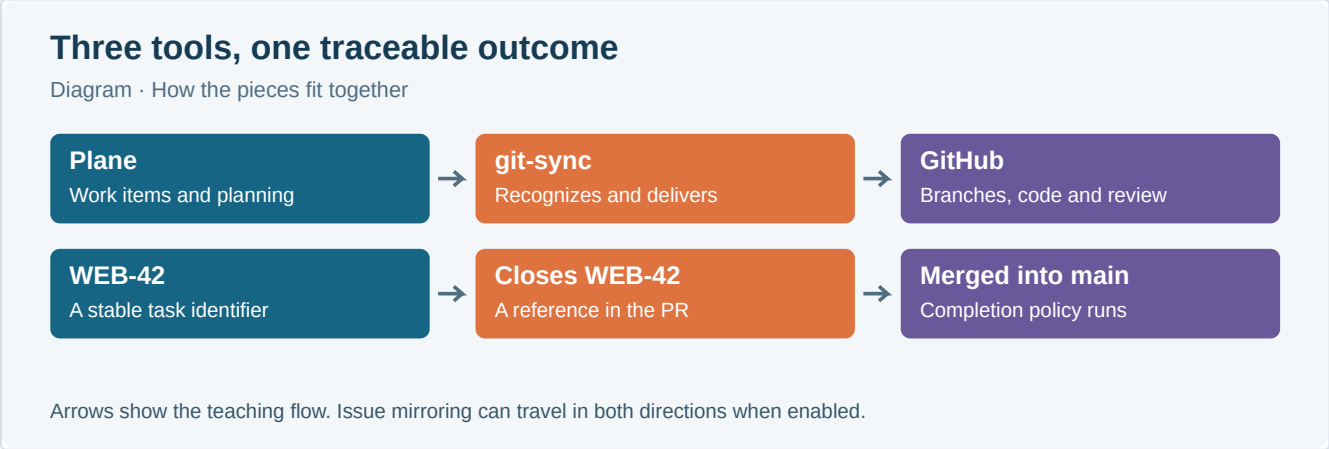


Diagram: Plane holds tasks, GitHub holds code, and git-sync connects them through identifiers and events.

1.1 The five things to recognize

A workspace is the broad team area you join. A project is a more focused area inside it, such as Website. A work item belongs to a project. A state describes where that item is in the workflow. A view controls which items you see and how they are arranged. These ideas are separate: changing from a list to a board does not move work into another project, and selecting a filter does not change the items it hides.

The fictional team in this book is Acme. Alex Developer implements the Website change and Sam Reviewer checks it. Names and example addresses are fictional. The captured Plane workspace has been given an Acme display alias in the screenshots; its actual test address and unrelated test content are hidden. GitHub screenshots show the authorized demonstration repository. The capture notes explain the environment and every cosmetic redaction.

> Example: “WEB-42 is In Progress, assigned to Alex, in the October cycle” gives four different pieces of information. The identifier selects the record. Its state describes workflow. Its assignee describes responsibility. Its cycle describes a planning window.

1.2 How to use the book

Read the first three chapters before changing work. Chapter 4 is the reference for a single work item. Chapters 5 and 6 explain how to organize many items and plan delivery. Chapter 7 covers staying informed. Chapters 8–11 explain the GitHub connection and exact automation rules. Chapter 12 walks through a complete change. The remaining chapters provide suggested conventions, troubleshooting, answers and a printable reference.

Numbered steps give one action at a time. Orange numbered markers in a screenshot point to the controls discussed in its caption. A screenshot is evidence of the captured interface; it does not mean every account has the same permissions. Some actions are available to project or workspace administrators. If your menu has fewer entries, first check your role and project membership.

A **Tip** offers a useful habit. A **Warning** explains a consequence to consider before proceeding. An **Example** gives sample wording or a teaching scenario. Suggested team habits are explicitly identified as suggestions; they are not built-in requirements imposed by Plane.

The PDF contents and bookmarks are clickable. You can also search for an identifier, a button label or an error message. When you need a quick answer during work, start with the troubleshooting table or the last page. Return to the detailed chapter when the underlying idea is unfamiliar.



Diagram: join, find, read, connect, practise and verify form the first-day journey.

1.3 What administrators have already arranged

An administrator has installed Plane, configured sign-in, deployed git-sync, registered the GitHub App and arranged repository access. Your project also needs a repository mapping and the appropriate policies. Your personal GitHub connection is an additional step: it identifies you as a member and can authorize writes as you. It does not install the App or give you access to a repository you otherwise cannot read.

If installation or server configuration is your job, use the administrator manual and its user guide. This handbook concentrates on everyday member work. Do not copy a server token into a work item, a screenshot, a commit or a support message.

1.4 Your first day: one-page checklist

Use this page with a teammate. A checked box means you have verified the result, rather than merely opened a menu.

- [ ] Accept the intended workspace invitation.
- [ ] Sign in with your own Plane account.
- [ ] Set your name and recognizable display name.
- [ ] Check your timezone in Preferences.
- [ ] Choose comfortable theme and notification settings.
- [ ] Join the project containing your team’s work.
- [ ] Find Home, Your work, Drafts and the inbox.
- [ ] Open the command palette and shortcuts dialog once.
- [ ] Read one assigned work item completely.
- [ ] Identify its state, priority, owner and blockers.
- [ ] Open your personal GitHub account settings for the workspace.
- [ ] Connect your own GitHub account if the team requires it.
- [ ] Check that Development shows the intended mapped repository.
- [ ] Practise the branch, commit and PR loop in chapter 12.
- [ ] Confirm that the merged PR and completed item agree.

Ask your onboarding teammate which project view is the team’s source for selecting work, which state means ready for review, and whether a PR should finish or only contribute to its referenced item. These are team decisions. Write the answers somewhere shared so the next developer has the same starting point.

> Tip: Keep `WEB-42`, its Plane link, the repository name and your first branch name together in a temporary note. The identifiers help you recover your place when several tabs are open.

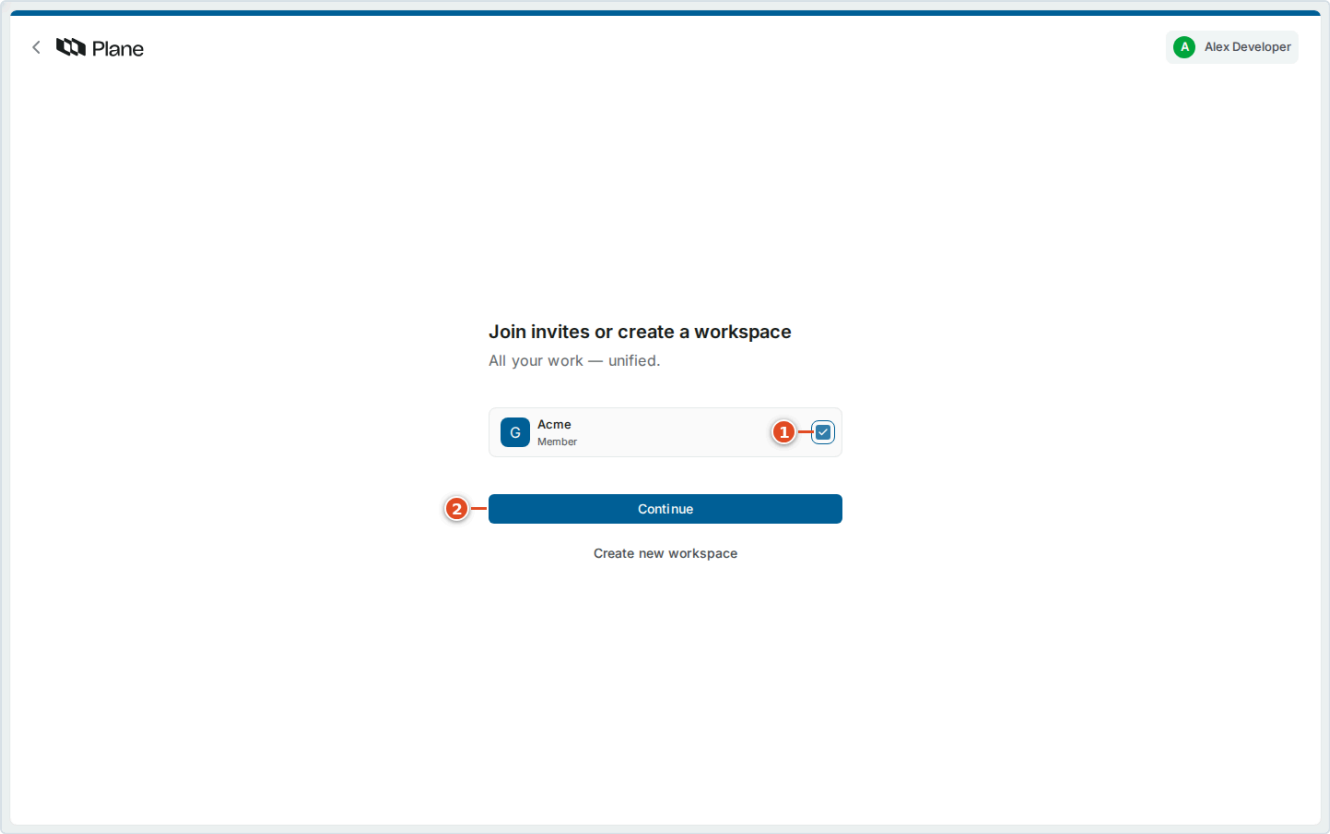
02. Getting in and making Plane yours

An invitation gives your account membership in a workspace. Signing in proves which account you are using. Joining a project gives you a place in that project’s work. These can be separate steps, so successfully signing in does not by itself mean every project is already in your sidebar.

2.1 Accept the right invitation

Your team may send an invitation link by email or tell you to open Plane and find the pending invitation. Use the address provided by your administrator. The examples use `plane.example.com`. Check that you are joining the intended team before accepting. If you have accounts at more than one organization, confirm the email shown in the account menu.

1. Open the invitation link supplied by your team.
2. Sign in or create your own account when Plane asks.
3. Select the checkbox for the intended workspace.
4. Select **Continue**.



Accept the invitation: (1) select the workspace checkbox; (2) Continue.

The captured first-time invitation screen includes a workspace name and a membership role. Select the actual checkbox; clicking nearby text is not a reliable substitute. The Continue button stays disabled until an invitation is selected. An existing account may enter through Workspace invites instead of the first-time onboarding screen.

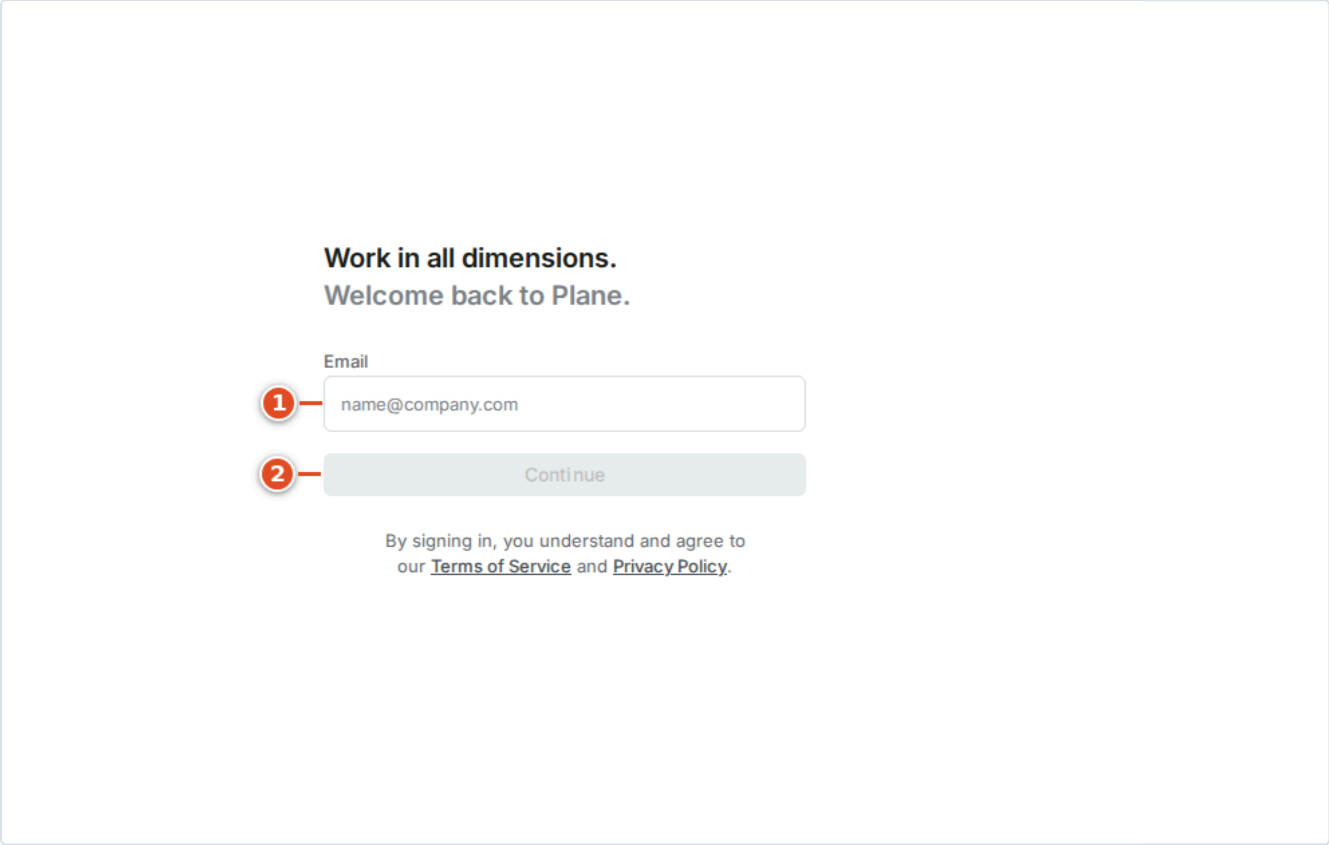
If there is no invitation, do not create a second workspace merely to get past onboarding. Ask the inviter to check the exact email address and pending invitation. An invitation sent to an alias may not match the account you have signed into. If an old link fails, return to the workspace invitations screen or ask for a fresh invitation.

2.2 Sign in

The demonstration instance uses email and password. Your administrator may enable another sign-in provider. Use the methods actually displayed on your sign-in page. Plane’s own account sign-in and the later git-sync GitHub authorization serve different purposes; the presence of one does not establish the other.

1. Open your team’s Plane address.
2. Enter your account email.
3. Select **Continue**.
4. Enter your password when prompted.

5. Submit the sign-in form.



Sign in: (1) email field; (2) Continue. The next screen asks for a password.

Use the password recovery control offered by your instance if you cannot sign in. Do not send your password to a teammate. If a recovery message never arrives, ask an administrator whether outgoing email is configured. The handbook instance has no SMTP configured, so delivery of recovery or invitation mail was not verified. An on-screen success message alone is not evidence that a mailbox received anything.

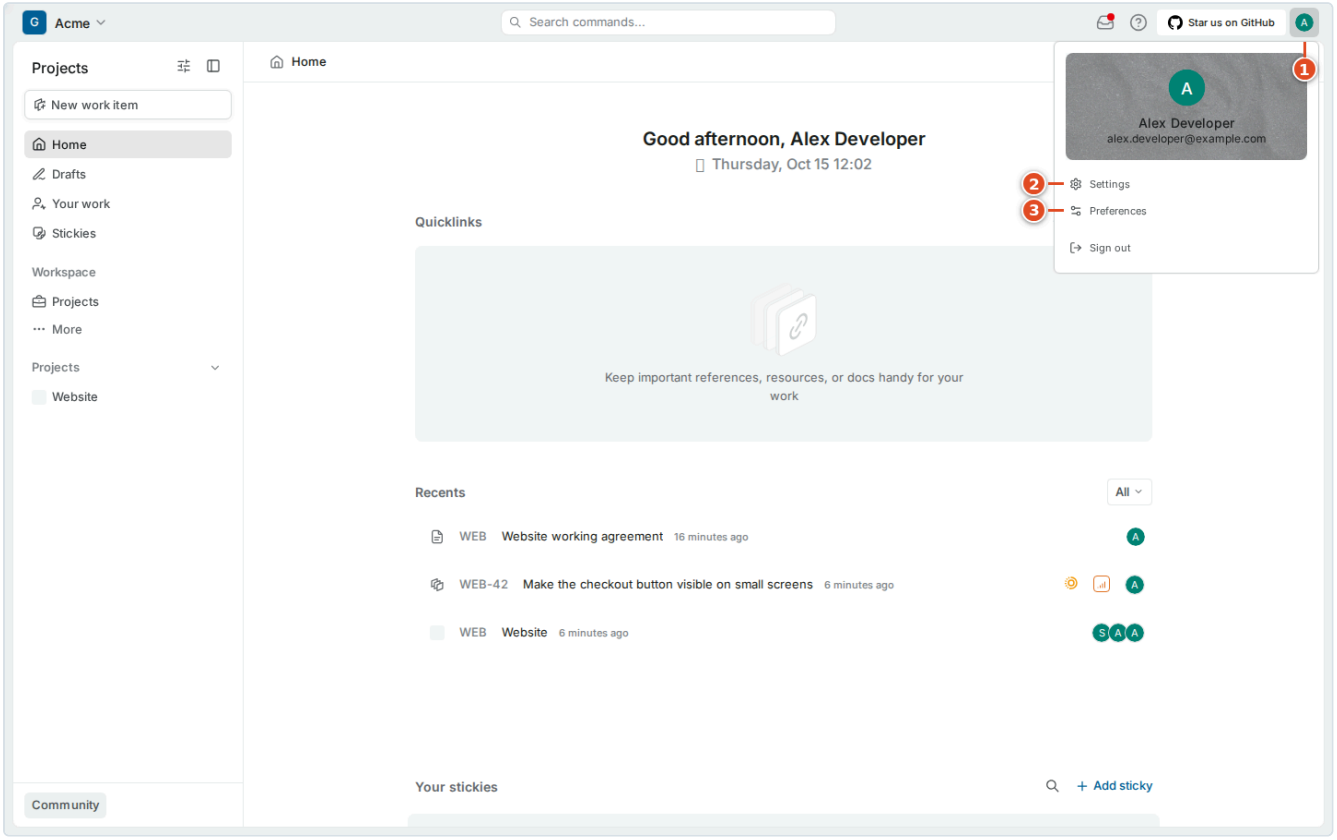
> Warning: A service account used by git-sync is not your everyday account. The integration deliberately ignores some changes made by its service account to prevent echoes. Using it for normal work can make your edits appear to be ignored.

2.3 Set your profile

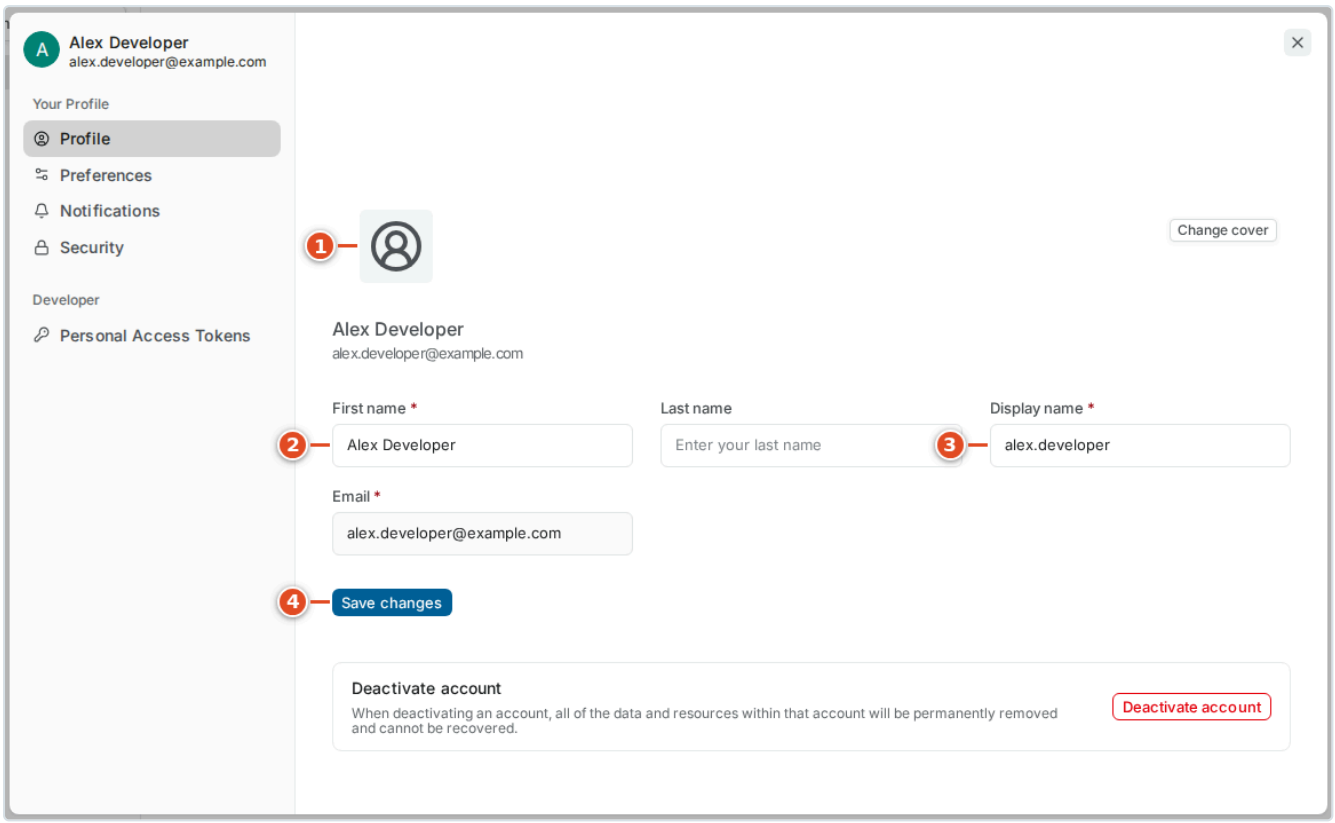
Your profile helps teammates recognize who is working and who is commenting. Your full name is the human-readable identity. Your display name is the shorter identity shown in parts of the interface. An avatar is a small picture or initial marker used beside assignments and comments. A consistent display name is more useful than one that changes with each task.

1. Select your avatar in the upper-right corner.
2. Select **Settings**.

3. Select **Profile** if another settings section is open.
4. Update **First name**.
5. Update **Last name** if you use one.
6. Update **Display name**.
7. Select **Save changes**.



Personal menu: (1) avatar; (2) Settings; (3) Preferences.



Profile: (1) avatar; (2) name; (3) display name; (4) Save changes.

The profile editor is a modal over the current page in this version. You can close it and return to the work underneath. Use the avatar control if you want to upload a picture; choose a simple image you are comfortable showing to teammates. The captured demonstration uses initials. Avoid uploading customer information or credentials as a profile image.

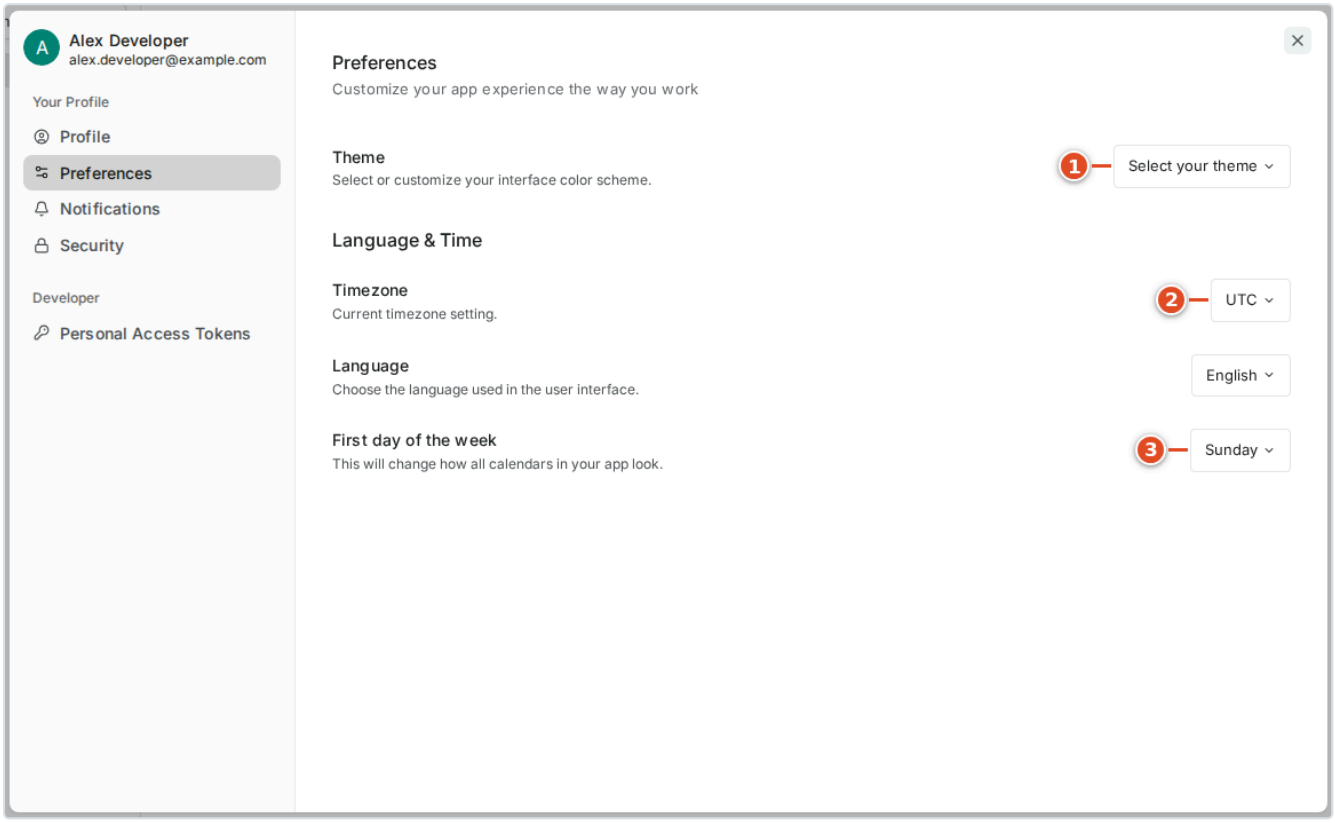
The email field is also visible here. Changing an account email can affect future invitations and how you identify the account. Ask your administrator about the team’s account policy before using that field for a company email change. Do not create multiple accounts to solve a display-name problem.

2.4 Check timezone, theme and calendar preferences

Your timezone controls how time-based information is presented. A due date and a notification time are easier to interpret when your settings match where you work. Your theme changes the interface appearance; it does not change a work item’s state or another member’s preferences.

1. Open your avatar menu.
2. Select **Preferences**.
3. Open the theme selector.
4. Choose the theme you prefer.

5. Open the timezone selector.
6. Choose your working timezone.
7. Set the first day of the week if the default differs from your calendar habit.



Preferences: (1) theme; (2) timezone; (3) first day of the week.

The same area includes language preferences. Select a language you can comfortably use while following team instructions. If teammates share screenshots, control positions may be the same even when labels differ. The instructions in this book use the English labels captured in the demonstration.

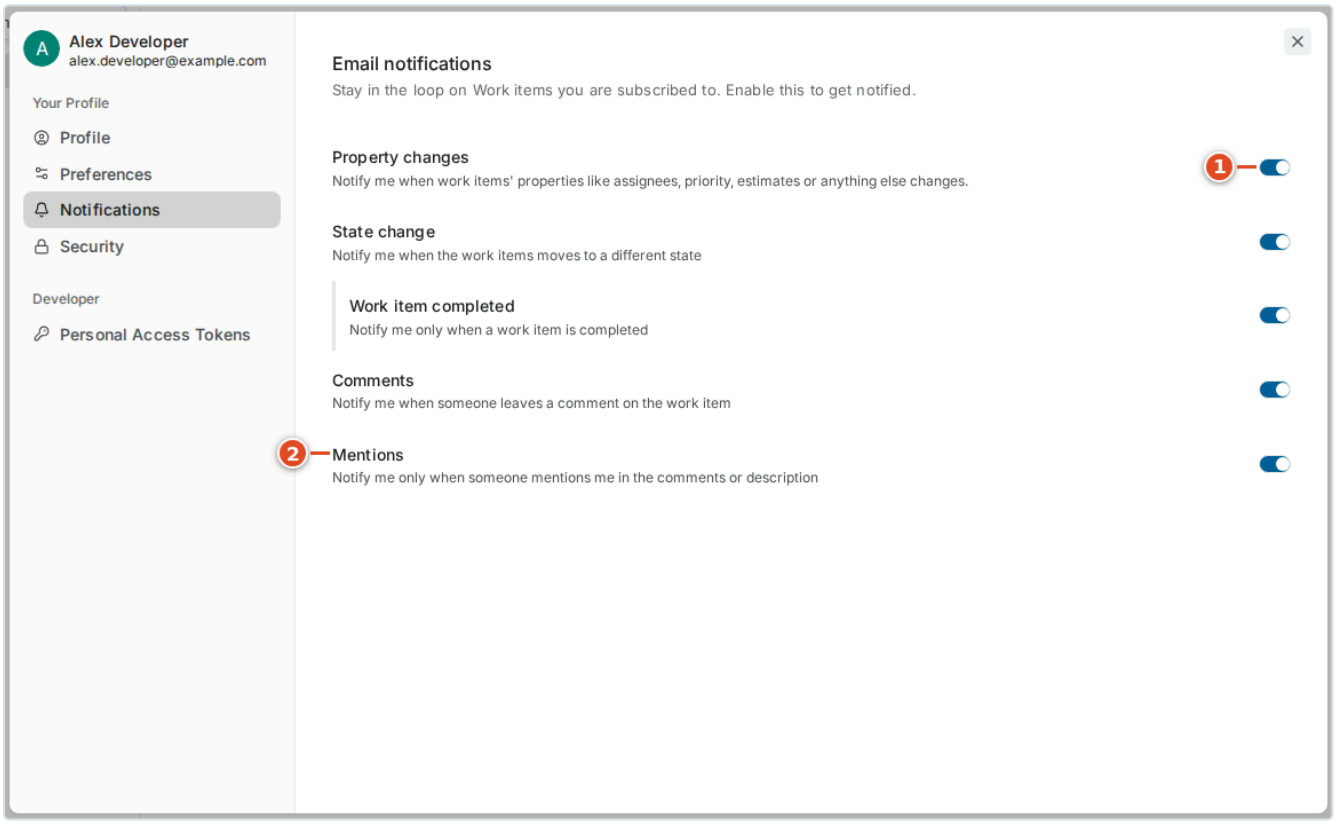
> Tip: When a meeting or deadline looks a few hours wrong, compare timezone settings before changing the work item’s date. A display setting and a planning mistake need different fixes.

2.5 Set notification preferences deliberately

A notification is a message that something relevant changed. A subscription says that you want updates about a particular work item. Email notification preferences control which of those updates should reach your mailbox. Turning off an email category is different from removing a subscription or changing an assignment.

1. Open personal **Settings**.
2. Select **Notifications**.

3. Review **Property changes**.
4. Review **State change**.
5. Review **Work item completed**.
6. Review **Comments**.
7. Review **Mentions**.



Email notifications: (1) property-change switch; (2) Mentions category and its switch.

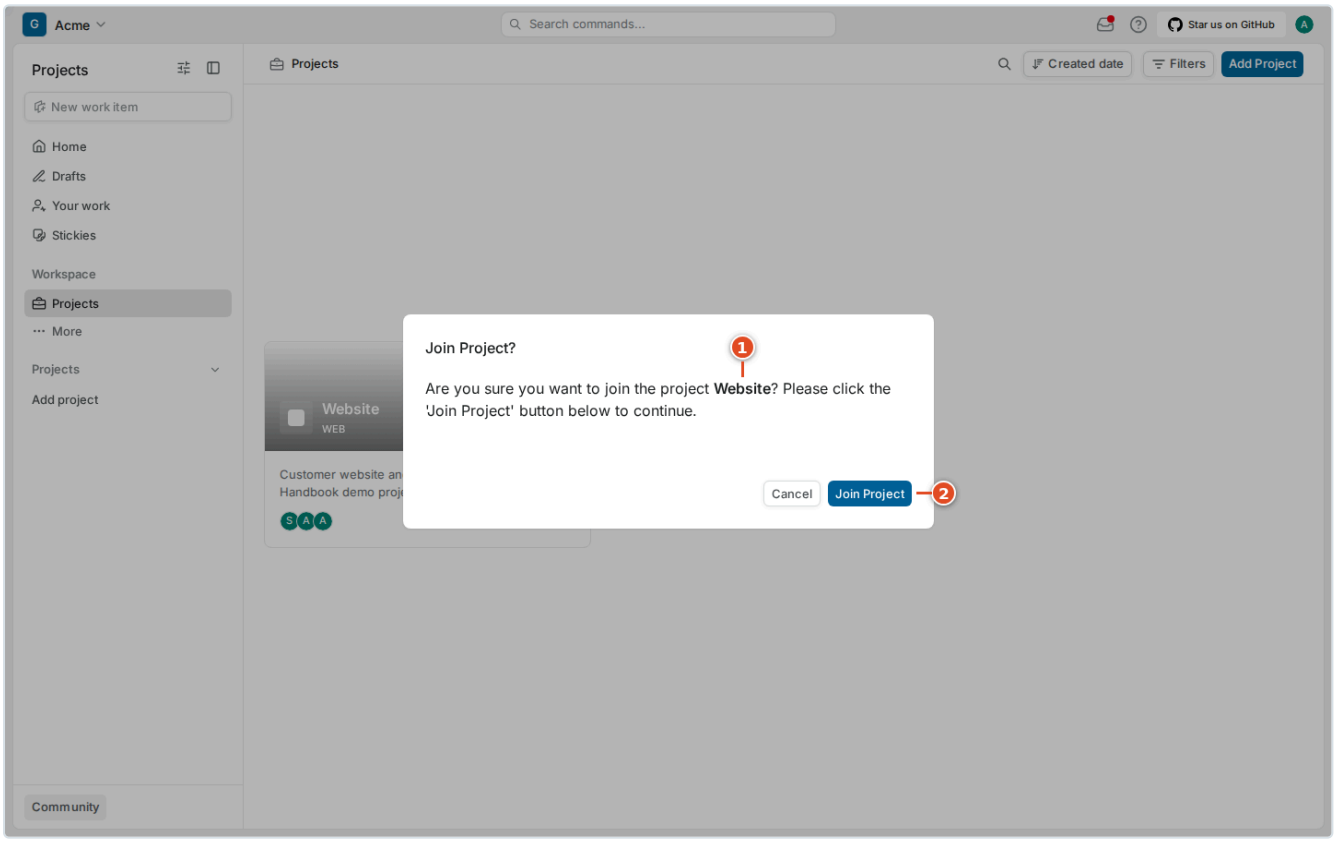
Choose settings that help you notice requests without making every small edit equally urgent. A useful starting suggestion is to keep direct mentions and comments visible, then adjust property-change email after experiencing a few days of normal activity. Your team may have its own expectations about checking the inbox during working hours.

Email requires a functioning mail configuration on the server. If an expected message does not arrive, first confirm the change appears in Plane itself. Then check your category preferences and mailbox filtering. Ask an administrator about mail delivery after those checks. Do not change instance settings yourself to troubleshoot ordinary member notifications.

2.6 Join the Website project

The projects page lists projects you are allowed to discover. A visible project can still have a **Join** action because you are not yet a member. Private projects may require an explicit invitation from a project administrator.

1. Select **Projects** in the workspace navigation.
2. Find the project your teammate named.
3. Select its **Join** action.
4. Check the name in the confirmation dialog.
5. Select **Join Project**.
6. Open **Work items** under the project in the sidebar.



Join Website: (1) confirm the project name; (2) Join Project.

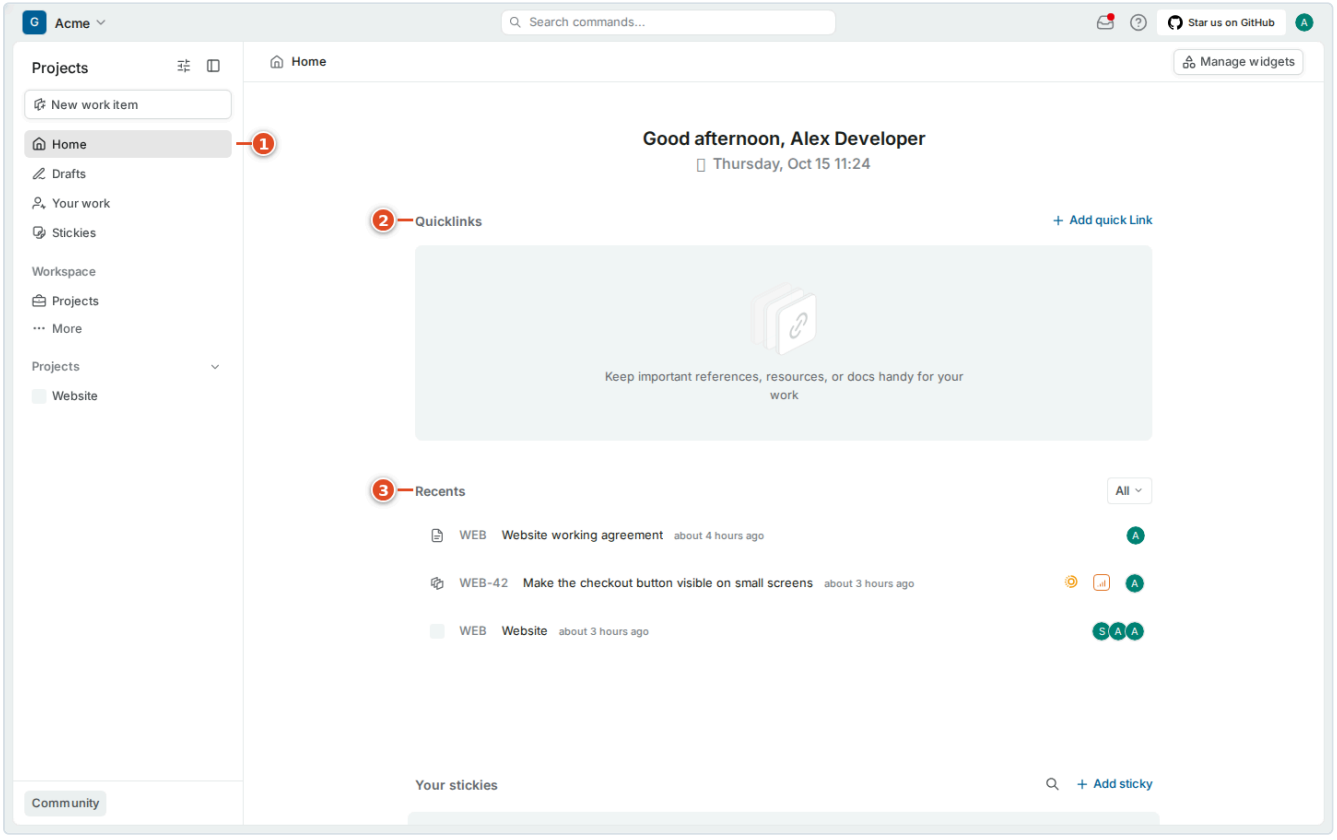
Joining the correct workspace does not mean you should join every visible project. Start with the area you actually work in. If Website is missing, check the workspace switcher before asking for more permissions. If it is present but you cannot join, tell the project administrator what you see and which account you are using. Avoid sending an entire account settings screenshot when a project name and an error message will do.

03. Finding your way around Plane

Plane has two useful scales of navigation. Workspace navigation takes you to places across the team. Project navigation takes you to the work, plans and documents within one project. Recognizing which scale you are using prevents a common first-day mistake: looking for an item in a view that belongs to another project.

3.1 Read the screen from the outside in

The top-left workspace switcher identifies the current team area. The sidebar holds Home, Drafts, Your work and the workspace projects list. Expanding a project reveals its enabled features. The main area shows the selected page. The upper-right controls include the inbox, help and your avatar.



Home: (1) sidebar Home; (2) Quicklinks; (3) Recents.

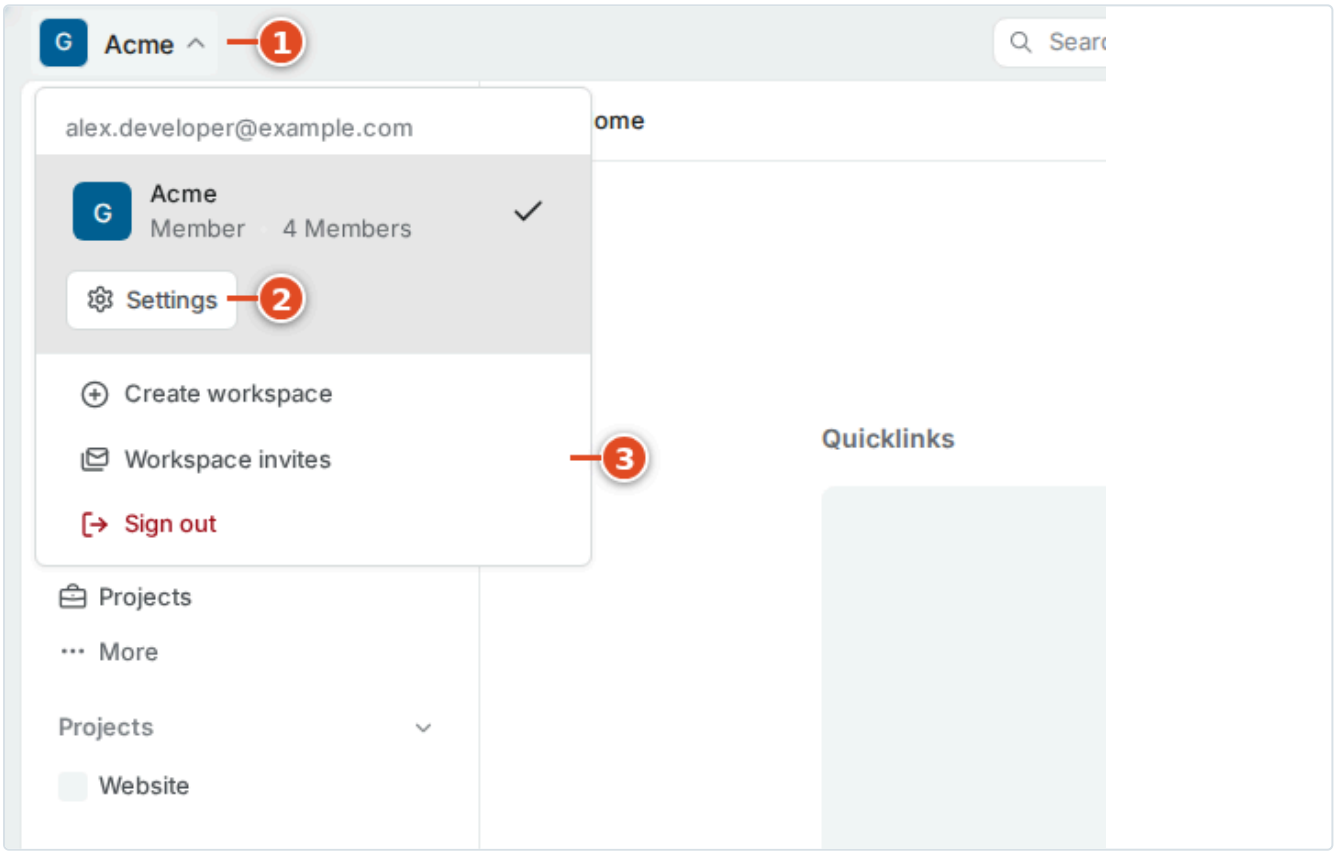
The exact widgets on Home can differ from a teammate’s. Home provides a useful starting point and recent destinations. It is not a guarantee that every unfinished task is listed there. Use Your work or a project view when you need a deliberate list of assigned or filtered work.

1. Check the workspace name at the top-left.
2. Select **Home** in the sidebar.
3. Look at the recent work area.
4. Open a recent item only after checking its project and identifier.

Quicklinks let you keep useful destinations nearby. A sticky is a personal note for an idea or reminder. A shared work item should hold the durable task and its acceptance criteria. Avoid keeping the only copy of a team decision in a personal note where other developers cannot discover it.

3.2 Switch workspace carefully

1. Select the workspace switcher.
2. Check the currently selected workspace.
3. Choose the workspace you need.



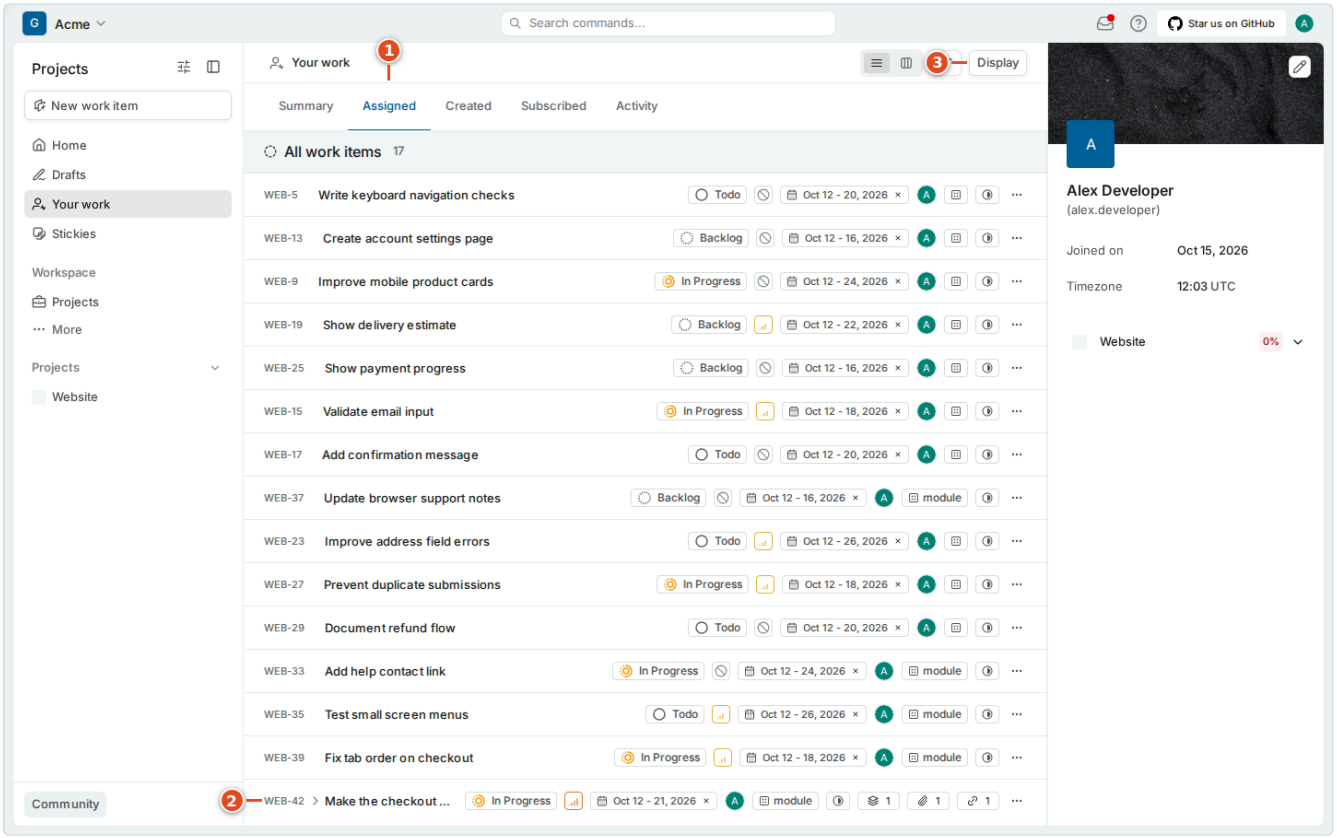
Workspace switcher: (1) current workspace control; (2) Settings; (3) Workspace invites.

The menu can also expose workspace settings and invitations. Creating a workspace makes a different team area; it does not repair a missing project invitation. If you accidentally enter the wrong workspace, switch back before creating a work item. An identifier such as `WEB-42` is interpreted in the context of the project's workspace and configured mappings, not as a globally unique public ticket number across every Plane installation.

3.3 Find Your work and the project list

Your work is a personal work area. It helps you inspect work relevant to you instead of scanning all project items. Its visible categories and filters are useful for separating assigned, created or subscribed activity where provided by the page. A filtered personal list can hide a task, so use the project when you need to verify that a record exists.

1. Select **Your work** in the sidebar.
2. Review the selected category.
3. Check the filters before concluding that nothing is assigned.
4. Open the item you want to inspect.



Your work: (1) Assigned category; (2) matching item WEB-42; (3) Display controls.

1. Select **Projects** under Workspace.
2. Find **Website**.
3. Open the project.

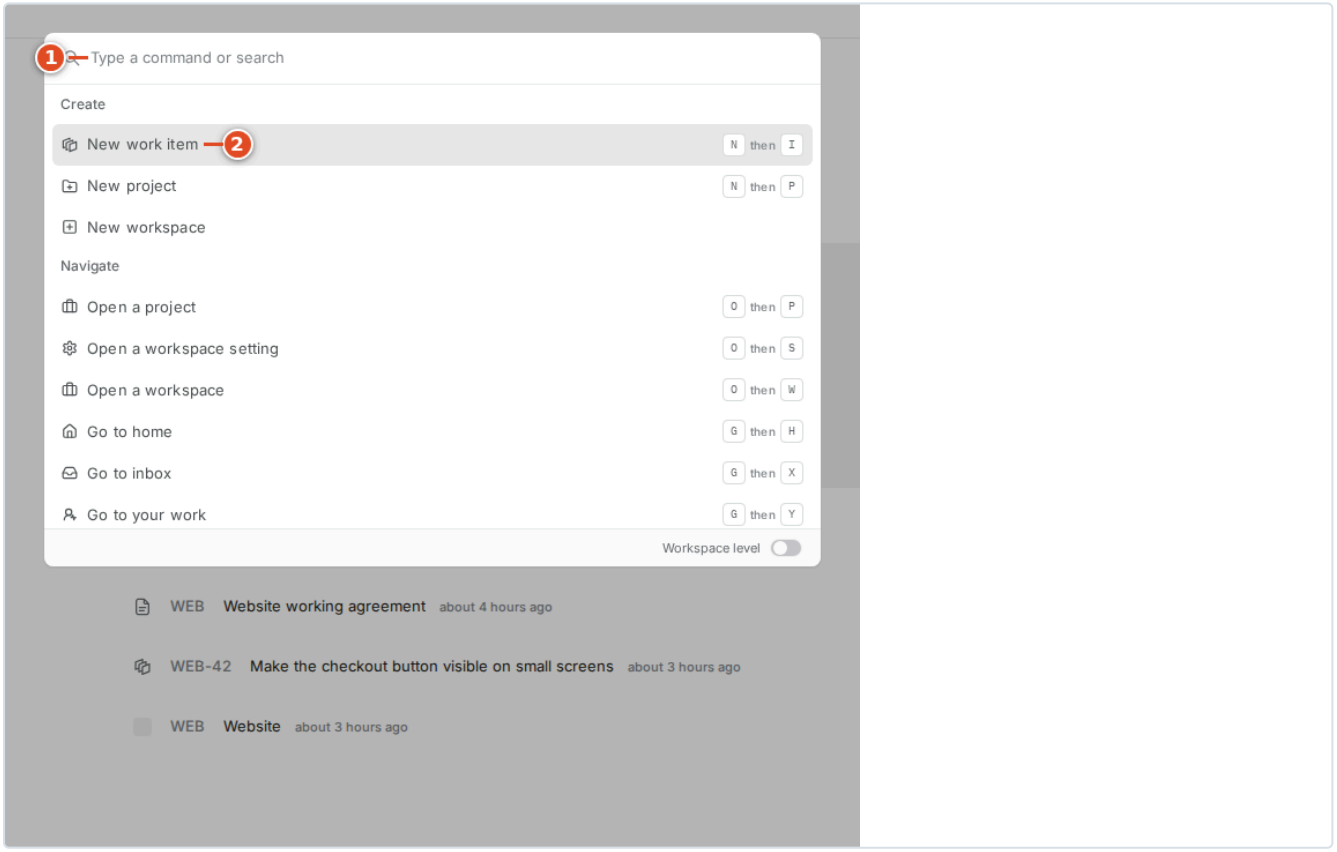
4. Select **Work items** in its expanded navigation.

The project navigation in this handbook includes Work items, Cycles, Modules, Views, Pages and Intake. A project administrator can disable optional features, so a missing entry may be a project configuration choice. Ask whether the team uses that feature before assuming the installation is broken. The Community button and billing pages are not evidence that a paid feature is included in your current edition.

3.4 Use the command palette

The command palette is a searchable menu for finding destinations and actions from the keyboard. It is especially helpful when you know the name of something but cannot remember which menu contains it.

1. Press **Ctrl + K** on Windows or Linux, or **Cmd + K** on macOS.
2. Type part of the action or item you want.
3. Read the highlighted result.
4. Press **Enter** to select it.
5. Press **Escape** to close the palette when you do not want to act.

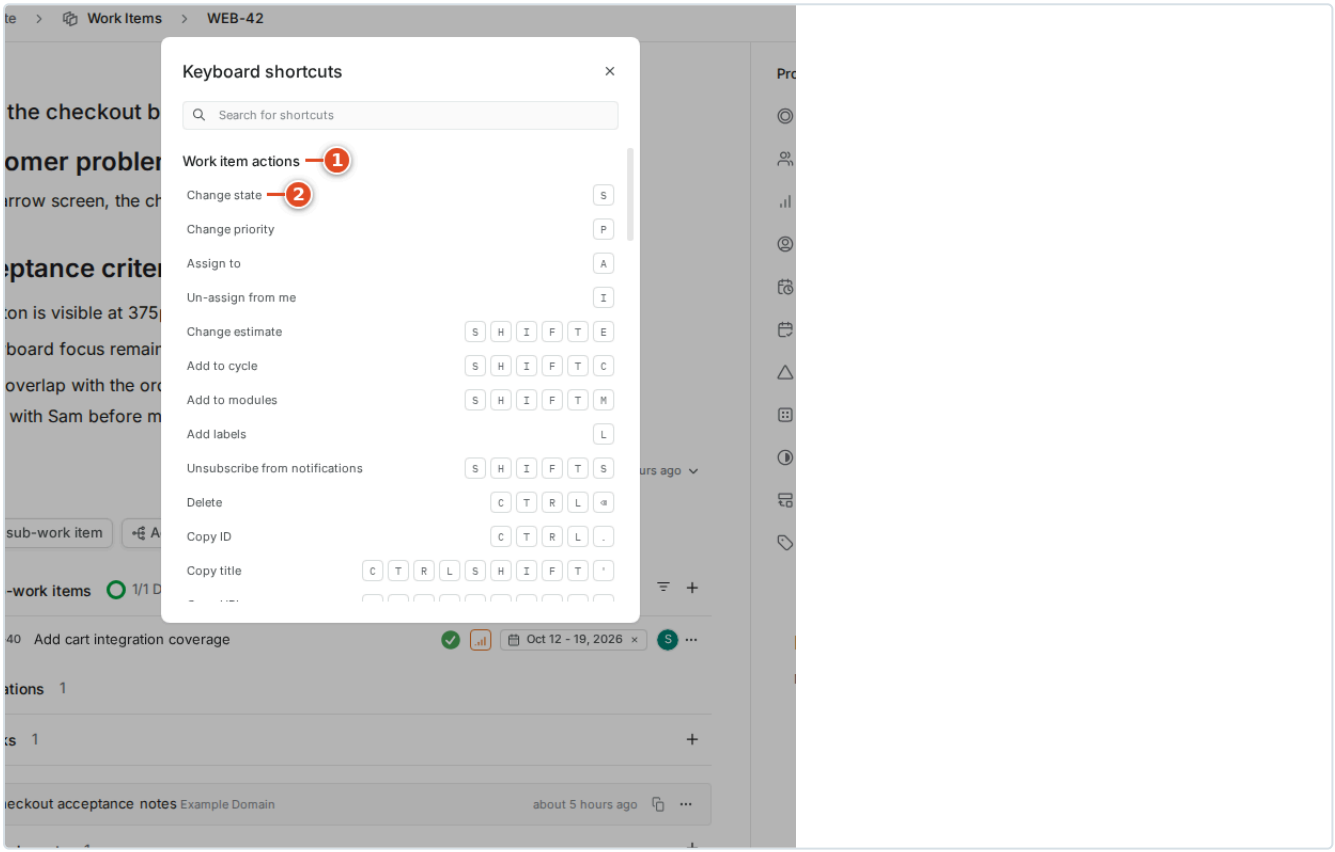


Command palette: (1) search field; (2) a creation action in the results.

A palette action can change data, not merely navigate. Read its label before pressing Enter. If you are typing in a description or code block, close or leave that editor before using a global shortcut. Otherwise the editor may interpret your keystroke as formatting or text input.

3.5 Learn shortcuts from Plane’s own dialog

The shortcuts dialog is the reference for your running version and operating system. Open it through the help control in the top bar. The table below is transcribed from the running CE dialog, rather than borrowed from a different edition. The interface displays a Command symbol on macOS where appropriate; use the Control equivalent shown on your system.



Keyboard shortcuts: (1) Work item actions category; (2) action labels with their adjacent keys.

The dialog groups its shortcuts by context. “Then” means release the first key before pressing the next one. A plus sign means hold the modifier while pressing the other key. The dialog lists some identical combinations in different contexts; the current page determines which action applies.

Context	Action	Keys shown on Linux
Work item	Change state	S
Work item	Change priority	P

Context	Action	Keys shown on Linux
Work item	Assign to	A
Work item	Un-assign from me	I
Work item	Change estimate	Shift + E
Work item	Add to cycle	Shift + C
Work item	Add to modules	Shift + M
Work item	Add labels	L
Work item	Unsubscribe from notifications	Shift + S
Work item	Delete	Ctrl + Backspace
Work item	Copy ID	Ctrl + .
Work item	Copy title	Ctrl + Shift + ’
Work item	Copy URL	Ctrl + Shift + ,
Cycle	Add to favorites	Shift + F
Cycle	Copy URL	Ctrl + Shift + ,
Module	Add/remove members	M
Module	Change status	S
Module	Add to favorites	Shift + F
Module	Copy URL	Ctrl + Shift + ,
Page	Lock	Shift + L
Page	Make public	Shift + A
Page	Archive	Shift + R
Page	Add to favorites	Shift + F
Page	Copy URL	Ctrl + Shift + ,

Context	Action	Keys shown on Linux
Create	New work item	N then I
Create	New page	N then D
Create	New view	N then V
Create	New cycle	N then C
Create	New module	N then M
Create	New project	N then P
Open	A cycle	O then C
Open	A module	O then M
Open	A project view	O then V
Open	A project setting	O then S
Open	A project	O then P
Open	A workspace setting	O then S
Open	A workspace	O then W
Go to	Home	G then H
Go to	Inbox	G then X
Go to	Your work	G then Y
Go to	Work items	G then I
Go to	Pages	G then D
Go to	Cycles	G then C
Go to	Modules	G then M
Go to	Project views	G then V
Go to	Intake	G then K
Go to	Project settings	G then S

Context	Action	Keys shown on Linux
Go to	Project archives	G then R
Go to	Workspace analytics	G then A
Go to	Workspace settings	G then S
Go to	Workspace drafts	G then J
Go to	Workspace archives	G then R
Go to	Projects list	G then P
Miscellaneous	Toggle app sidebar	Ctrl + B
Miscellaneous	Copy current page URL	Ctrl + Shift + C
Miscellaneous	Focus search input	Ctrl + F
Help	Open keyboard shortcuts	Ctrl + /

These are the displayed bindings, including potentially surprising overlaps. A Page action appearing in the dialog does not establish that your role can publish a particular page. Use the visible permission controls and your team’s sharing policy. The command palette was also opened with Ctrl + K during capture; that launcher is described separately above.

Do not memorize the entire table on day one. Begin with the palette, closing a dialog and creating a work item. Add navigation shortcuts after you recognize the destinations. A shortcut can be convenient while still depending on your permissions: it does not grant access to a private project or enable an administrator-only action.

> Tip: If a shortcut seems ineffective, check whether a text editor, browser address field or dialog has focus. Mouse navigation is a good way to verify the destination before troubleshooting the keyboard combination.

3.6 Keep a reliable route back

A direct work item link is usually the most useful thing to share in a discussion. It selects one record rather than describing the clicks another person must repeat. Include the identifier alongside the link so people can recognize it in a message, commit or PR.

1. Open the work item.
2. Use its copy-link control.

3. Paste the link into your intended team discussion.
4. Check that the pasted label or surrounding text includes the correct identifier.

A link does not override permissions. If a teammate receives a not-found screen, ask them to check their workspace and project membership. Do not make a project public as a quick workaround for a single missing invitation. The copied address identifies your team’s server; avoid pasting private project links into public GitHub discussions unless your team’s policy permits that disclosure.

04. Work items in depth

A work item is a shared record of an outcome. Its description explains what needs to change. Its properties help people choose, organize and track that work. Its comments preserve the decisions made along the way. You can edit an item from several layouts, but those controls all update the same underlying record.

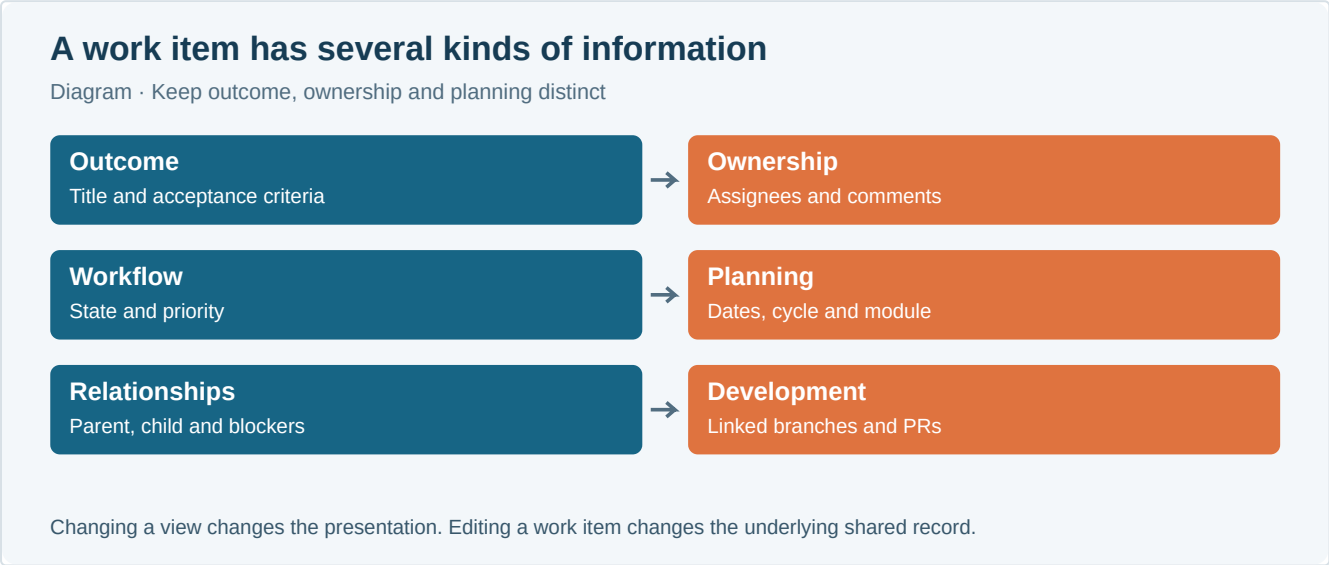
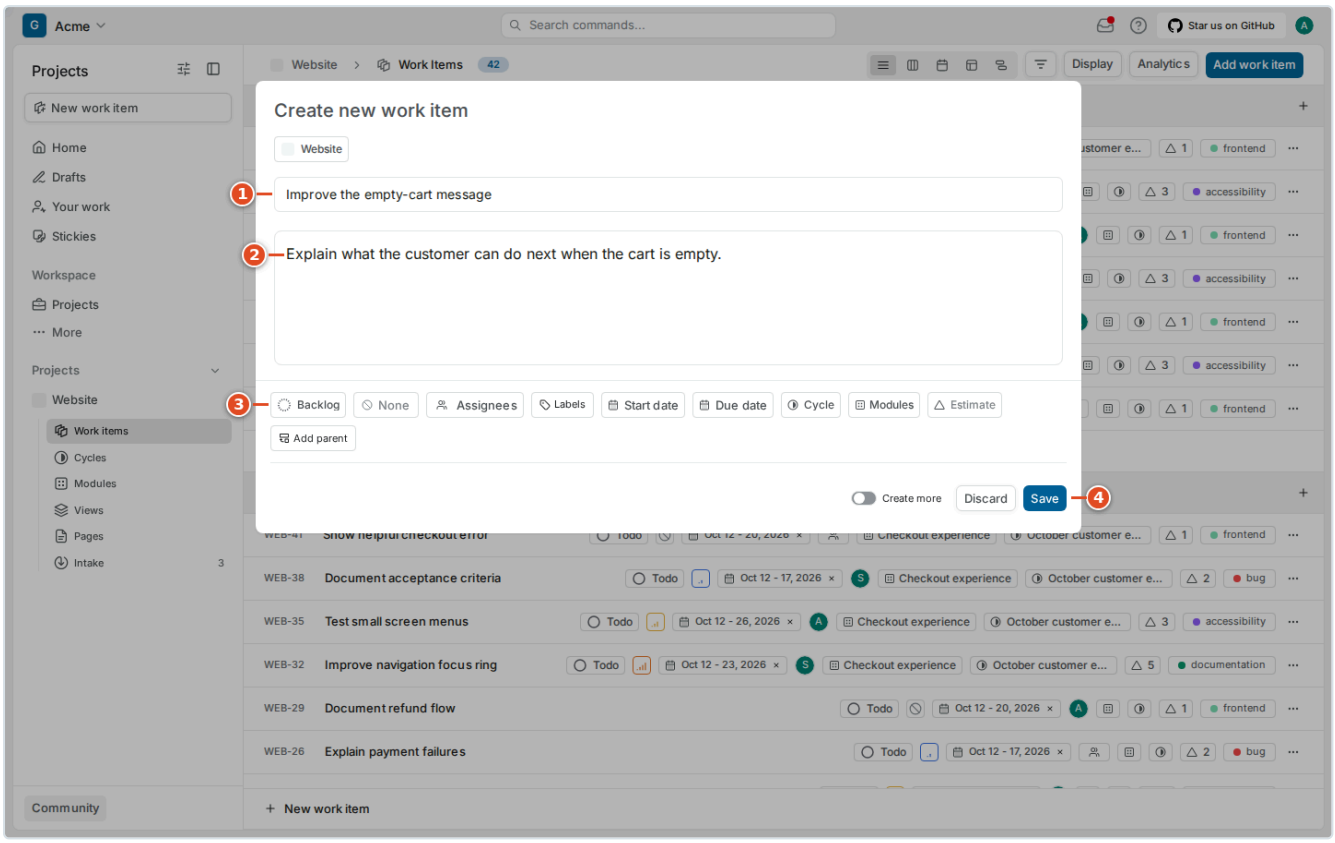


Diagram: outcome, ownership, workflow, planning, relationships and development are separate parts of a work item.

4.1 Create an item with enough context

Before creating a new item, search for an existing one. Two records describing the same bug split the discussion and can cause duplicate implementation. If the request already exists, add useful evidence to that record or use a relation to connect a distinct follow-up.

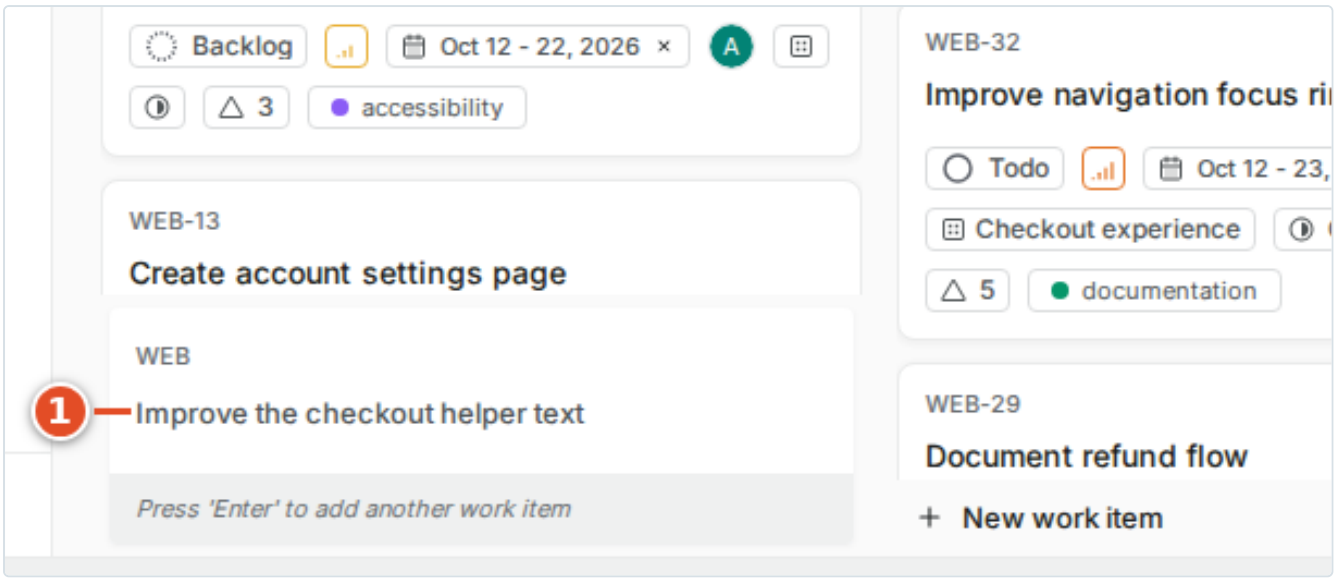
1. Open the intended project.
2. Select **Add work item** in the project header.
3. Check the project selected in the form.
4. Enter a title describing the outcome.
5. Enter the description.
6. Set the initial state if the default is inappropriate.
7. Set the priority agreed with the team.
8. Add the responsible assignee when known.
9. Select **Save**.



Create a work item: (1) title; (2) description; (3) properties; (4) Save.

A useful title is “Keep the checkout button visible on small screens.” It tells the reader what should become true. “Checkout” alone forces everyone to open the description to discover the request. Avoid putting a long troubleshooting diary in the title; keep that evidence in the description and comments.

The quick-add controls in a list or board are useful when you already know the project and group. Use the plus or New work item control in the intended group, enter the title and submit the small form. The chosen group can supply the state. Open the created record afterward to add the context that did not fit the quick form. A fast creation path does not remove the need for acceptance criteria.



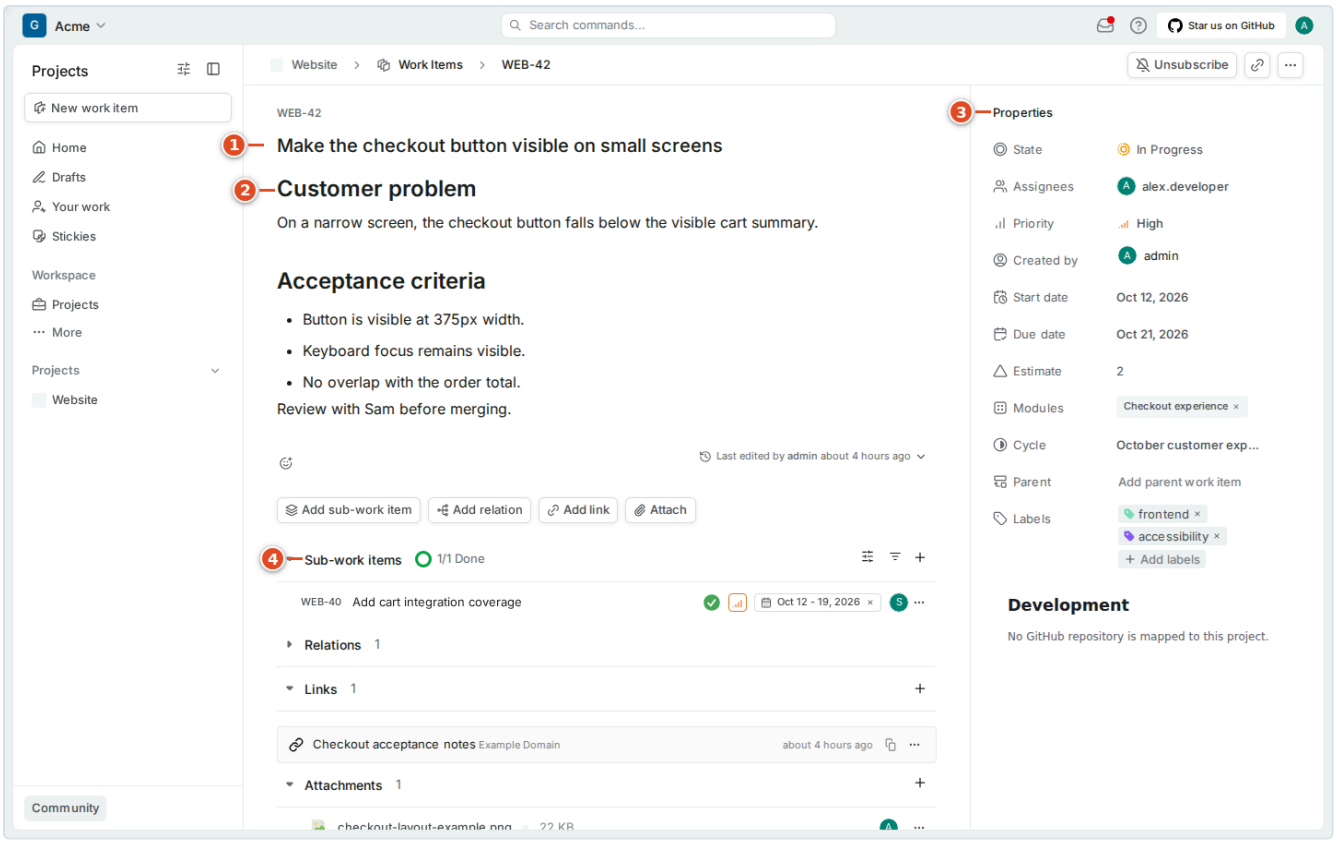
Quick add: (1) inline title input. Enter submits; Escape cancels.

> Example: A small bug description can contain four short sections: what happened, what should happen, how to reproduce it, and how to confirm the fix. For WEB-42, the key evidence is the affected screen width and whether the button remains keyboard accessible.

4.2 Understand the identifier

The project supplies a short identifier such as `WEB`. Plane adds a sequence number to produce `WEB-42`. Use that exact key in discussions and the supported GitHub reference locations. A title is editable prose; an identifier is the compact handle that lets tools and people refer to the same work.

Do not invent an identifier for an unsaved draft. Open the saved item and copy the displayed key. Preserve the hyphen and digits. `WEB_42` and `WEB42` are different strings. Case handling varies by reference location: chapter 9 explains why a lower-case branch key works while some bare lower-case prose does not.

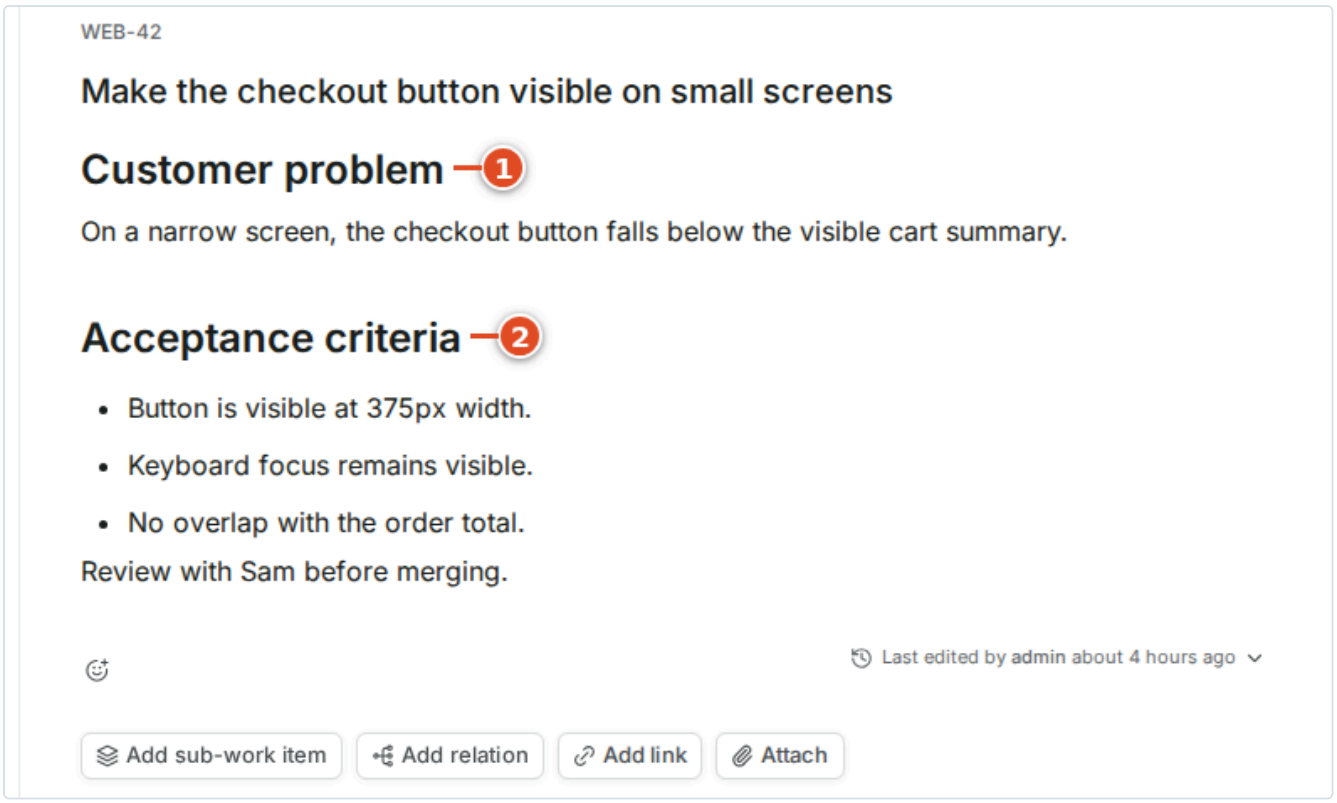


WEB-42: (1) title and identifier; (2) description; (3) properties; (4) sub-work items.

4.3 Write and maintain the description

The description editor supports structured writing. Use headings to separate the customer problem, acceptance criteria and implementation notes. Use a checklist for independently checkable outcomes. Use a code block for code or a literal error message so it stays readable. Use a link for supporting material rather than pasting an entire unrelated document.

1. Open the item.
2. Select the description area.
3. Write the customer problem in plain language.
4. Add concrete acceptance criteria.
5. Add useful reproduction steps or references.
6. Finish editing and wait for the saved indication.



Description: (1) customer problem; (2) acceptance criteria.

Select text to reveal the formatting controls. The editor also offers insertion controls for richer content. Choose the appropriate block rather than imitating a checklist with a paragraph of symbols. A code block should include only the small example needed to explain the issue. Remove credentials, customer details and machine-specific secrets from copied logs before saving them.

An image can explain a visual bug more clearly than several paragraphs. Include enough surrounding interface to identify the problem, and add a sentence explaining what the reader should notice. Use fictional or redacted data when the screenshot might be shared beyond the team. An image of a failure is evidence; it is not itself an acceptance criterion unless the expected result is also described.

A mention identifies a teammate within the editor. Type `@` and choose the intended member from the suggestions when the editor offers them. A typed name that is never selected as a mention is just text. Use mentions to request specific help, such as a decision about expected behavior, and include the question beside the mention. Avoid tagging an entire group for a small property edit.

Keep the description current as decisions change. A comment can explain why the team changed its mind; the description should still tell the next developer which outcome is now expected. If you learn that an acceptance criterion is wrong, resolve that with the owner instead of silently implementing a different task.

4.4 States and state groups

A **state** is a named point in the workflow, such as Todo, In Progress, In Review or Done. A **state group** is the broader classification Plane and automation use. The captured project has Backlog, Unstarted, Started, Completed and Cancelled groups. Its Todo state belongs to Unstarted; both In Progress and In Review belong to Started.

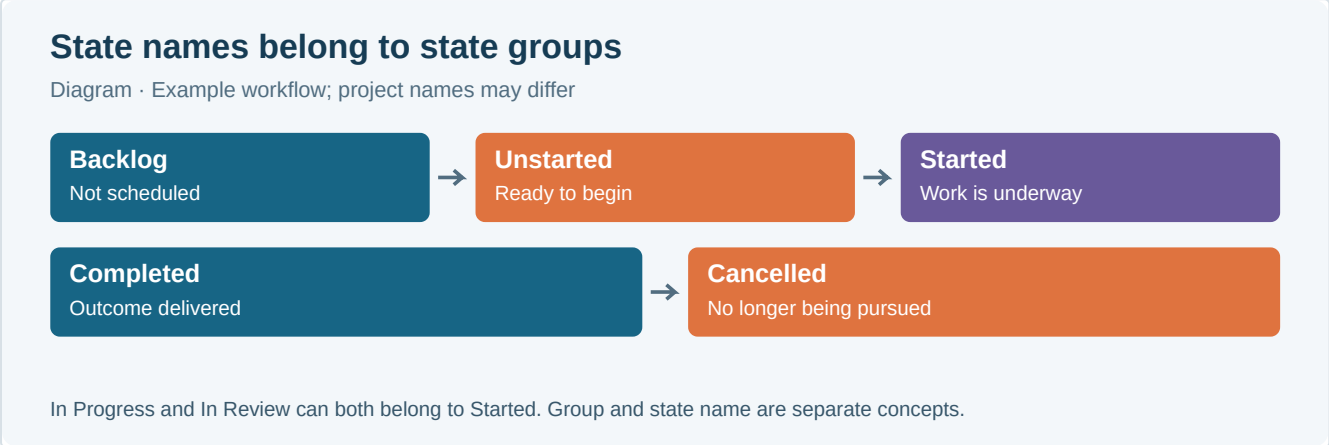
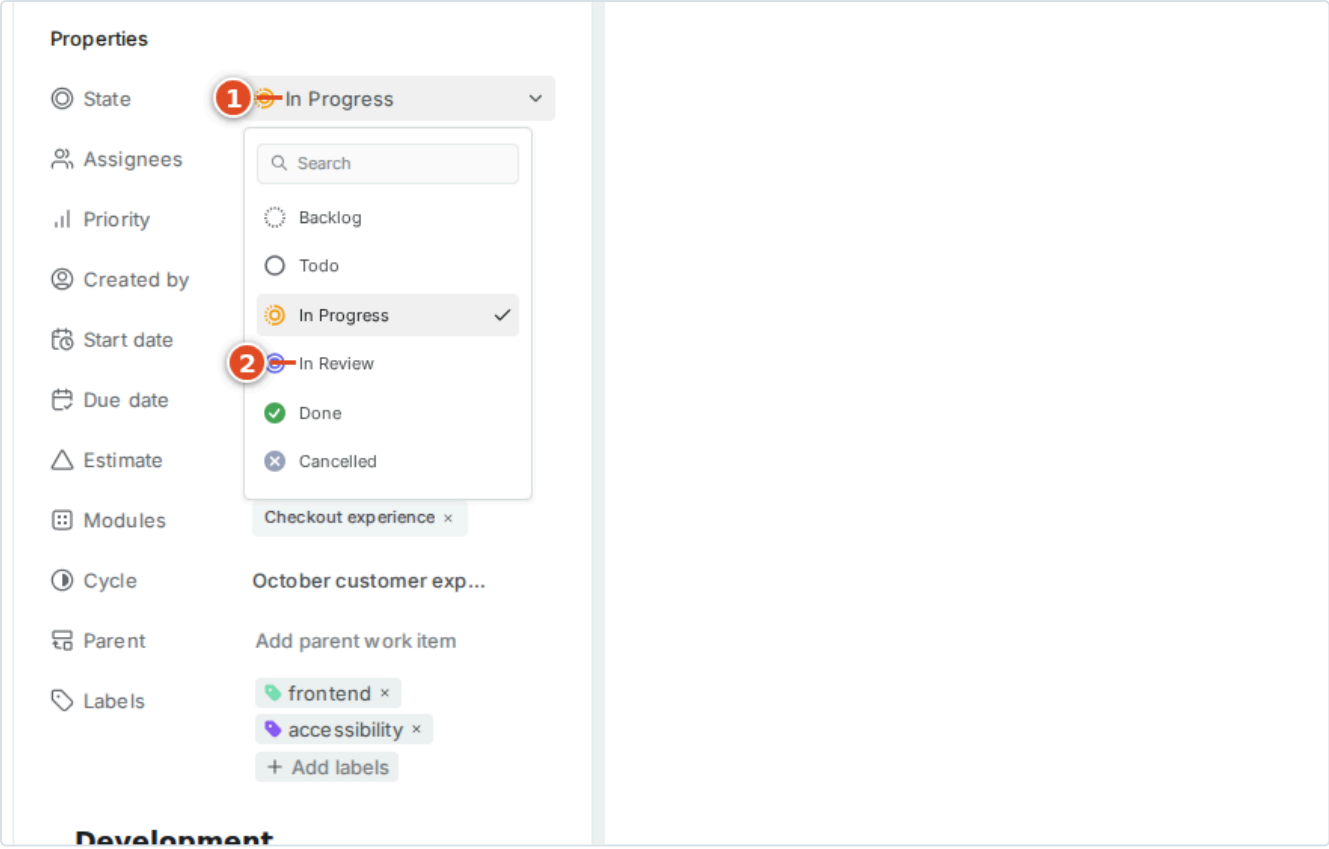


Diagram: state groups classify project-specific state names.

1. Open the **State** property.
2. Read the available project states.
3. Select the state that reflects the actual situation.
4. Confirm the updated value on the item.



State: (1) current state control; (2) In Review among the project choices.

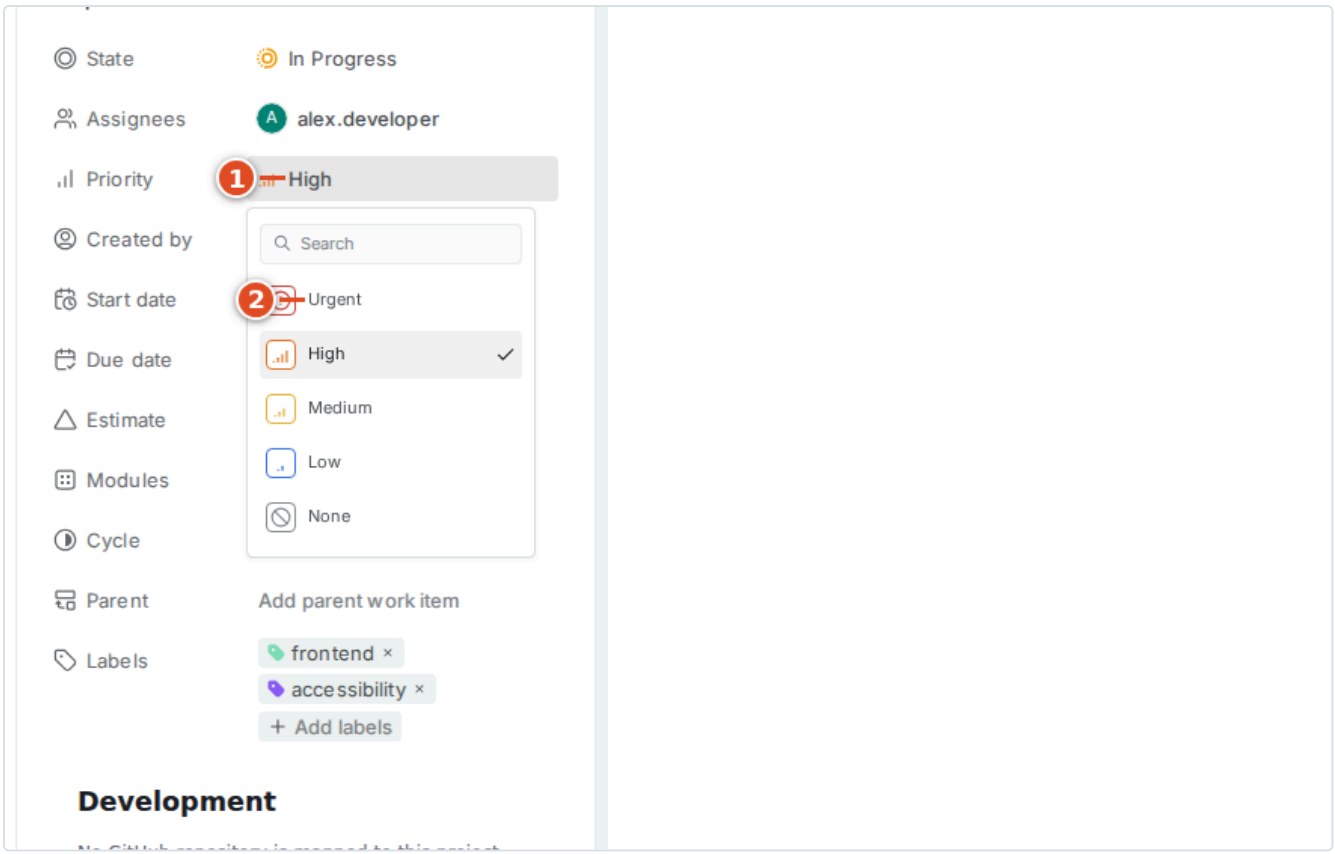
Moving a task to Done means the agreed outcome is complete according to the team's workflow. It is not the same as archiving the record. Cancelled means the team is no longer pursuing the work; use a comment to preserve the reason. A blocked task usually still has a workflow state plus a blocking relation. Do not create a new state for each temporary obstacle without discussing the workflow with the project administrator.

GitHub automation can also change a state. If a merged PR moves an item to Done, the activity history should help explain the update. Do not repeatedly move it back without understanding the applicable rule. A mapping and its rule configuration determine automation; a familiar state name alone does not enable a transition.

4.5 Priority, ownership and labels

Priority expresses relative importance. The captured selector offers None, Urgent, High, Medium and Low. A high priority does not make an item automatically assigned, ready or unblocked. Discuss ordering with the person responsible for the team's plan instead of treating every personally inconvenient issue as urgent.

1. Open **Priority**.
2. Choose the agreed priority.
3. Add a comment if the reason for a significant change is not obvious.



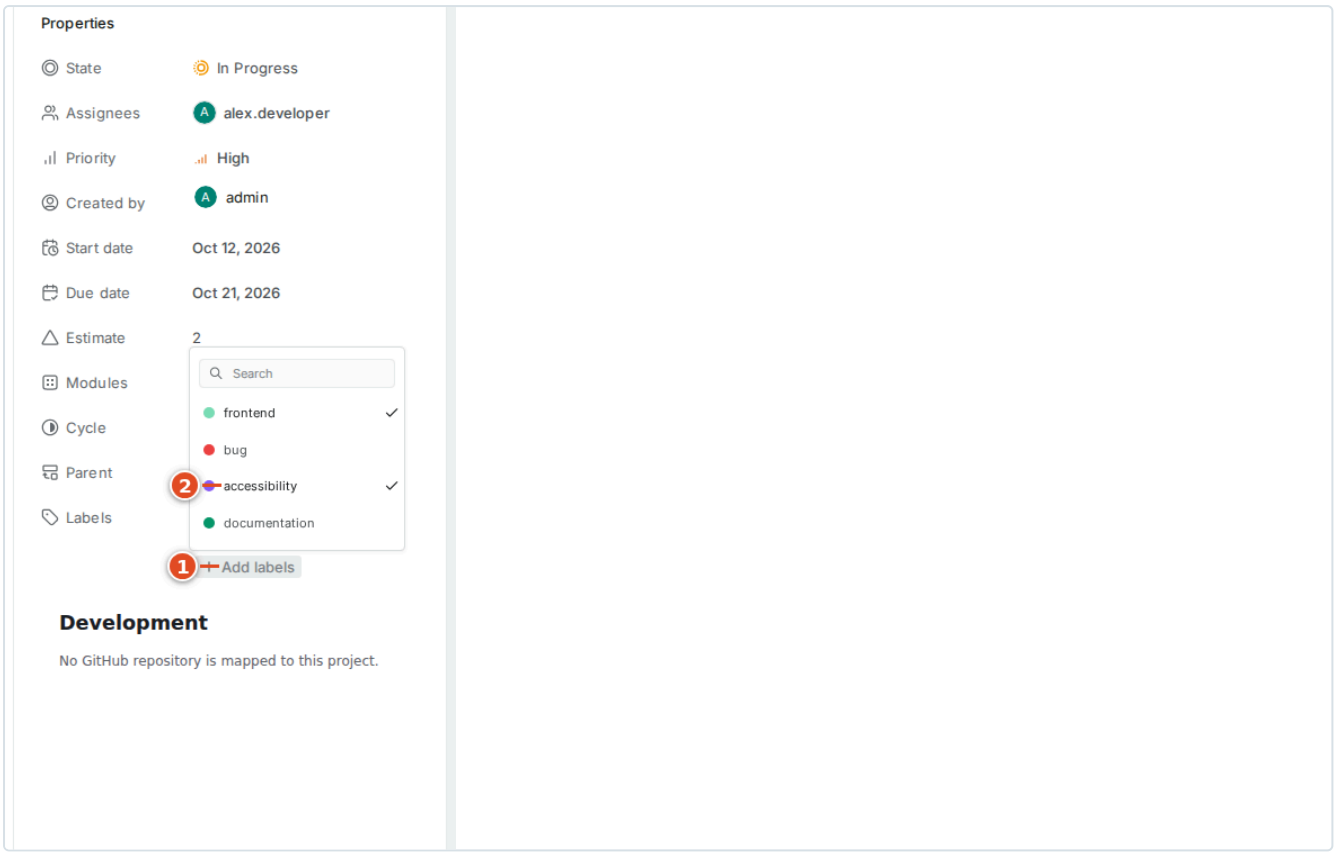
Priority: (1) current priority; (2) Urgent among the available priorities.

An assignee is a person responsible for moving the item forward. The selector can support more than one person. Multiple assignees are useful for a paired task, but the team should still know who will provide the next update. Removing an assignee does not delete their comments or mean the work is complete.

1. Open **Assignees**.
2. Search for the intended project member.
3. Select the member.
4. Close the selector.
5. Check the resulting names or avatars.

Labels describe reusable categories such as frontend, bug or accessibility. They are different from priorities and states. Use existing labels before creating near-duplicates with different capitalization. A project administrator may manage the label set. A GitHub label of the same name is synchronized only under the configured mapping and label policy; the visual resemblance does not establish a connection by itself.

1. Open **Labels** or **Add labels**.
2. Select the relevant existing labels.
3. Close the selector.

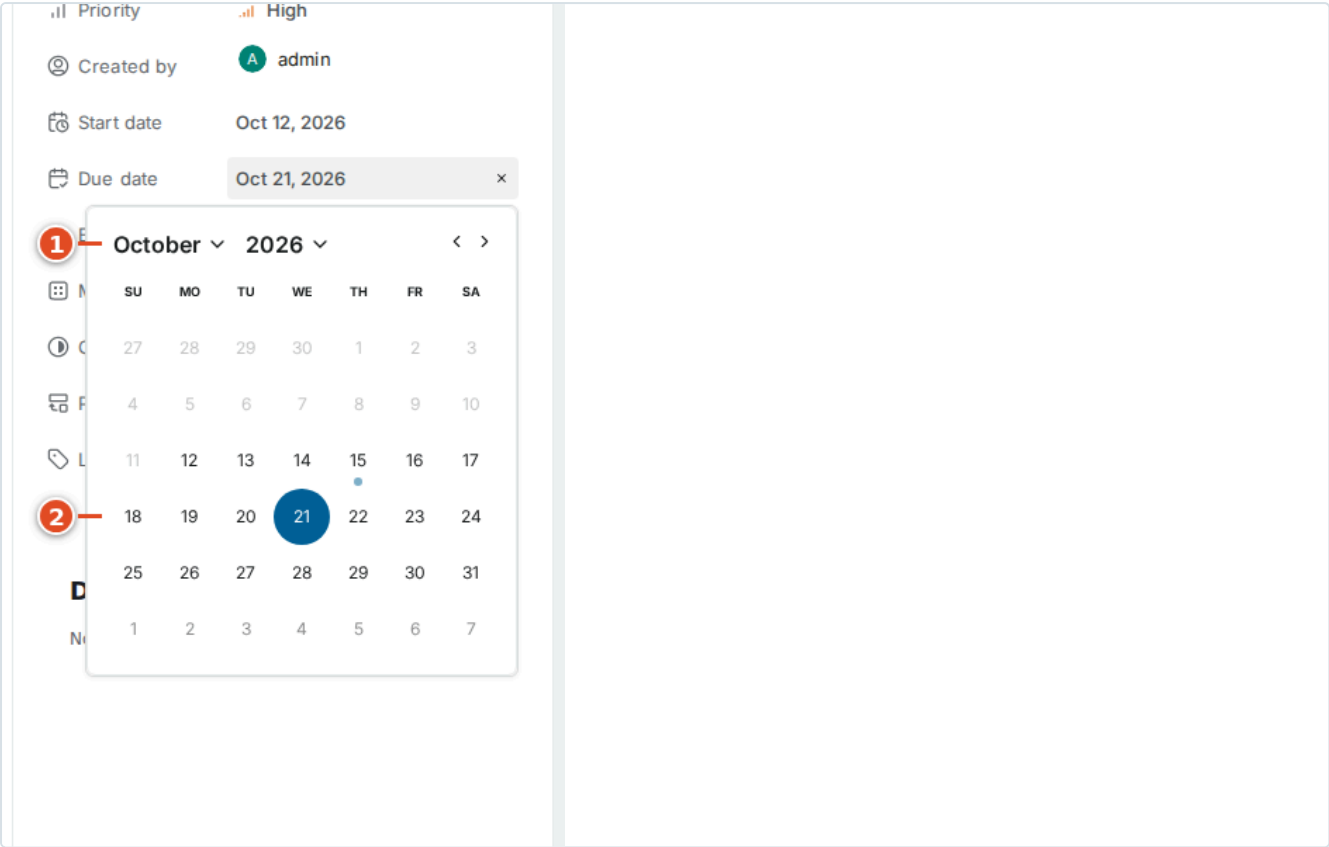


Labels: (1) Add labels; (2) accessibility in the project label choices.

4.6 Dates and estimates

A start date marks when work is intended to begin. A due date marks the intended deadline or target. These are planning properties; setting them does not create a calendar invitation. Calendar and Gantt layouts use dates to position work, so a missing date can explain an item absent from a dated view.

1. Open **Start date**.
2. Select the intended date.
3. Open **Due date**.
4. Select the intended date.
5. Check that the range makes sense.



Due date: (1) month selector; (2) selected day, 21 October.

An estimate is the team’s agreed expression of relative size or effort. The project administrator configures the estimation system. Do not assume that a value of 3 means three hours. Ask what the selected scale means and how the team uses it for planning. If no estimation system is enabled, the item may not offer an Estimate field. Setting an estimate is separate from the Smart Commit `#time` command: in this integration, that command creates a time-description comment rather than updating a native worklog or estimate.

4.7 Parents and sub-work items

A parent groups related work under a larger outcome. A sub-work item is still a work item with its own identifier, state and discussion. Use this structure when pieces can be implemented, reviewed or assigned independently. Use a checklist when the pieces are small checks that do not need their own lifecycle.

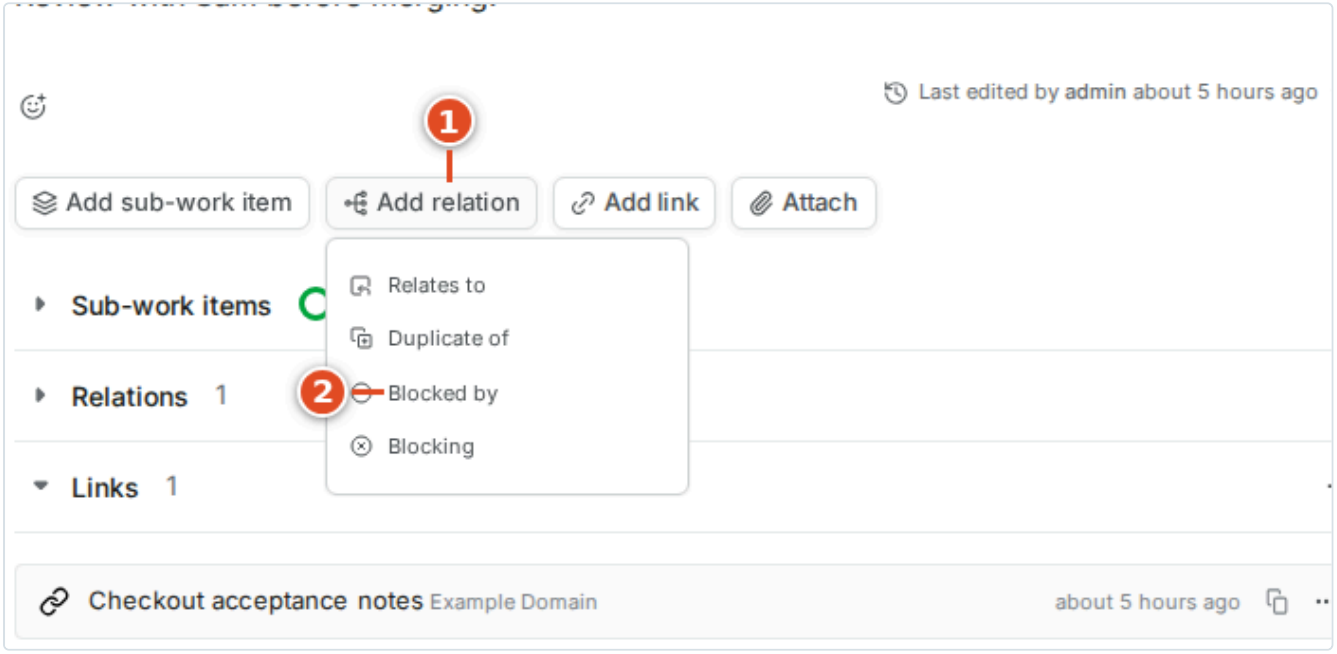
1. Open the intended parent.
2. Select **Add sub-work item**.
3. Choose an existing item or create the separate child using the offered form.
4. Check that the child appears in the parent’s sub-work item section.

The **Parent** property can also place an item under an existing parent. Check identifiers before moving it. A completed child does not prove the entire parent outcome is complete. Similarly, closing a parent's linked PR is not a documented automatic instruction to close every child. Reference the actual records each change should affect.

4.8 Relations explain dependencies and duplicates

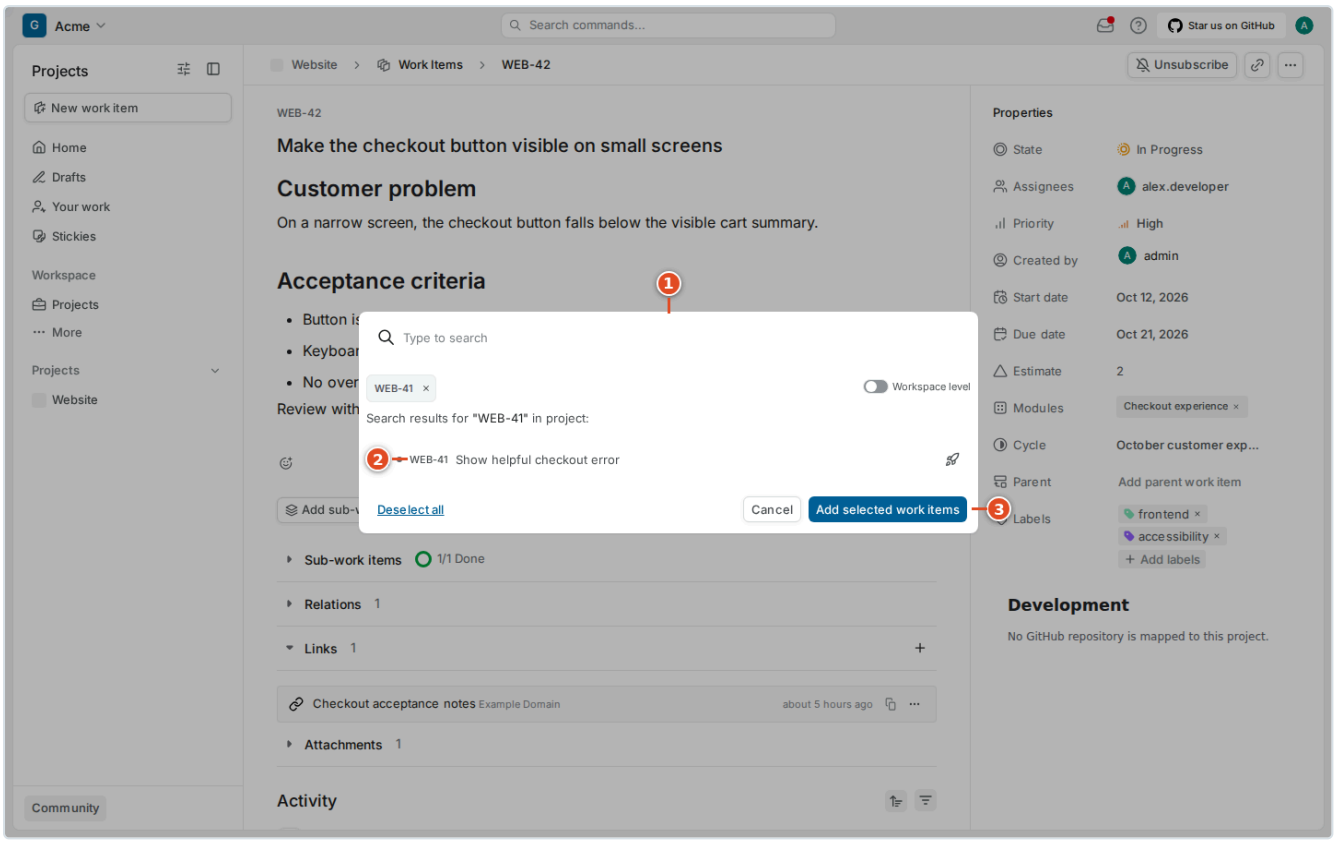
Relations connect records without making one a child of another. The captured menu offers **Relates to**, **Duplicate of**, **Blocked by** and **Blocking**. Direction matters. If WEB-42 cannot proceed until another task is finished, WEB-42 is blocked by that other task. Reading the relation backward can mislead the person choosing work.

1. Select **Add relation**.
2. Choose the relation type.
3. Search for the other work item.
4. Select the correct identifier.
5. Confirm the relation if the dialog asks.



Relations: (1) Add relation; (2) Blocked by, alongside Relates to, Duplicate of and Blocking.

Add a short explanatory comment when a dependency is not self-evident. A relation records a fact for the team; it does not guarantee that git-sync will prevent a PR merge. GitHub merge requirements and Plane planning relations are different systems. Mark duplicates deliberately and preserve the useful evidence on the canonical item rather than deleting the second report immediately.

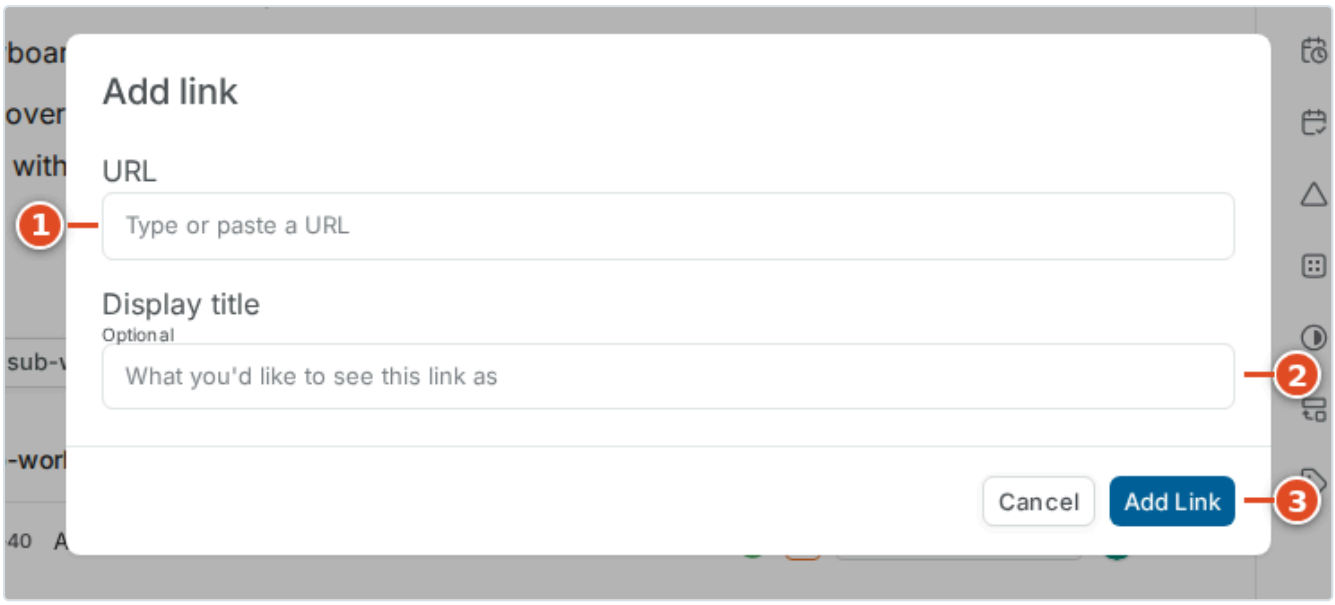


Choose a related item: (1) search by identifier; (2) selected result; (3) Add selected work items.

4.9 Links and attachments

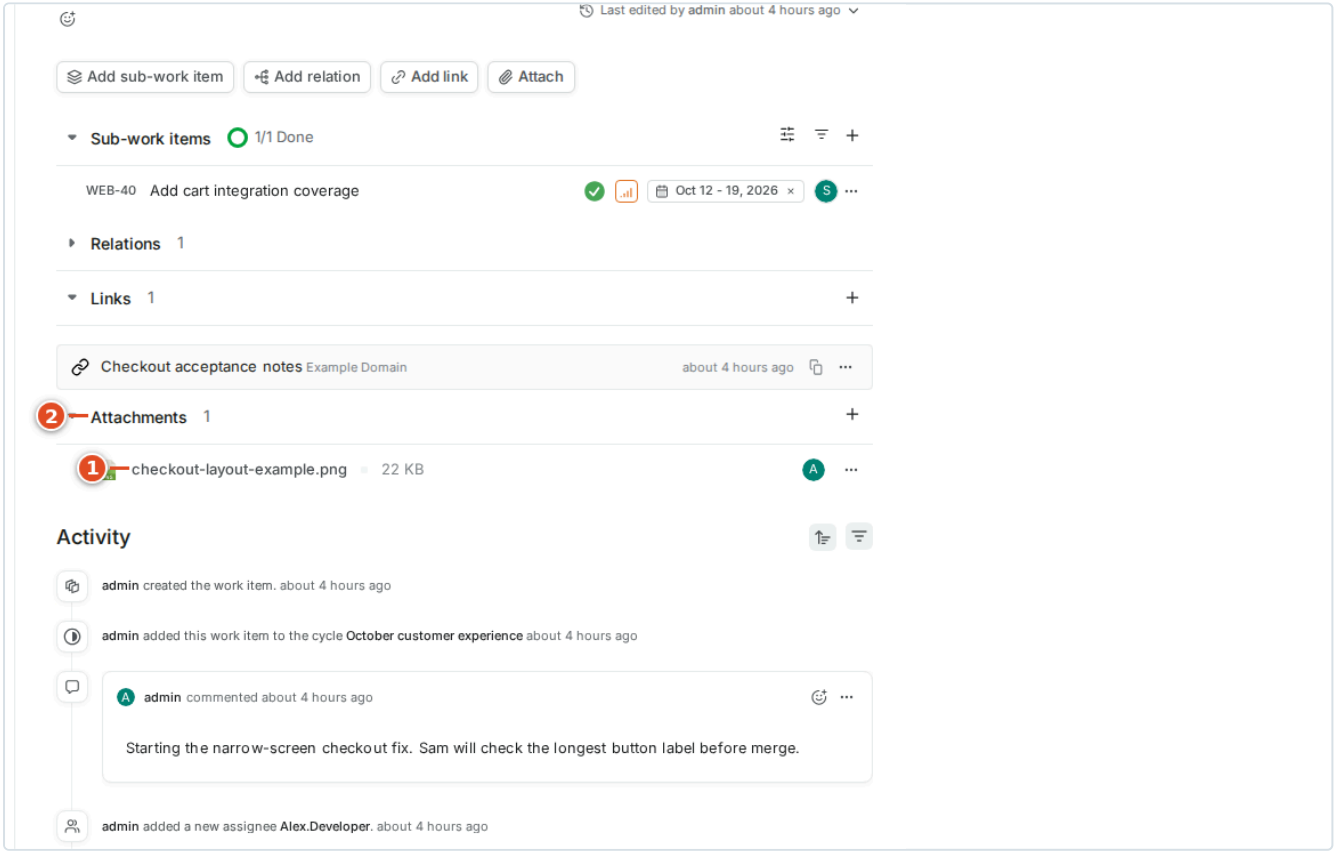
A link points to a web resource. An attachment is an uploaded file. Use a link for a maintained document and an attachment when the item needs a stable copy of evidence. Check whether the intended reader can access the destination before relying on a private link.

1. Select **Add link**.
2. Enter the destination URL.
3. Enter a useful display title.
4. Select **Add Link**.



Add link: (1) URL field; (2) display-title field; (3) Add Link.

1. Select **Attach**.
2. Choose the intended file in the file picker.
3. Wait for the upload result.
4. Open the attachment entry to verify that it is the correct file.



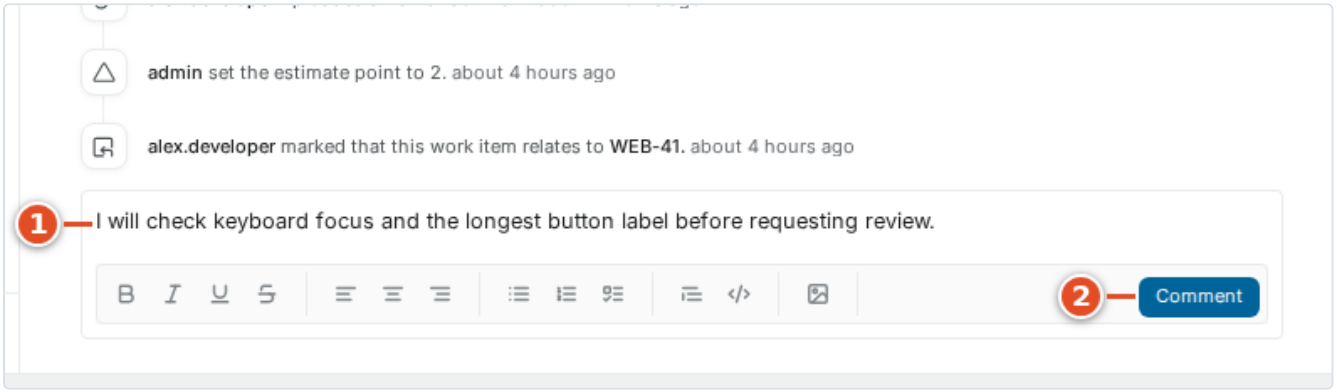
Attachment: (1) uploaded checkout-layout-example.png; (2) attachment section.

The server may limit file size and accepted uploads. If a large file fails, keep the work item and add an accessible link to the team’s approved storage instead. Avoid uploading a second file repeatedly while the first upload is still processing. A visible filename is useful, but opening the file once is a better confirmation that the upload completed.

4.10 Comments, reactions and activity

Use a comment to report progress, ask a focused question or record a decision. Keep the action clear: “I need a decision about the maximum label width” is more useful than “Any thoughts?” A reaction is a lightweight acknowledgment attached to an existing comment; it should not replace a clear approval when the team needs an explicit decision.

1. Open the comment editor below Activity.
2. Write the update or question.
3. Select **Comment**.
4. Check the posted comment.



Comment editor: (1) write the update; (2) Comment submits it.

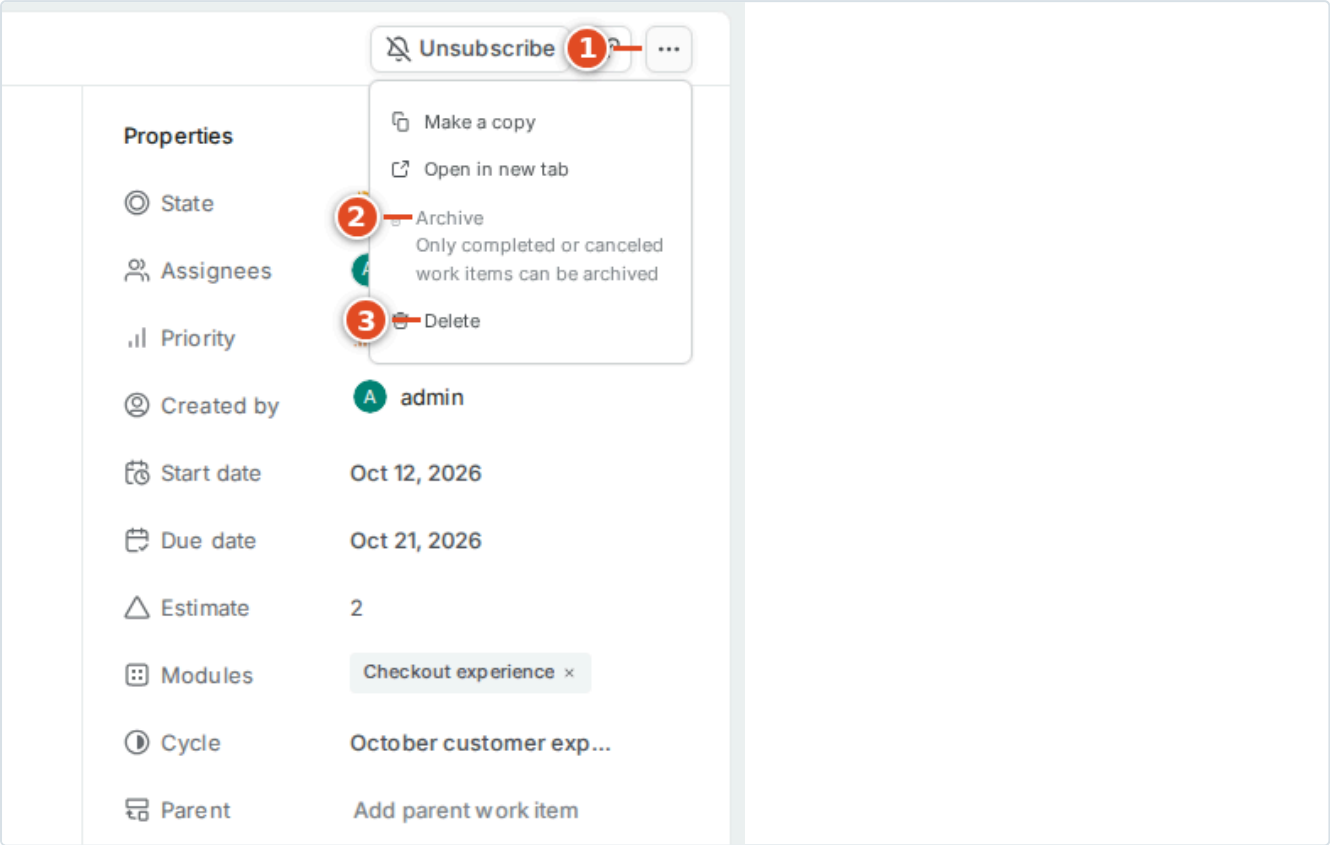
Use the reaction control on a posted comment to choose an acknowledgment. Use the comment’s own menu for edit or delete actions available to you. Deleting a comment can remove useful context, so explain a corrected decision instead when that history matters.

Activity records property changes and other events alongside comments. It helps answer who changed a state, when an assignee was added and whether a cycle or module changed. Read it before concluding that an unexpected value is a sync defect. A queued integration action may have occurred after your last manual edit.

4.11 Subscribe, share, archive and delete

Subscribe asks Plane to keep you informed about that item’s changes. The captured subscribed state reads **Unsubscribe**. Open an item you need to follow and select the subscription control near its identifier. Subscription does not assign the work to you. Whether an update also arrives by email depends on your preferences and the server’s mail configuration.

Use the item header’s copy controls or action menu to copy its link or identifier. The item menu also exposes actions such as Make a copy, Open in new tab, Archive and Delete. The captured CE menu disables Archive for work that is not completed or cancelled.



Item menu: (1) menu button; (2) archive availability; (3) Delete.

Archive completed or cancelled work when the team wants it out of normal active lists while retaining the record. Deletion is a stronger action and should follow the team’s policy. Read the confirmation carefully and check the identifier. Do not delete a task merely to undo a state transition, a label change or an accidental comment.

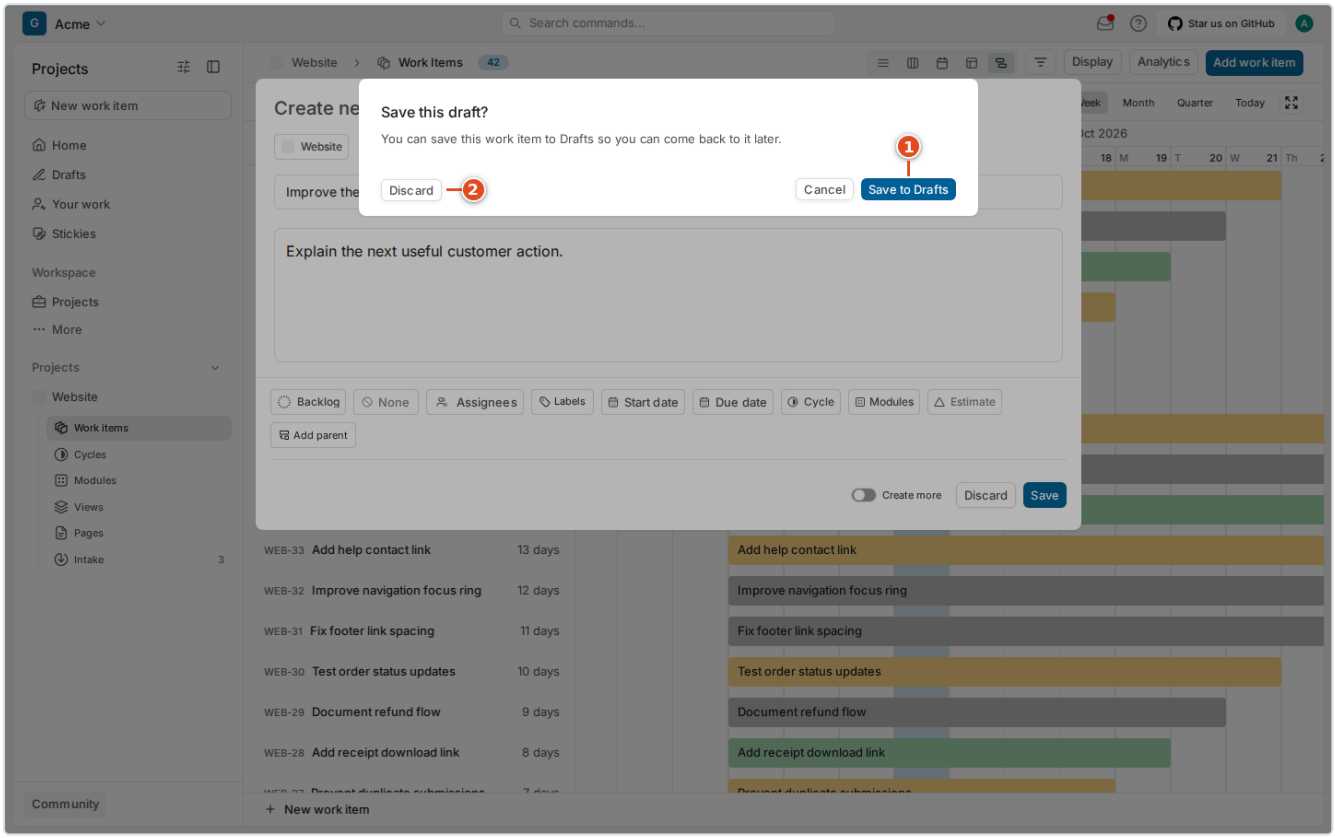
> Warning: Archive, deletion, cancellation and completion are different actions. Closing a GitHub PR without merging is another different action. Use the one that describes what actually happened.

4.12 Keep unfinished writing in Drafts

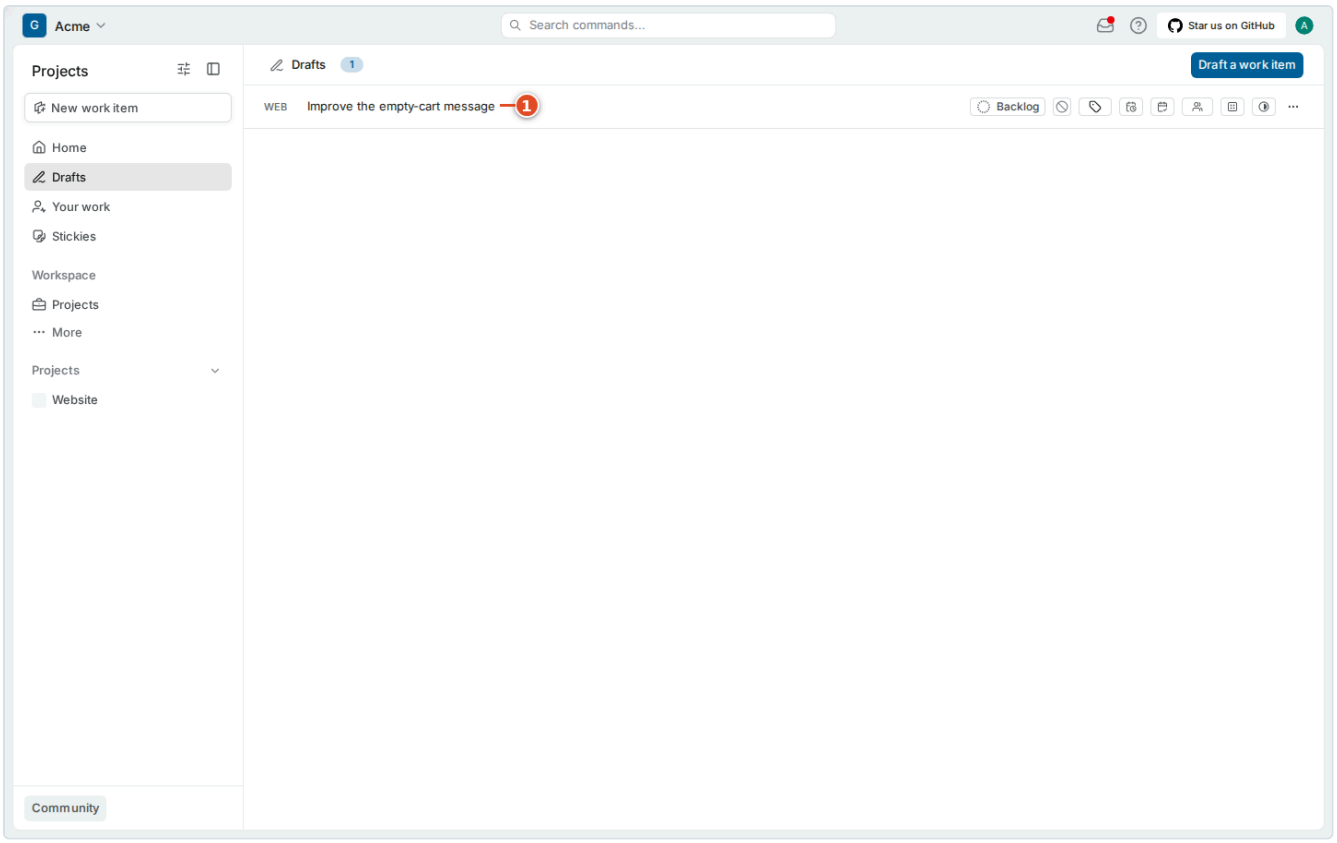
If you start creating an item and close it before saving, Plane can offer to keep that unfinished writing in Drafts. This is useful when you need more evidence before creating a shared task. A draft is not the same as a ready work item, and it should not be used as the only place to track something the team has already committed to deliver.

1. Close the unfinished creation form.
2. Select **Save to Drafts** in the confirmation.

3. Open **Drafts** from the sidebar.
4. Open the saved draft when you are ready to finish it.
5. Review its project and properties before publishing it as a work item.



Unfinished item: (1) Save to Drafts preserves the writing; (2) Discard removes it.



Drafts: (1) the saved unfinished item. Open it to continue editing.

Discard removes the unfinished draft instead. If you are uncertain, save the draft and review it later. Before turning a long-lived draft into a new task, search again: another teammate may have created the shared item while you were gathering context.

05. Seeing work your way

A project can contain more work than you can usefully read at once. A layout changes how records are arranged. A filter chooses which records appear. Grouping divides the matching records into useful sections, and ordering decides which record comes first. A saved view keeps a useful selection so you can return to it without rebuilding the same settings every morning.

These controls affect your presentation of shared work. Editing a property’s value still changes the actual work item. Hiding the Due date column does not remove due dates, and filtering to In Progress does not put every hidden item into another state.

5.1 Choose a layout

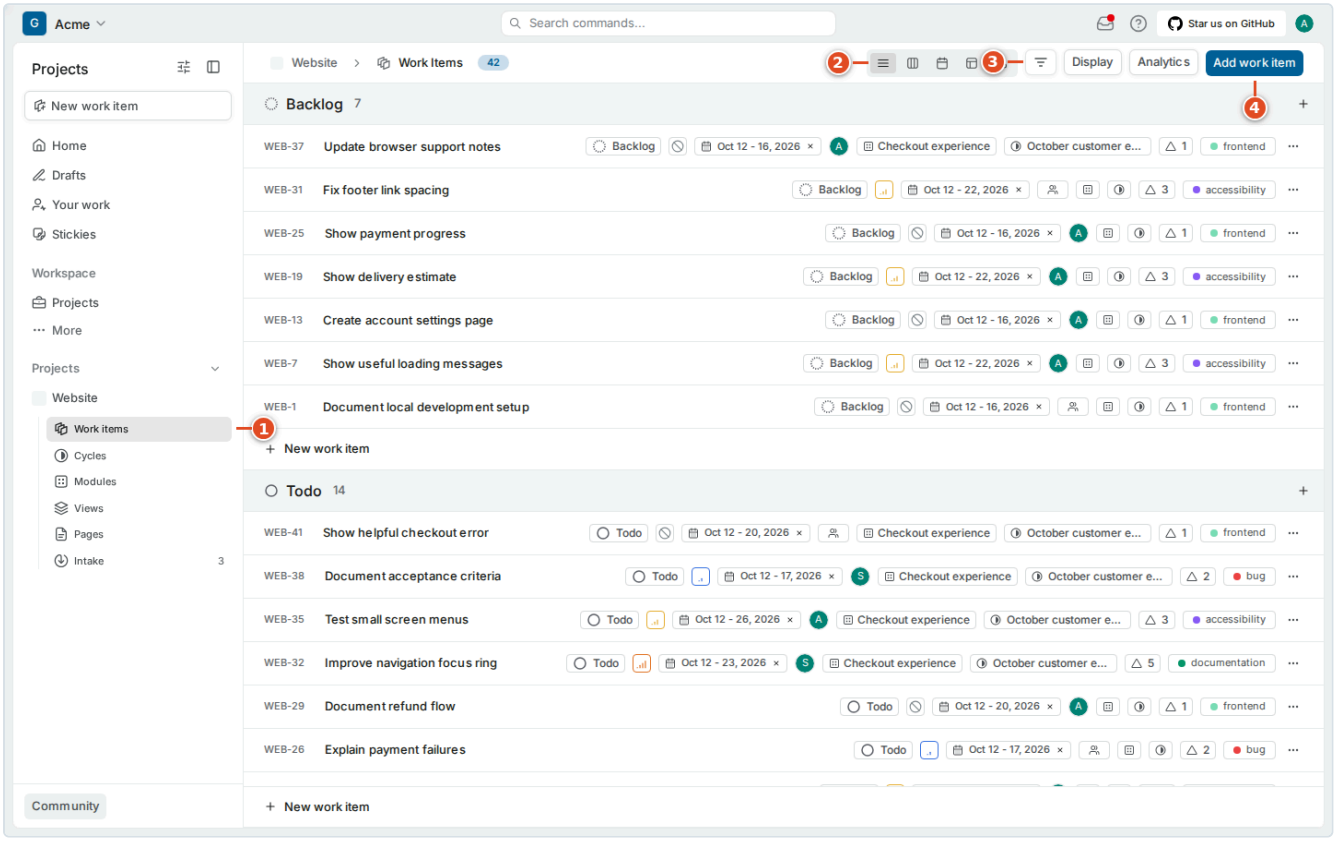
The running CE 1.4.2 project provides five layout buttons in the work-item header. The screenshot markers help locate the selector. Hover an unfamiliar icon to read its tooltip rather than guessing from the shape.

1. Open the project’s **Work items** page.
2. Find the row of layout icons in the header.
3. Select the layout suited to your question.

Layout	A useful question	Remember
List	What is ready for me to pick up?	Grouping and ordering determine the visible sequence.
Board, or Kanban	Where is work accumulating?	A column often represents a state; inspect grouping first.
Calendar	What has a date in this period?	An undated item may be absent from the calendar.
Spreadsheet	Which properties need a quick comparison?	Editable cells update the shared work item.
Gantt	How do the planned date ranges overlap?	A bar’s position represents dates, not proof of progress.

5.2 Use the list for a readable queue

List layout gives each work item a row with its identifier, title and selected properties. It is useful when you need to scan titles or compare a small set of fields without the horizontal space a board uses. Expand the item for its description before deciding that the short title tells the full story.



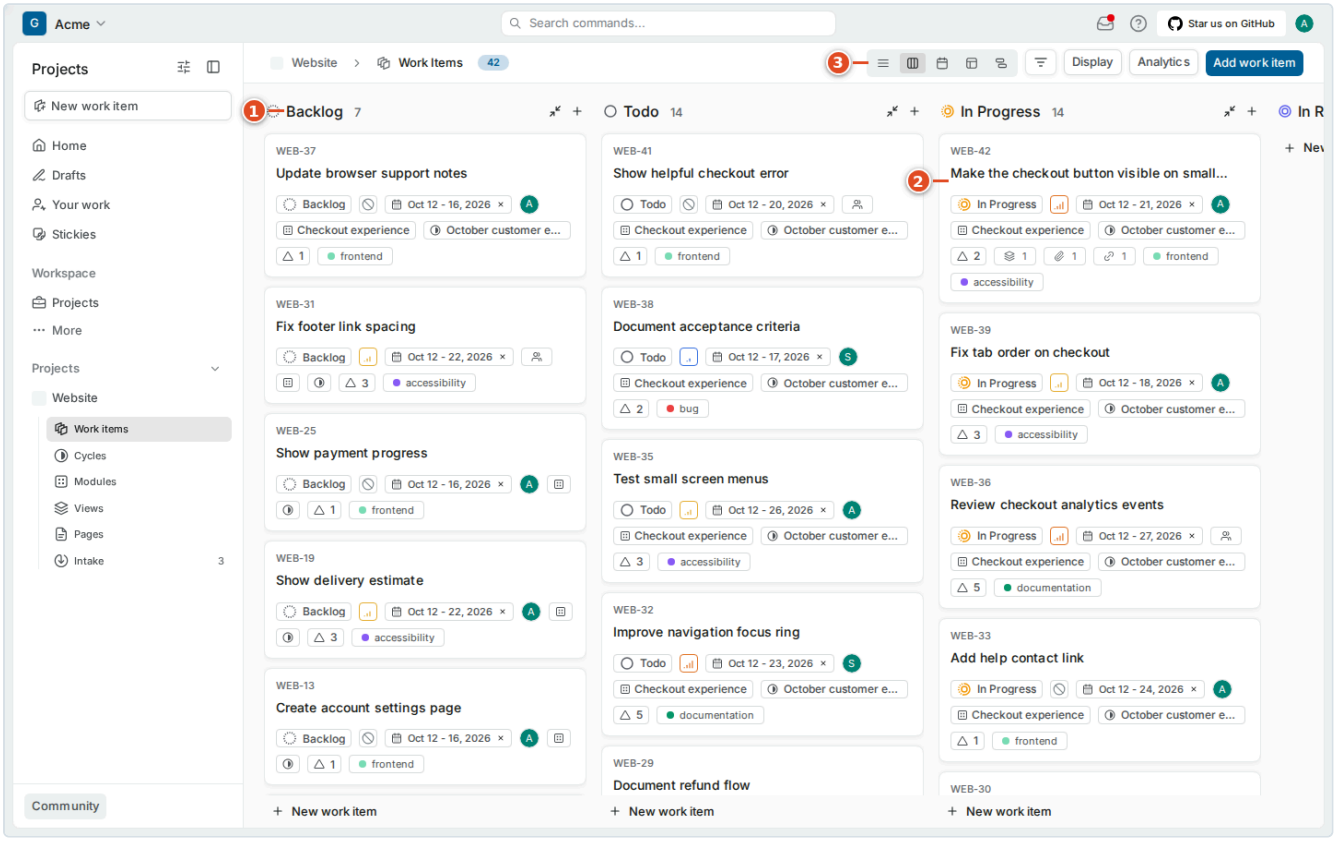
List layout: (1) project navigation; (2) layout buttons; (3) filter button; (4) Add work item.

1. Choose the list icon.
2. Read the group heading above the rows.
3. Check the ordering in Display.
4. Open the item you want to inspect.

A count beside a group describes the items matching the current view. If an item is unexpectedly missing, inspect filters and sub-work item visibility before creating a duplicate. A collapsed group can also hide work that still matches the selection.

5.3 Use the board to discuss flow

A board arranges work in columns and shows each item as a card. When grouped by state, it is a good shared picture for a stand-up conversation: what is waiting, what is being implemented, and what is ready for review. It also makes a growing review queue easier to notice.



Board layout: (1) state columns; (2) work item cards; (3) layout selector.

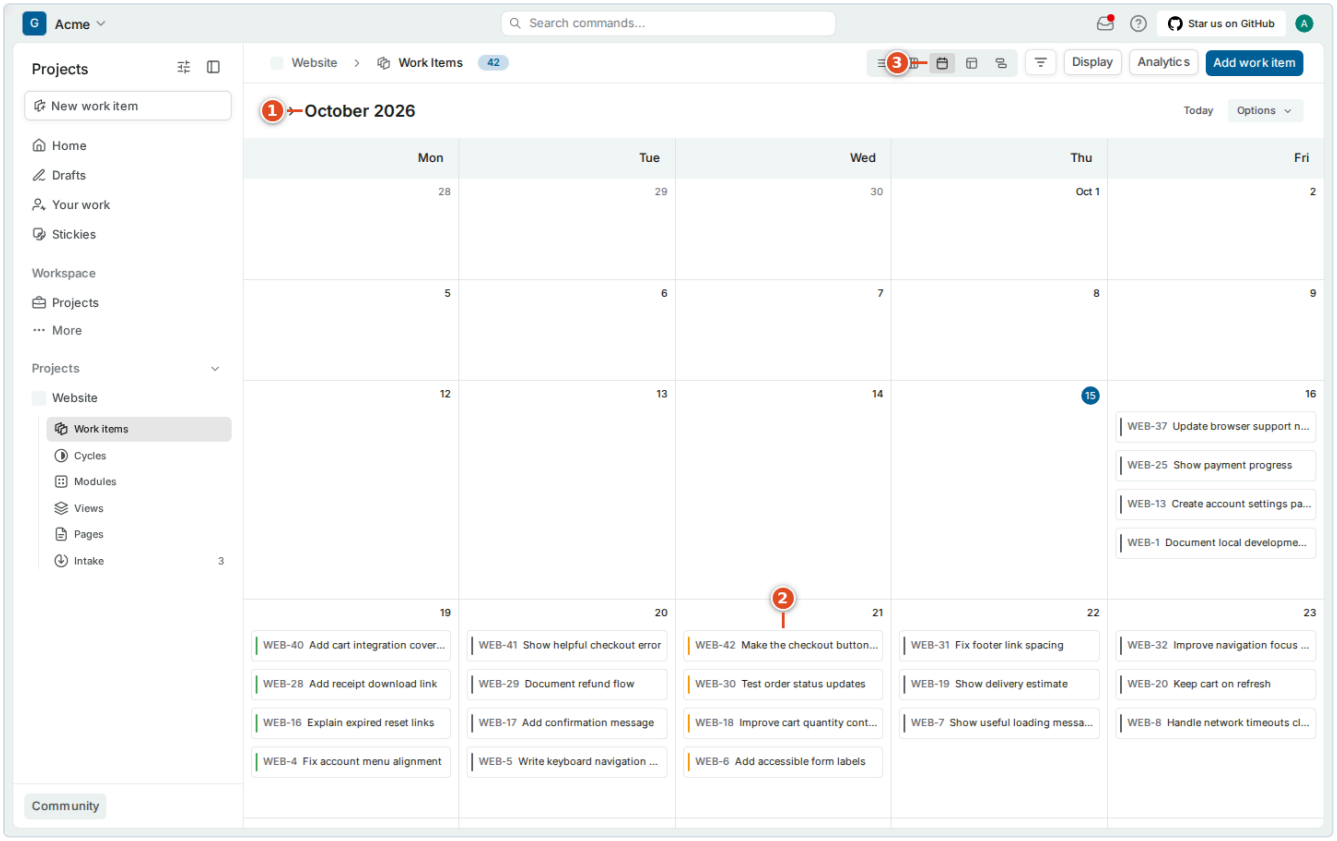
1. Choose the board icon.
2. Check what the column headings represent.
3. Open a card before changing its workflow state.
4. Change the item to the state that reflects the work.

Dragging a card between state columns can update its state. Verify the resulting value when using that interaction. If the board is grouped by a different property, moving a card has a different meaning. Grouping by assignee is not the same workflow operation as grouping by state.

> Tip: A wide board may extend horizontally. Scroll sideways before concluding that Done or Cancelled is missing. Filters can remove entire groups from the visible result as well.

5.4 Use the calendar for dated work

Calendar layout places work according to the date information used by that view. It is useful for reviewing a delivery week or noticing several items with the same target date. It is less useful as a complete backlog, because a task without a relevant date has no obvious calendar position.



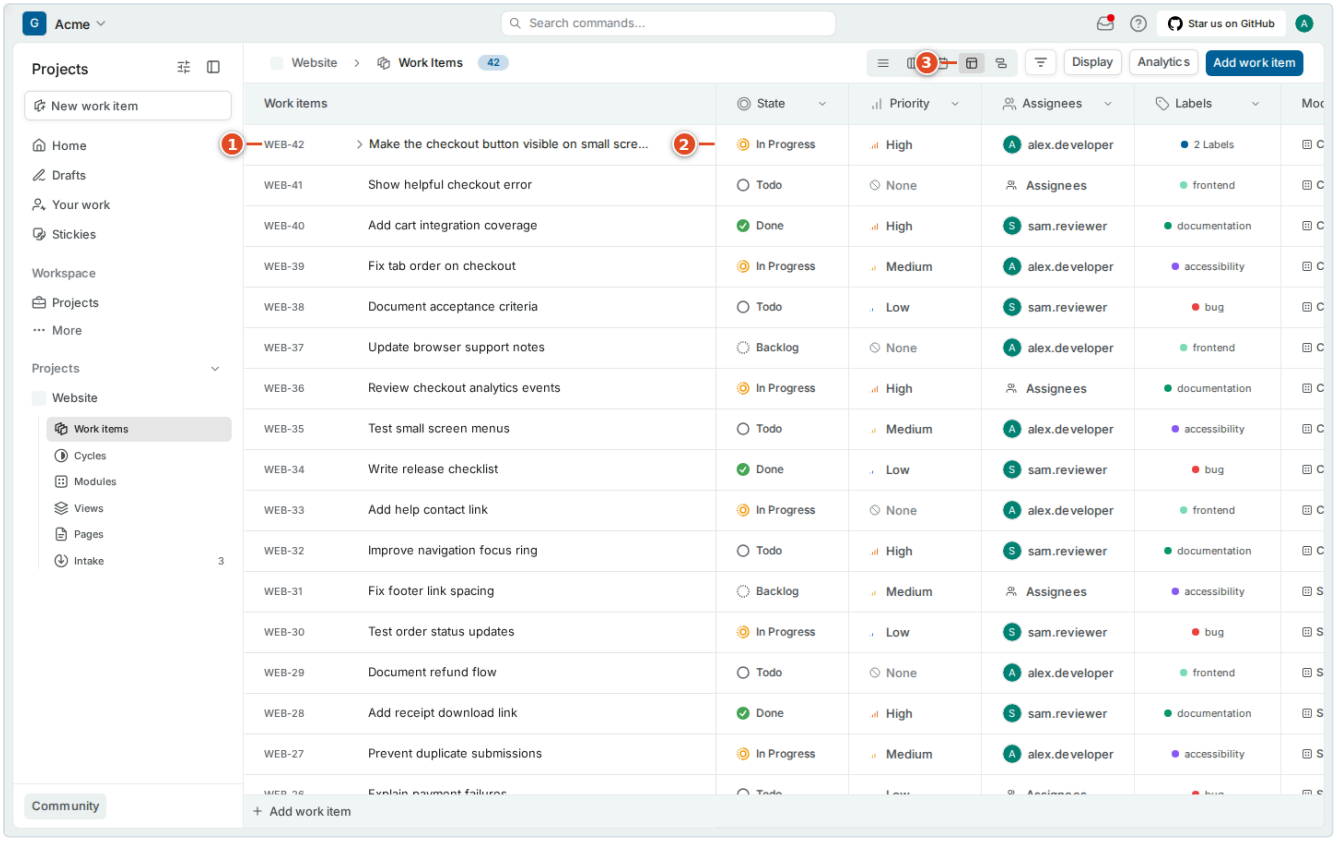
Calendar: (1) displayed month; (2) WEB-42 on its due date; (3) calendar layout button.

1. Choose the calendar icon.
2. Check the displayed month or period.
3. Use the date navigation to reach the relevant period.
4. Open the dated item you want to inspect.
5. Verify the date in its properties before changing it.

Changing a deadline should be a planning decision. Do not drag a date simply to improve the appearance of an overcrowded calendar. If several items legitimately share a deadline, discuss scope or capacity with the team and record the decision on the affected work.

5.5 Use the spreadsheet for careful property edits

Spreadsheet layout places properties in columns. It helps you compare state, priority, assignees, dates, modules and cycles across a set of items. Some columns contain counts or derived information rather than an editable input. Open an unfamiliar cell to see what the interface permits.



Spreadsheet: (1) WEB-42 row; (2) state cell; (3) spreadsheet layout button.

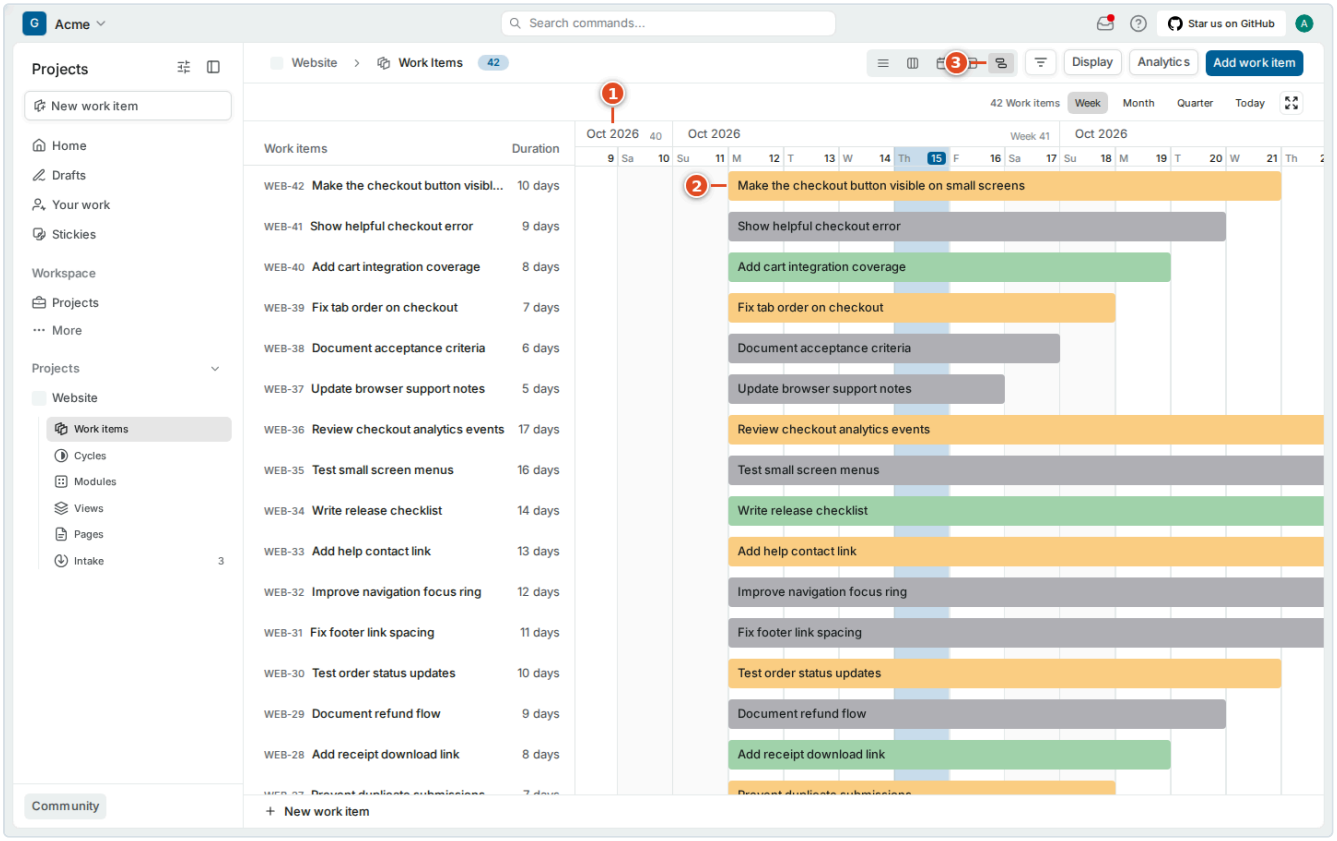
1. Choose the spreadsheet icon.
2. Check the identifier at the left of the row.
3. Select the property cell you intend to change.
4. Choose the new value from its control.
5. Wait for the update to finish.
6. Check the same row before editing another item.

The first identifier column helps preserve context while scrolling horizontally. Keep that context in view when editing similar titles. A row selected for a bulk operation is different from a single open property cell. Read the action’s scope before applying it to several records.

> Warning: A spreadsheet is a convenient editor, not a private draft of the project's data. A property edit can trigger a webhook and configured sync behavior just like an edit on the detail page.

5.6 Use Gantt to compare date ranges

Gantt layout shows work as bars on a time scale. Use it to see overlapping plans or to discuss a start and target range. Check the scale and visible period before interpreting a short bar as a small task. A one-week range can look different at a monthly scale than at a daily scale.



Gantt: (1) time scale; (2) dated work-item bar; (3) Gantt layout button.

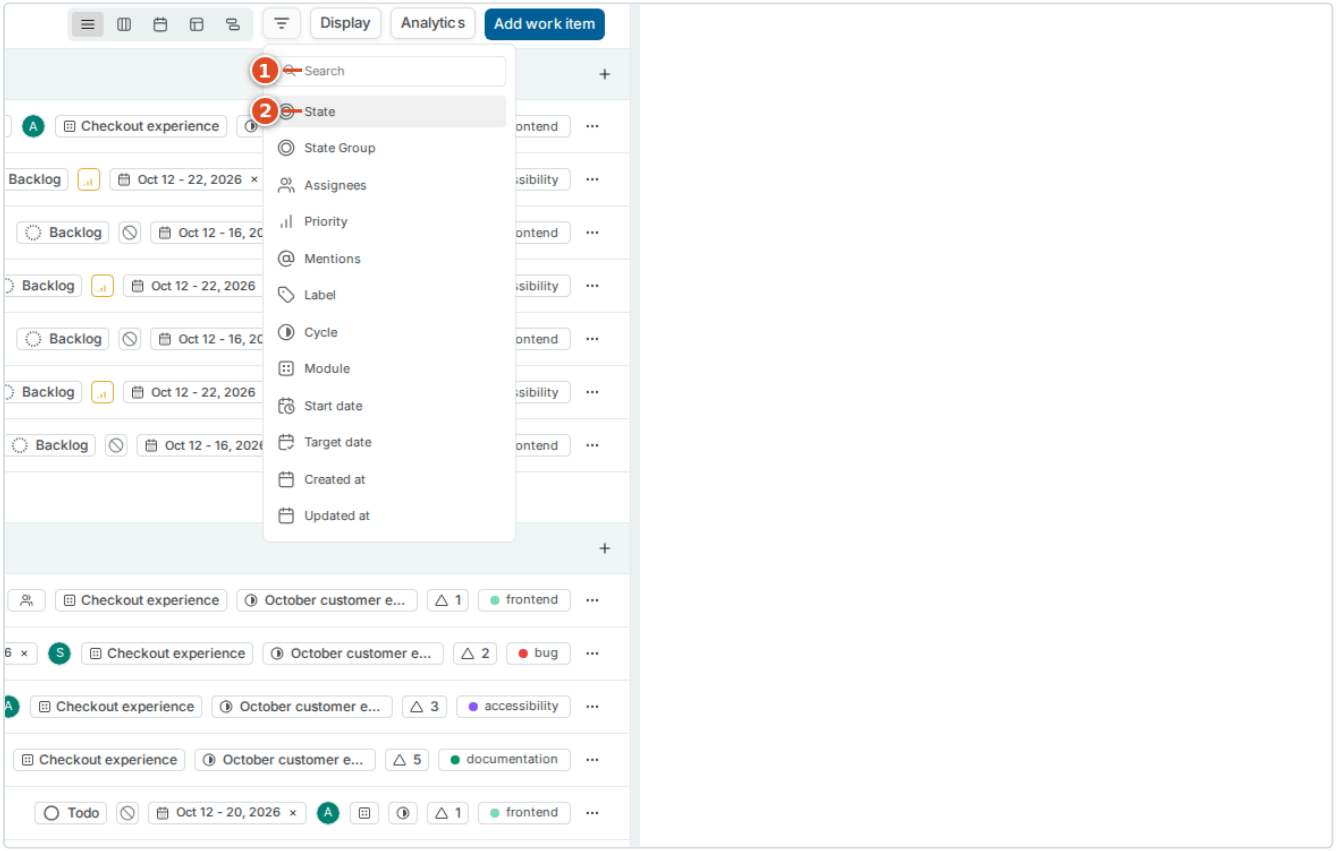
1. Choose the Gantt icon.
2. Check the time scale in the header.
3. Find the relevant item name.
4. Compare its bar with the displayed dates.
5. Open the item when you need the precise date fields or dependencies.

A date range is a plan. The bar does not establish that someone has started coding, and it does not replace a blocking relation or an explanation in a comment. Keep the state and dates consistent with the team’s actual plan instead of using one as a substitute for the other.

5.7 Filter without losing your place

The filter control in this project offers categories including State, State Group, Assignees, Priority, Mentions, Label, Cycle, Module, Start date, Target date, Created at, Updated at and Created by. Choose the narrowest useful question. “Assigned to me and not finished” is easier to reason about than a dozen unrelated conditions.

1. Open the filter control beside the layout selector.
2. Choose a property category.
3. Select its desired value or values.
4. Review the resulting list and count.
5. Remove the filter when you need the complete project again.



Filters: (1) search field; (2) available property categories. Choose a category to select its values.

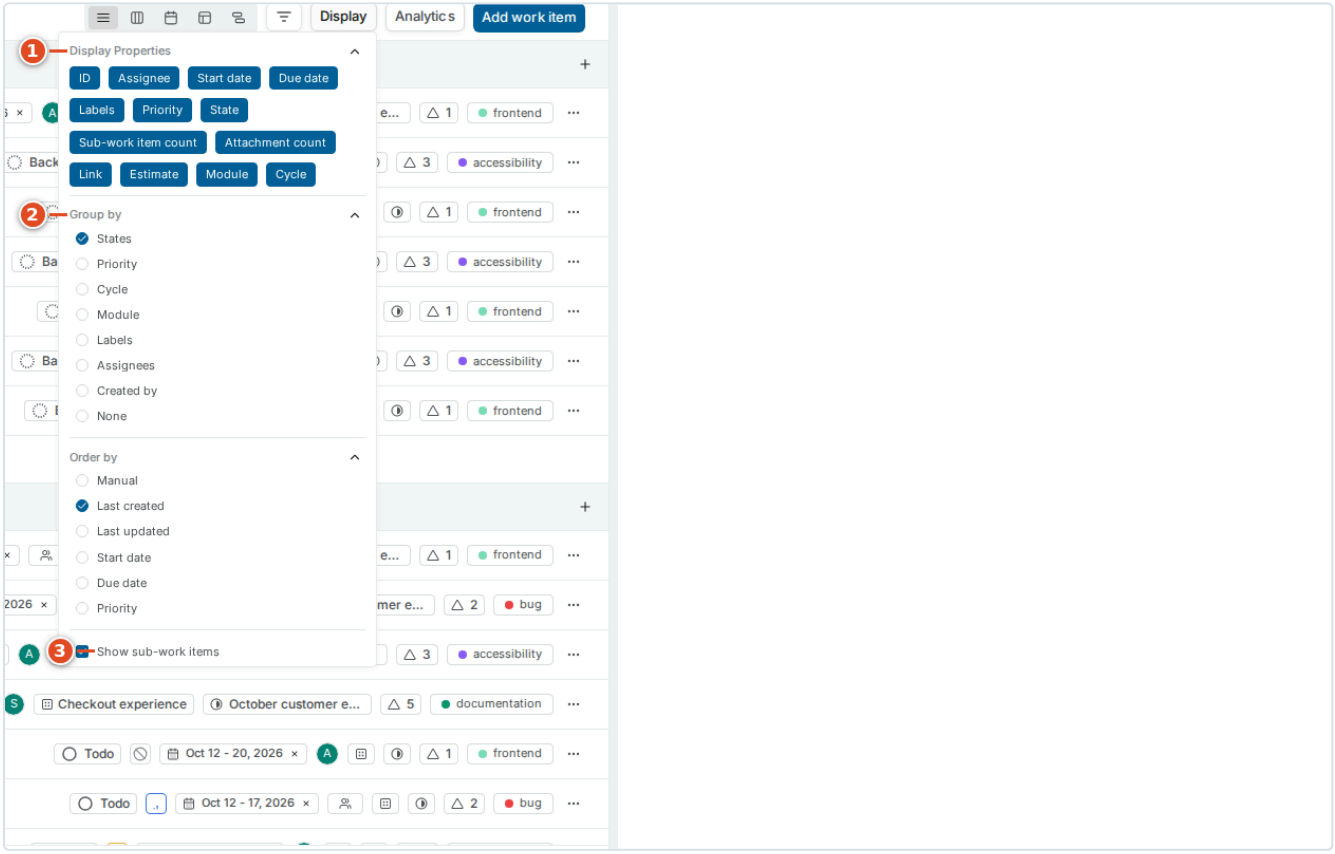
Different filter controls can combine selections differently. Read the displayed selection and resulting records instead of assuming every click adds a simple “and.” A date filter may use a relative range that changes as time passes. A saved view built around “this week” will show different records next week even if nobody edits the view.

When a teammate says an item is missing, compare the project, filters, selected layout and visible period. Sharing the direct work item link is a quick way to separate a presentation problem from an access problem.

5.8 Choose display properties, grouping and ordering

Display controls let you decide which properties are useful in the current layout. The captured list menu includes ID, Assignee, Start date, Due date, Labels, Priority, State, sub-work item count, attachment count, link, Estimate, Module and Cycle. Showing every property can make a narrow screen harder to read; begin with the fields your team uses to choose work.

1. Open **Display**.
2. Select the properties you want to see.
3. Choose a **Group by** option.
4. Choose an **Order by** option.
5. Check **Show sub-work items** if you want children included.
6. Close the menu.



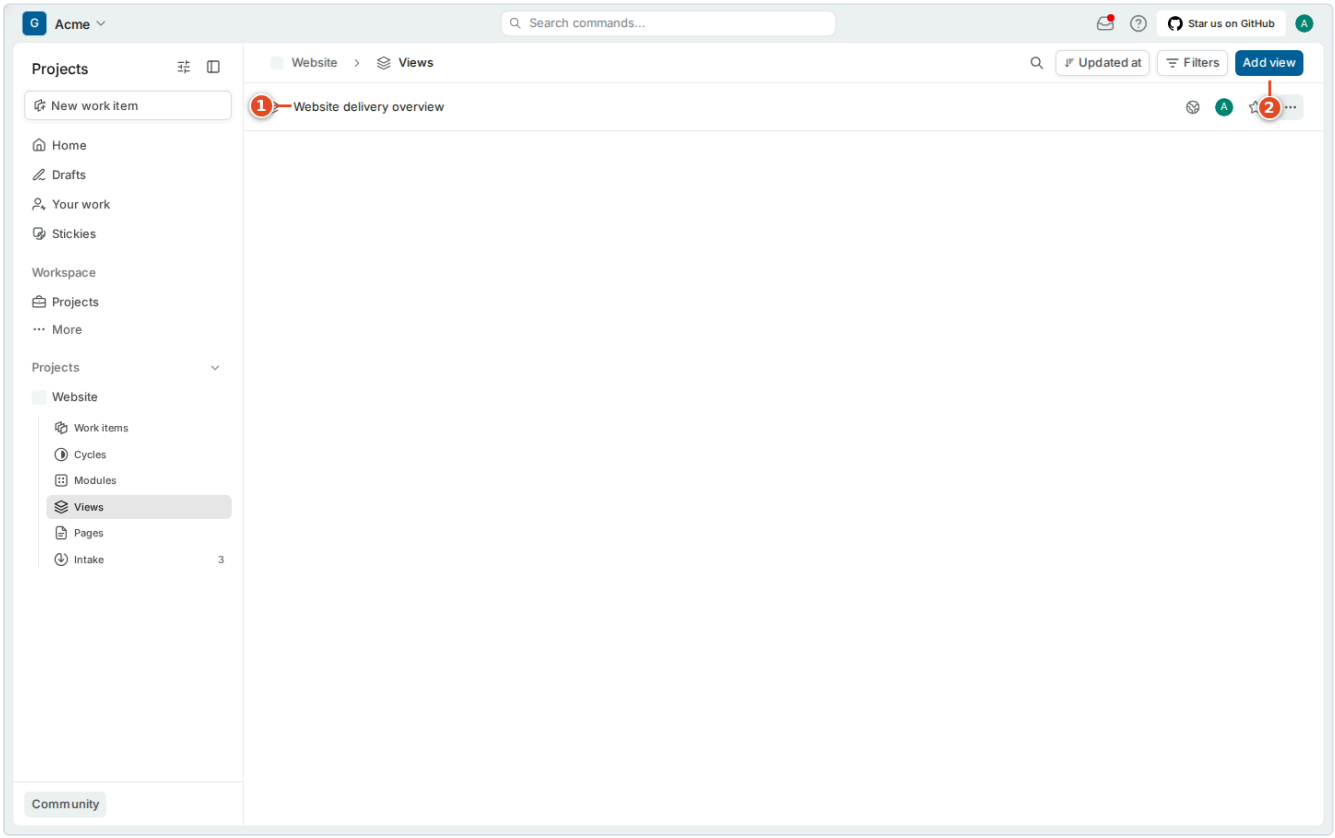
Display: (1) visible properties; (2) Group by; (3) Show sub-work items. Ordering is also in this menu.

Grouping can use states, priority, cycle, module, labels, assignees, created by or none in the captured list. Ordering includes manual order and property-based choices such as created, updated, dates or priority. The available choices can differ by layout. Sub-work item visibility controls whether children appear in the result; it does not detach them from their parents.

5.9 Save a useful project view

A saved view gives a useful query a name. Choose a name that communicates its selection, such as “Frontend work needing review.” Avoid a vague title such as “My view 2” when teammates will depend on it. Check the selected project’s scope and any access controls offered by the form.

1. Open **Views** under the project.
2. Start the new-view form.
3. Enter a descriptive name.
4. Add a short explanation of the intended selection.
5. Configure the relevant filters.
6. Save the view.
7. Open it again to verify the resulting records.



Project Views: (1) Website delivery overview; (2) Add view creates another saved selection.

A saved view does not copy the work items. An item can appear in several views, and a change made through one view becomes visible wherever the same item matches another query. If a shared view needs a new meaning, discuss that change before editing it; other developers may use its old selection in their daily routine.

06. Planning with cycles, modules, pages and intake

Planning tools answer different questions. A cycle provides a time window. A module groups a feature area or substantial outcome. A page holds written knowledge. Intake receives requests before the team accepts them into its regular plan. These features are present in the captured Community Edition 1.4.2 project; an administrator can enable or disable the optional project entries.

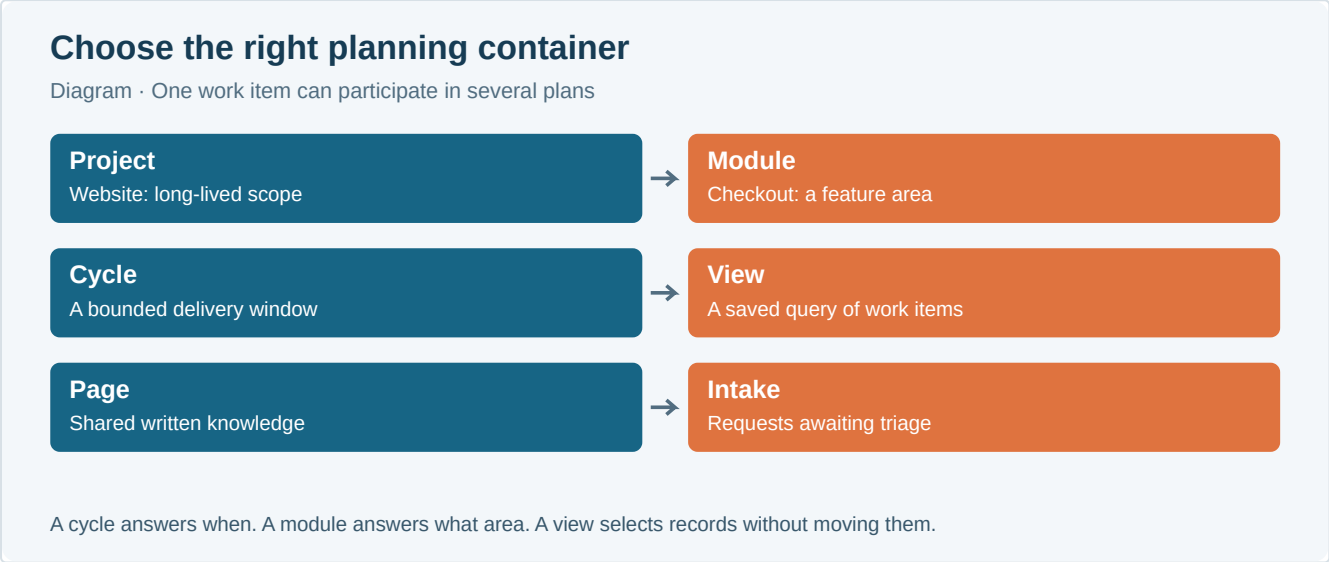


Diagram: project, module, cycle, view, page and intake serve different planning purposes.

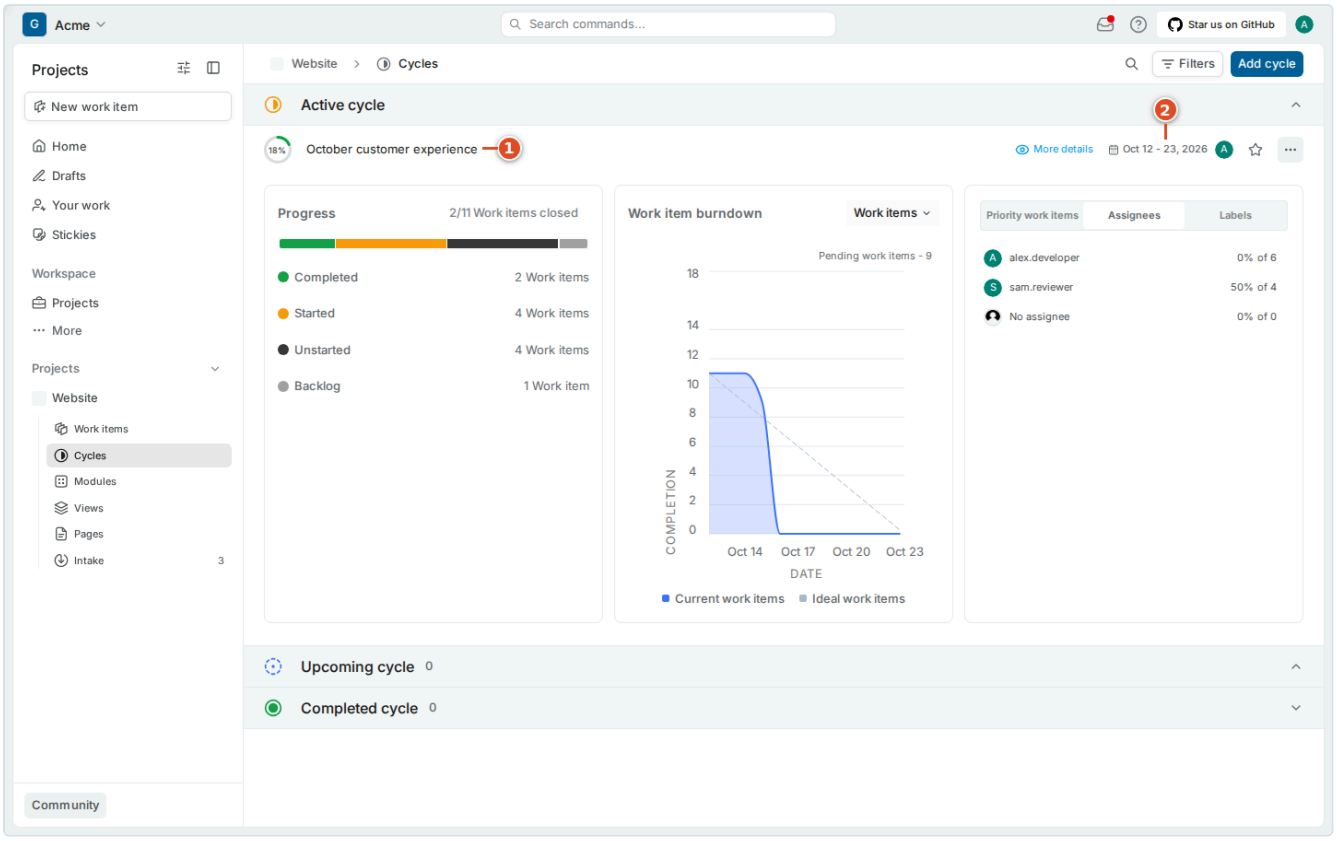
A work item can belong to a module and a cycle at the same time. Moving it into a planning container does not copy its description or create another independent task. Keep the original work item as the place to follow its implementation and discussion.

6.1 Work within a cycle

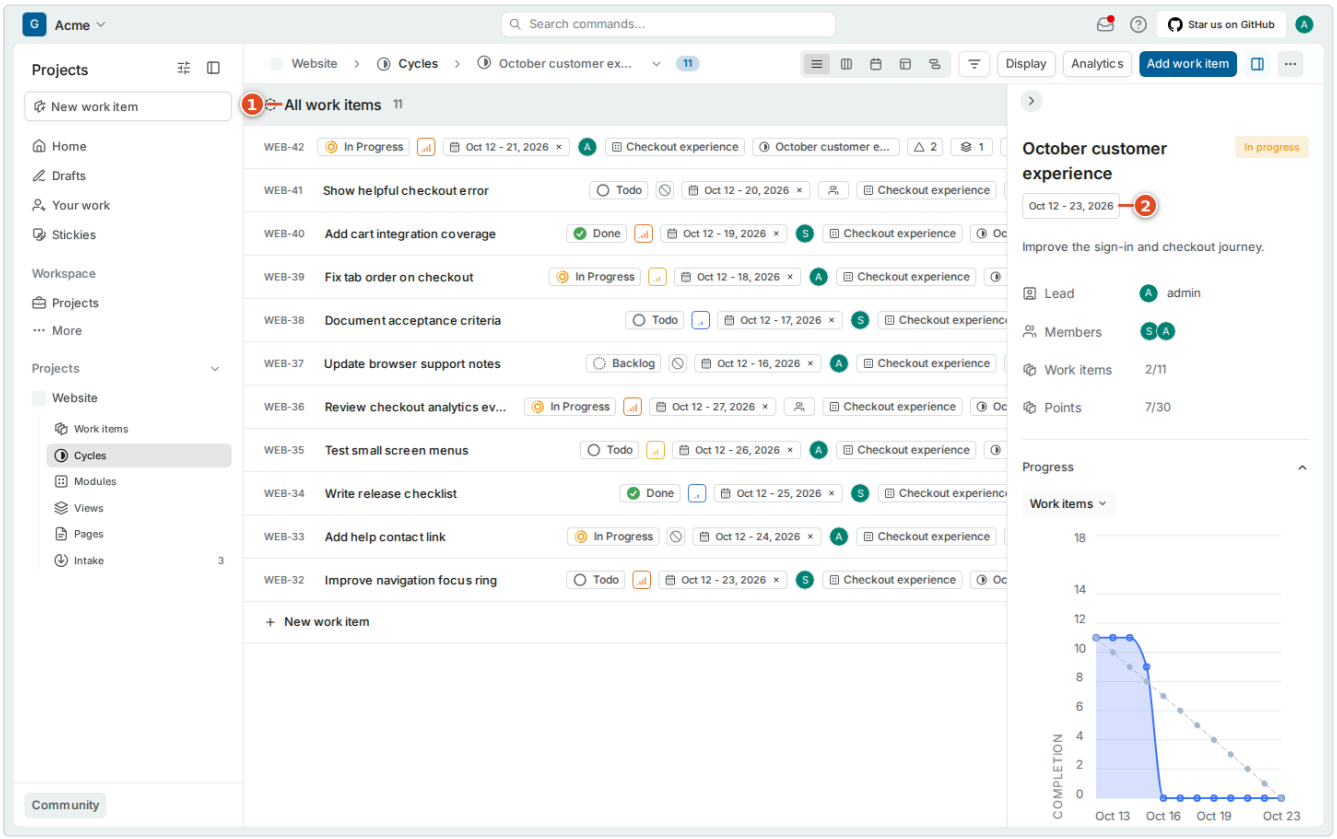
A cycle is a bounded period used to plan and inspect delivery. Some teams call this a sprint. The example **October customer experience** cycle has a start and end date and contains the Website items selected for that window. It is useful for discussing commitments and unfinished work without confusing the current plan with the entire backlog.

1. Open **Cycles** under the project.
2. Find the cycle your team is using.
3. Check its start and end dates.
4. Open the cycle.

5. Read its included work items and progress information.



Cycles: (1) cycle name; (2) scheduled date range.



Inside a cycle: (1) included work items; (2) dates and progress.

To place an item into a cycle, use its **Cycle** property and select the intended cycle. Check the dates before choosing a similarly named past cycle. A cycle that is already over is useful historical context, but it may not represent the team’s current commitment.

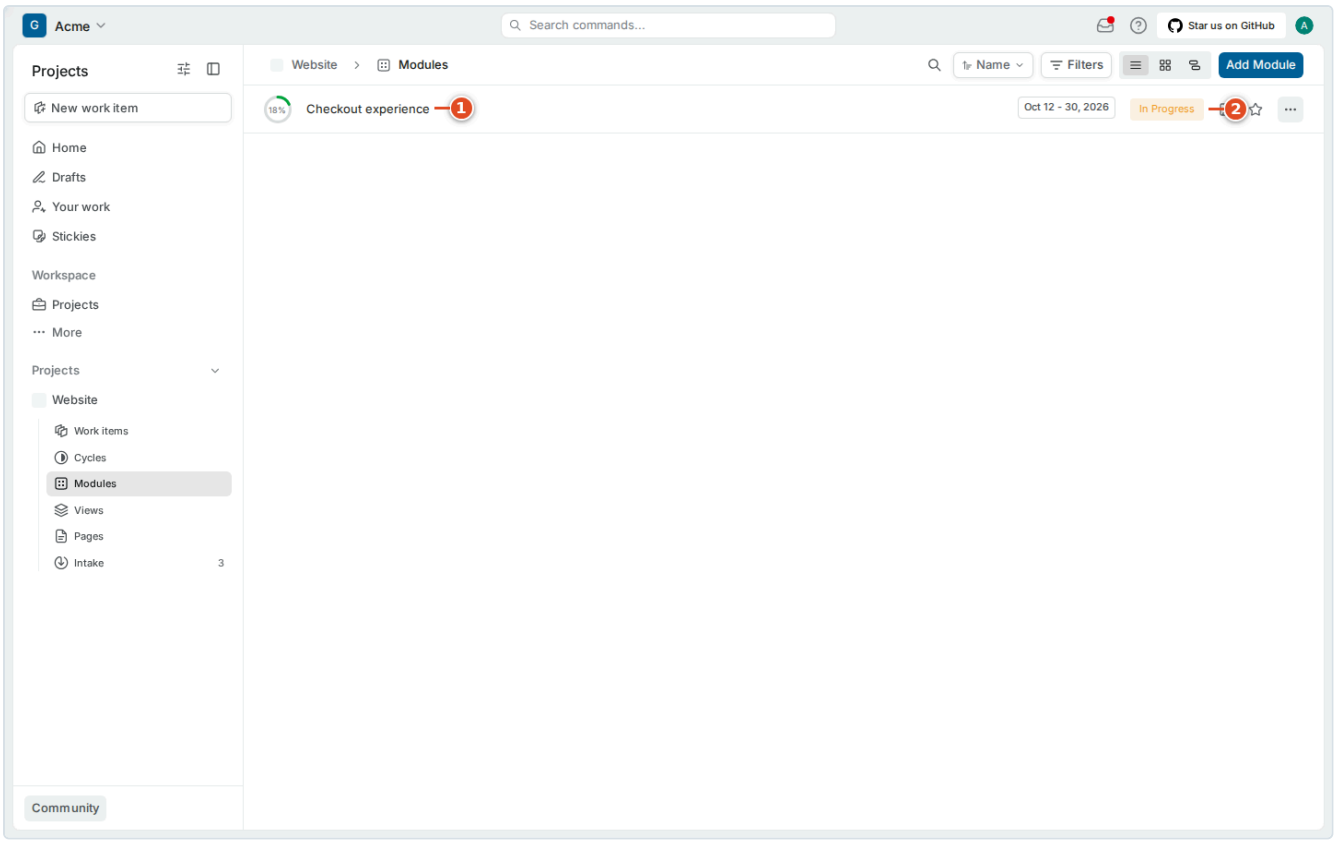
Cycle progress is derived from the work included and its workflow. It can change when items are added, removed or moved between states. Do not interpret a percentage as a promise that the remaining work will take the same proportion of the remaining time. A small unresolved dependency can matter more than several completed minor tasks.

> Tip: During a planning conversation, ask what will move out if an urgent item moves into a committed cycle. Updating the container is easy; making the capacity decision visible is the useful part.

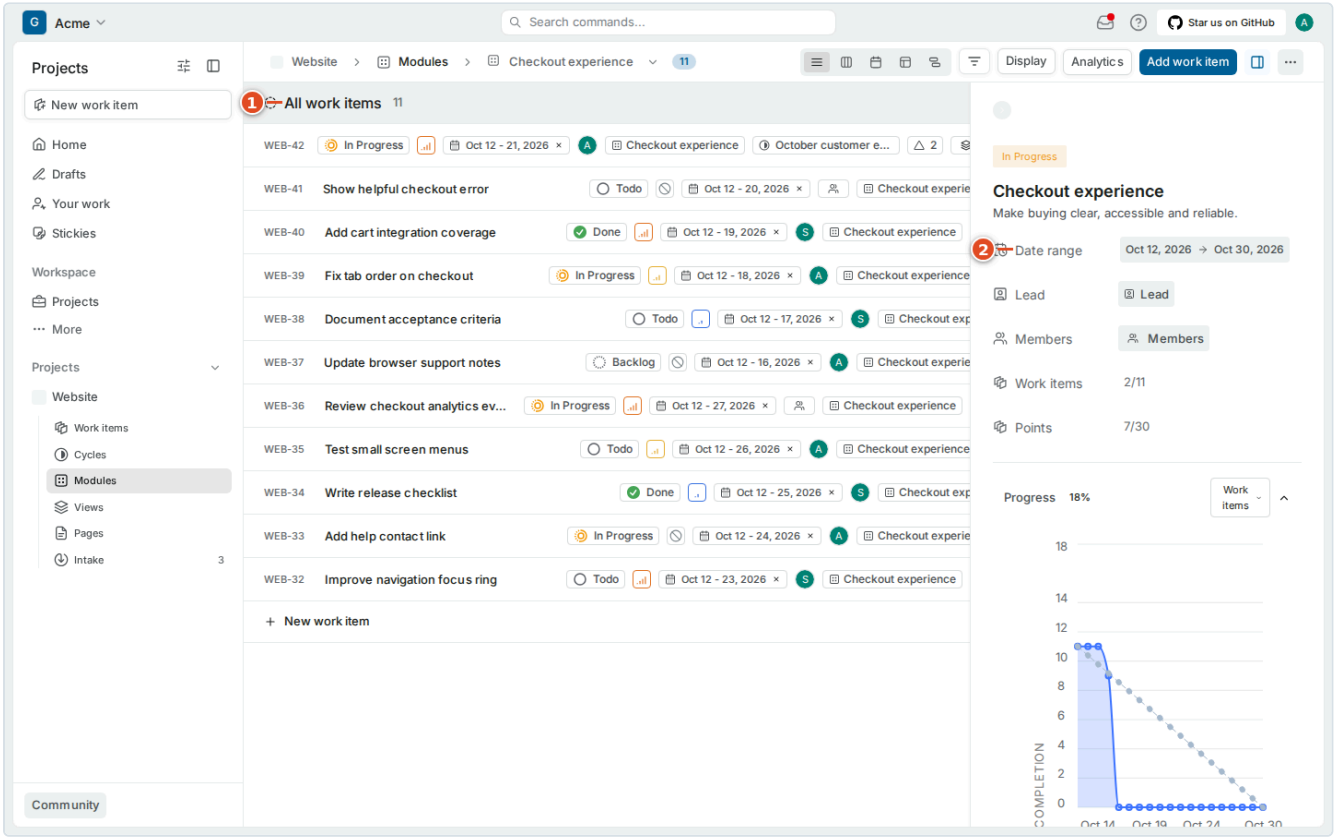
6.2 Use modules for a feature area

A module groups related work around an area or larger outcome. The example **Checkout experience** module contains several Website improvements. It is a useful view of progress across tasks that may span more than one cycle. Its status and dates describe the module’s plan, while each included item retains its own workflow state.

1. Open **Modules** under the project.
2. Find the module relevant to your task.
3. Open the module.
4. Read its description and properties.
5. Review its included work items.



Modules: (1) Checkout experience; (2) In Progress status.



Inside a module: (1) included work items; (2) module properties and progress.

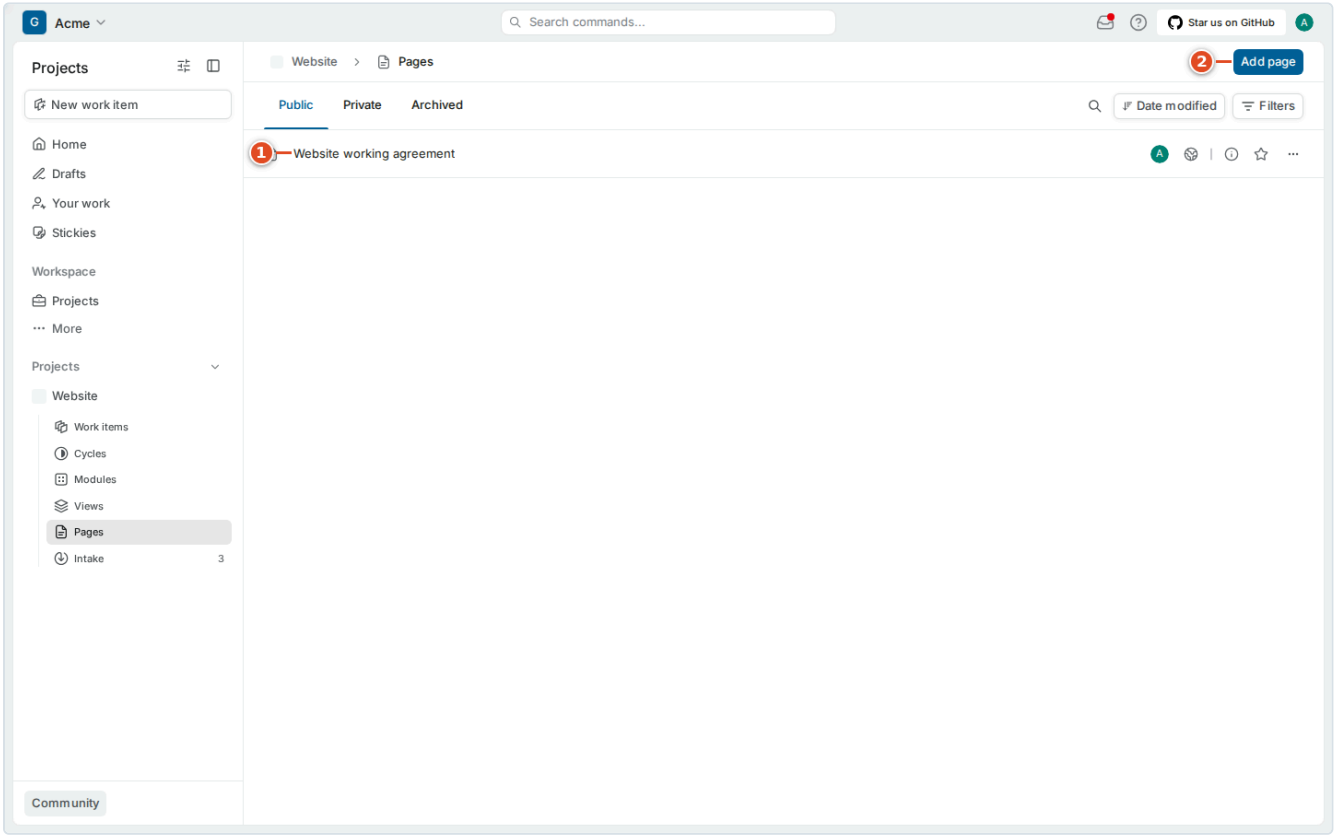
Use the work item's **Modules** property to add it to an appropriate module. An item can contribute to a broader area without being a sub-work item of every task in that module. A parent relationship describes decomposition; a module describes a planning collection. Avoid creating a very large parent merely to reproduce what a module already communicates.

If your team asks you to create a module, give it a useful name and a description of the outcome or area it covers. Add members and dates where your team's planning practice calls for them. Do not assume that marking a module complete automatically merges PRs or completes its unfinished work items.

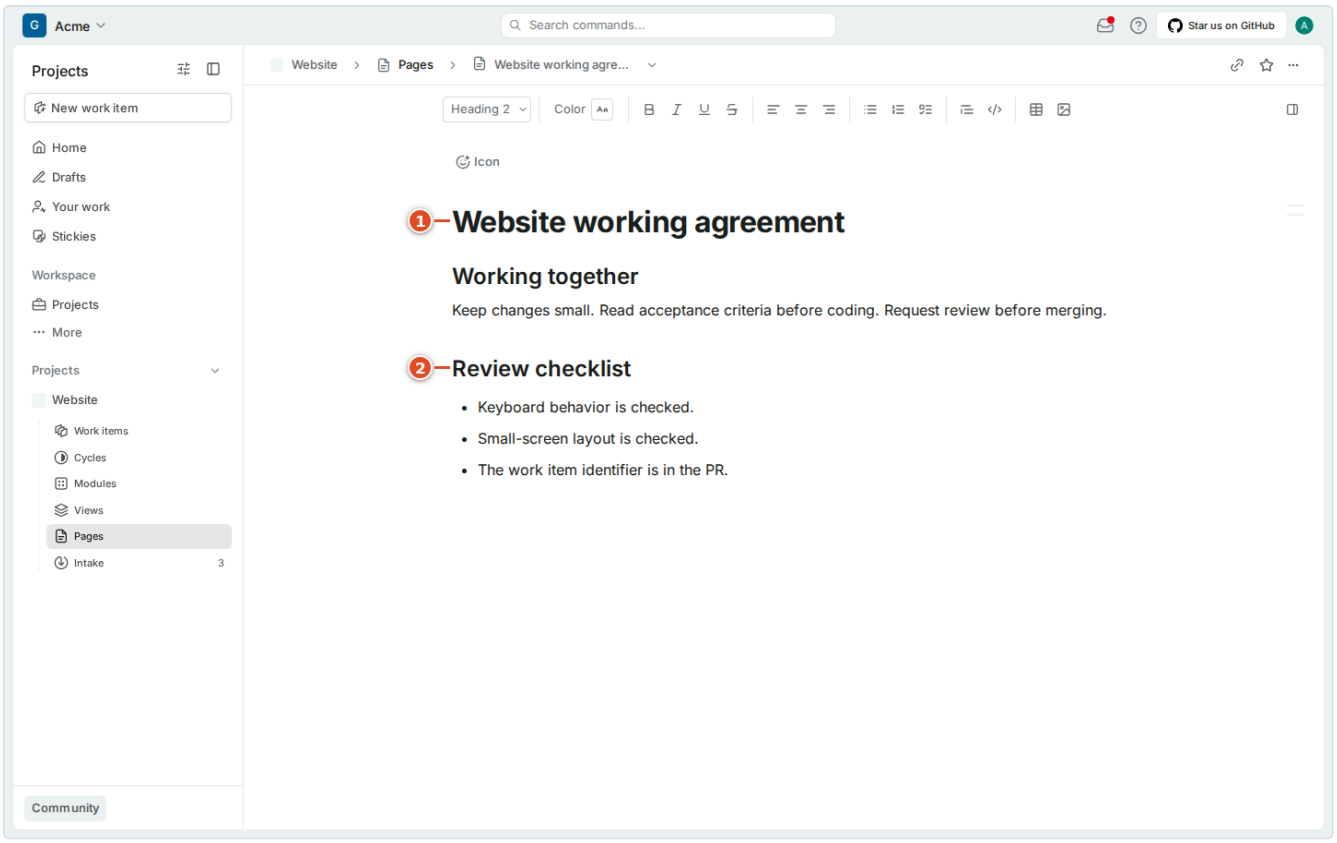
6.3 Keep team knowledge in Pages

A page is a shared document. It is useful for working agreements, design notes, review checklists and onboarding instructions that should outlive a single task. A work item's description should contain its own acceptance criteria, but it can link to a page describing a convention used by many items.

1. Open **Pages** under the project.
2. Find the document you need.
3. Open it.
4. Read its title and context before following its instructions.



Pages: (1) the shared working agreement; (2) Add page.



Shared page: (1) document title; (2) review checklist.

When creating a page, begin with a clear title and a short explanation of its intended audience. Use headings to separate topics. Keep a checklist small enough that a teammate will actually use it during the activity it supports. Link back to a work item for a specific implementation decision instead of duplicating a long comment thread.

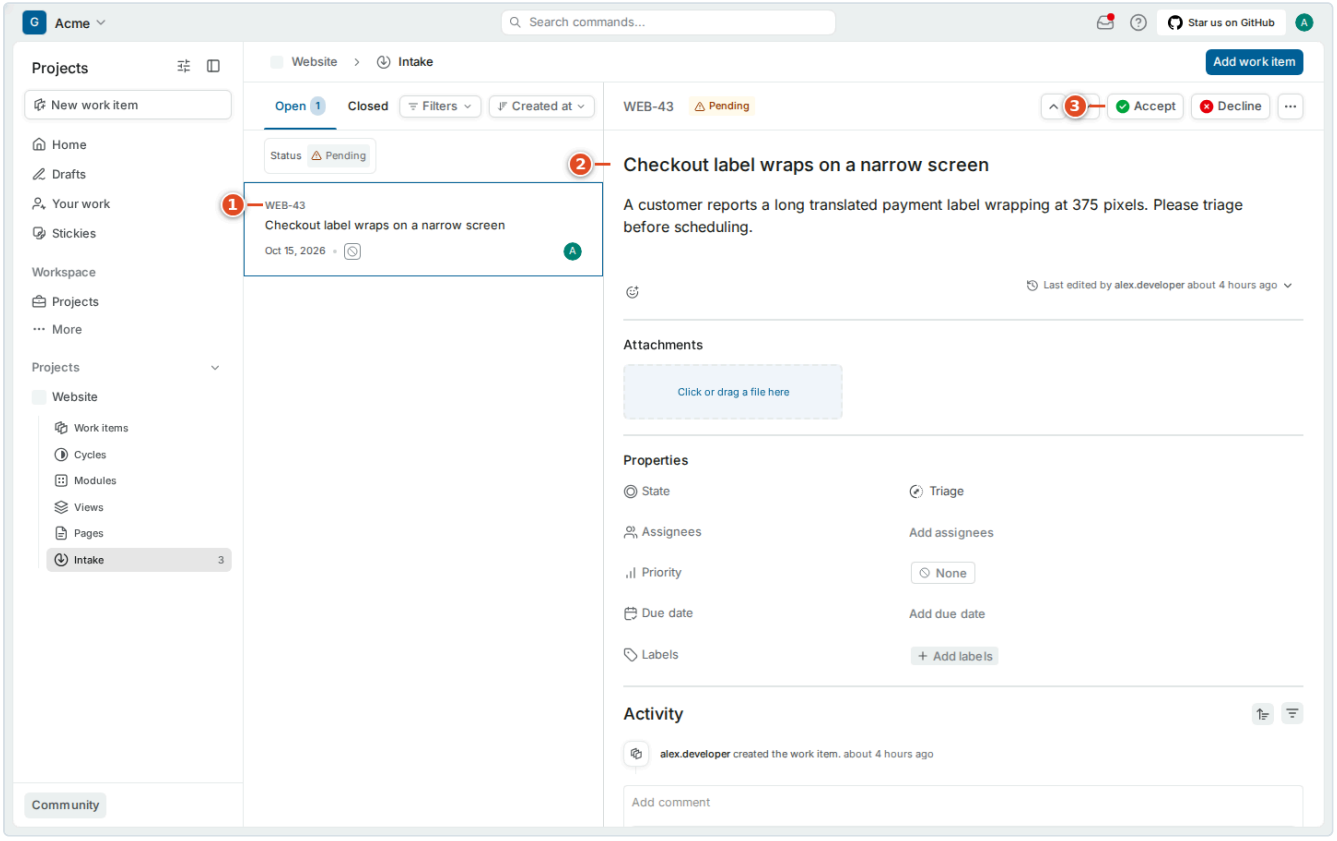
Page actions may include favorites, locking, archiving and sharing controls according to your role. A shortcut appearing in the keyboard dialog does not prove that every sharing action is available to your account. Check the actual page controls and permissions. Public sharing can expose content outside the team; use your organization’s policy before making a page public. This handbook does not require public page publication.

The captured CE editor and project page list support the ordinary documentation workflow described here. No AI-writing capability, paid knowledge-management feature or advanced publishing workflow is assumed to be included merely because newer Plane documentation mentions it.

6.4 Triage new requests through Intake

Triage means deciding what to do with a new report before treating it as planned work. Intake separates incoming bugs, ideas and feedback from the regular project queue. In the captured project, a new request is **Pending** and its work item state is **Triage**. The detail screen offers **Accept** and **Decline** actions.

1. Open **Intake** under the project.
2. Select **Add work item**.
3. Enter a specific report title.
4. Describe the observed behavior and supporting evidence.
5. Select **Create work item**.
6. Open the new request in the pending list.



Intake: (1) pending request WEB-43; (2) report title; (3) Accept action.

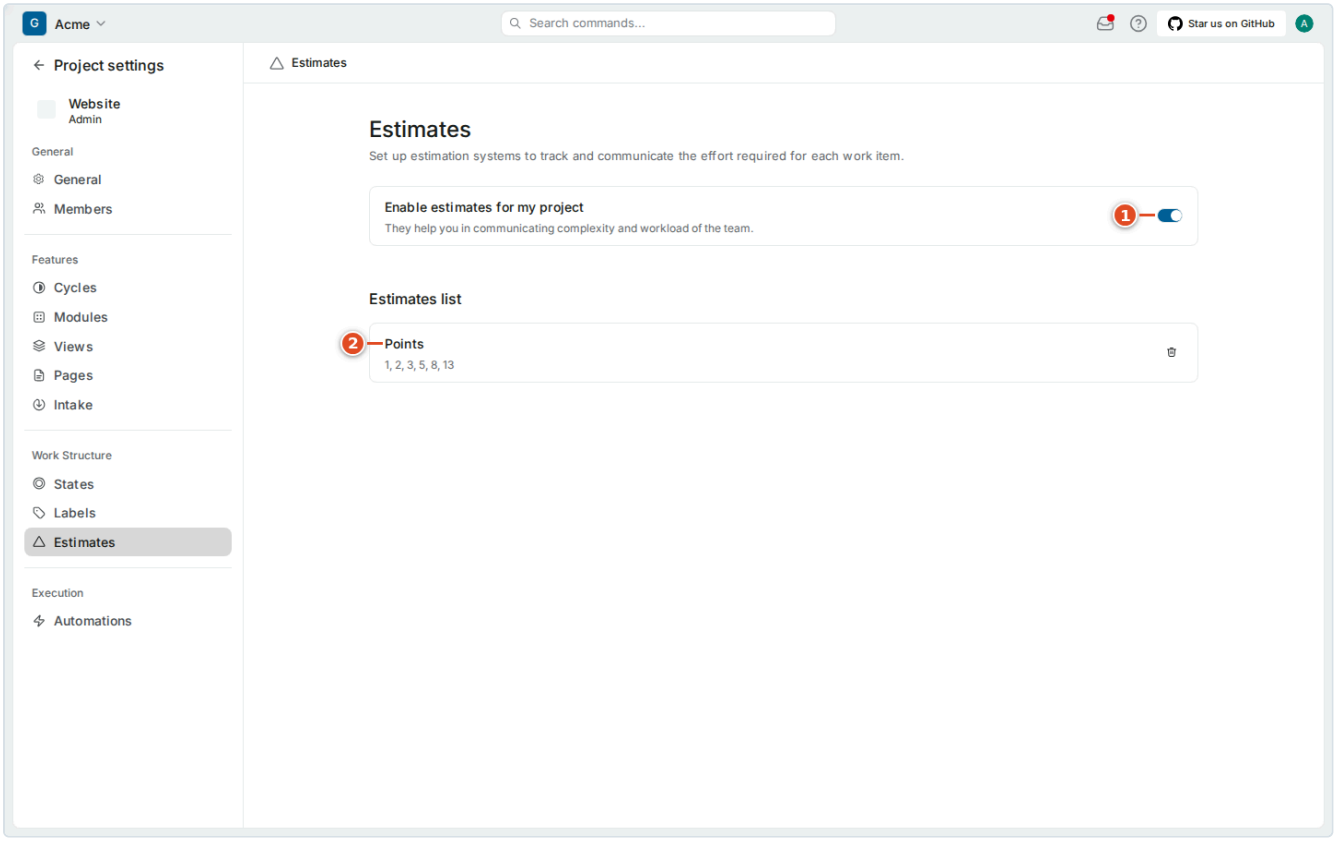
The person handling triage should check for duplicates, clarify the expected outcome and decide whether the team will pursue it. Acceptance belongs to that triage decision; it should not be used simply to clear an unread count. Declining a report is different from completing an implementation. Preserve a useful reason so the reporter knows what happened.

After accepting work, check its regular project state, priority and ownership according to your team’s process. Do not assume that a pending request is already scheduled in the current cycle. In this integration, normal reference resolution excludes triage items, and ordinary sync state changes do not promote work directly from triage. Have the request properly accepted before expecting the regular development automation to apply.

Intake must be enabled for the project. If its navigation entry is missing, ask whether the team uses it and who handles incoming requests. Do not create a separate private backlog as a workaround for an intentionally managed triage process.

6.5 Understand the project’s estimate system

The administrator’s estimate settings offer point systems and categories. The captured Points templates include Fibonacci values `1, 2, 3, 5, 8, 13`, Linear values `1, 2, 3, 4, 5, 6`, and Squares values `1, 4, 9, 16, 25, 36`, along with a custom option. Your team chooses the scale and what those values mean.



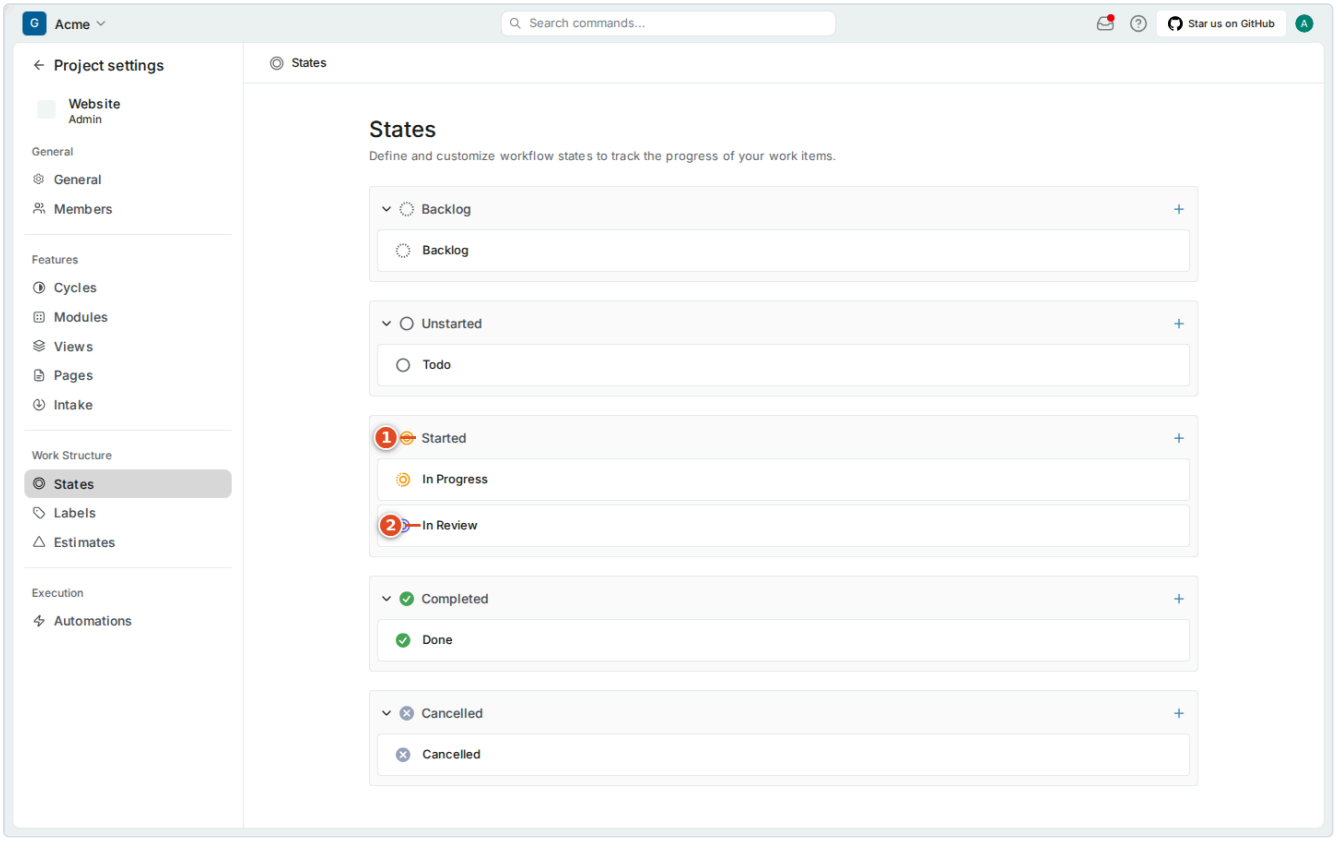
Estimates: (1) project enable control; (2) configured Fibonacci Points scale. Administrator view.

As a member, use the active estimate control on a work item when available. Compare the work with examples your team has already sized. If the team has not defined the scale, ask before converting values into hours in a report. An estimate expresses a planning judgment; recorded elapsed time and a deadline are separate concepts.

Changing the project estimate system can affect how many existing items are interpreted. Treat that as an administrator and team decision. The screenshot shows the setting to explain why a field may be absent; it is not a day-one instruction to reconfigure the whole project.

6.6 Read project states and labels

States and labels are shared vocabulary. The project’s settings organize states by group and list its labels. Reviewing them helps you understand why your team distinguishes In Progress from In Review or frontend from accessibility. Your role determines which settings you may edit.



Project states: (1) Started group; (2) In Review state in that group. Administrator view.

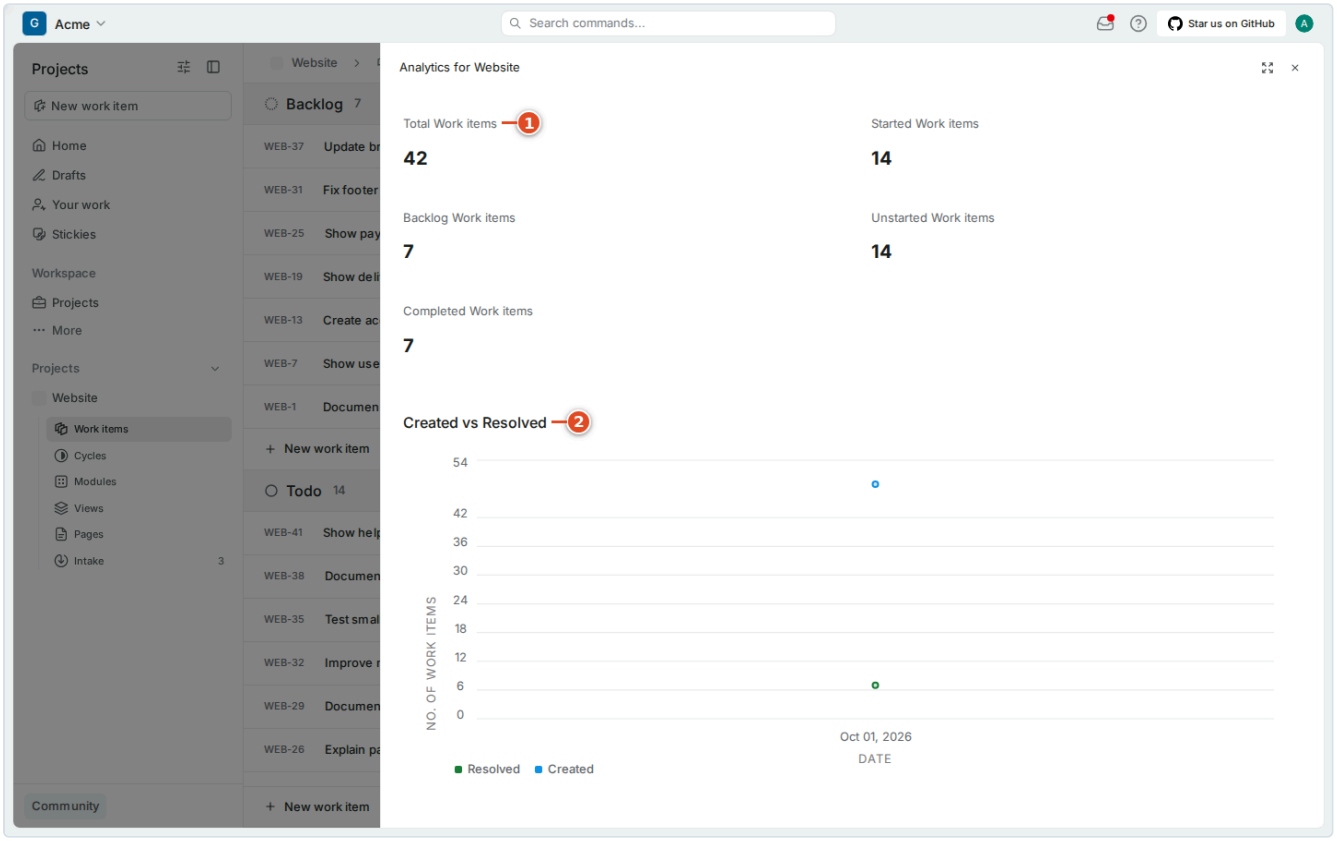
Ask a project administrator before renaming or deleting a state that automation uses. The integration can target a state name and group, so a casual workflow rename can change which action resolves successfully. Similarly, adding many nearly identical labels makes filters and label mapping harder to understand. Prefer the team’s established terms.

The Automations entry in Plane project settings is separate from git-sync’s mapping rules. Do not assume a rule configured in one place is represented in the other. For a GitHub-triggered state change, the git-sync mapping and rule reference in chapters 9–11 is the relevant starting point.

6.7 Use the analytics that are actually present

The captured CE project has an **Analytics** button in the Work items header. It opens a project summary containing total work items, counts by workflow group, a Created vs Resolved chart, customized breakdown controls, and priority and assignee tables. Export-as-CSV controls appear with the tables. This is observed running functionality, rather than a claim based on a paid-edition feature list.

1. Open the project's **Work items** page.
2. Select **Analytics**.
3. Read the scope and chart labels.
4. Compare the summary with the project records before drawing a conclusion.



Project analytics: (1) total work-item summary; (2) Created vs Resolved chart.

Counts explain a selection of records; they do not establish individual productivity or the quality of completed work. A developer can finish an important investigation with one item while another change needs many small records. Use charts to ask useful planning questions, such as why started work is accumulating or whether review capacity is sufficient.

Workspace-level Analytics is also present in workspace navigation. Check its broader scope before comparing its numbers with a single project. This handbook does not assume that every custom dashboard or advanced reporting feature advertised for another edition is available in CE 1.4.2.

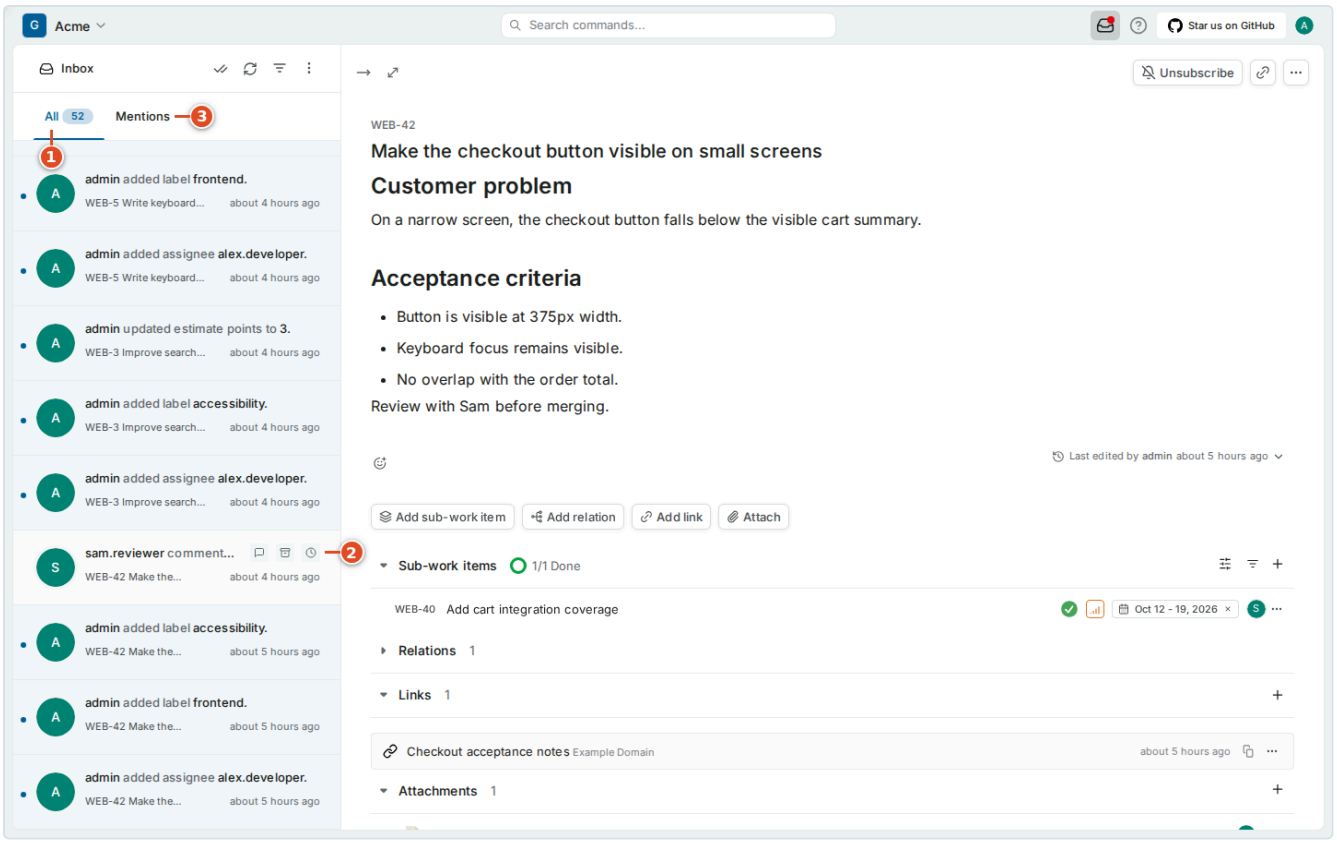
07. Notifications and staying informed

The inbox is a place to notice relevant changes without rereading every item in every project. A notification usually needs one of three responses: read it for context, answer a question, or take responsibility for a next action. Opening the notification is only the first step; decide which response is needed.

7.1 Open the inbox

The inbox control is in the top bar. Plane also lists **G then X** as the shortcut for going to the inbox. Your own notifications depend on your account’s assignments, subscriptions and activity, so your list will differ from a teammate’s.

1. Select the inbox icon in the top bar.
2. Review the currently selected notification category.
3. Open the relevant notification.
4. Read the work item and surrounding discussion.
5. Respond on the work item if a response is needed.



Inbox: (1) All category; (2) Sam’s comment notification; (3) Mentions category.

A notification can describe an event that has already been followed by another change. Read the current work item before acting on an old state or assignment. If you received a question and somebody else has already resolved it, a concise acknowledgment is more helpful than repeating the answer.

Use the inbox’s read, archive or filtering controls according to the labels shown on your account. Clearing an unread indicator does not complete the work item. An archived notification and an archived work item are different records with different purposes. If a notification disappears from the current selection, inspect its category or filter before assuming the underlying item was deleted.

7.2 Mentions are requests for attention

A mention points a comment or description at a specific member. Type `@` in a supported editor and select the intended person from the suggestion list. Include enough context for that person to answer without reconstructing the entire investigation.

> Example: “Alex, should the button keep its full translated label at 320 pixels, or may it wrap onto two lines?” asks for a decision. “Alex, please look” does not say what decision is needed.

A person mentioned in ordinary text is not necessarily a structured Plane mention. Similarly, a copied GitHub username is not automatically a Plane membership grant. git-sync translates supported mentions using its verified user mapping. When it cannot map a GitHub login, it can preserve a profile link rather than pretending it identified a Plane member.

Use mentions sparingly enough that they remain useful. Do not mention a teammate merely to make a routine edit seem urgent. If the work is urgent, set and discuss its priority through the team’s normal process.

7.3 Subscribe to work you need to follow

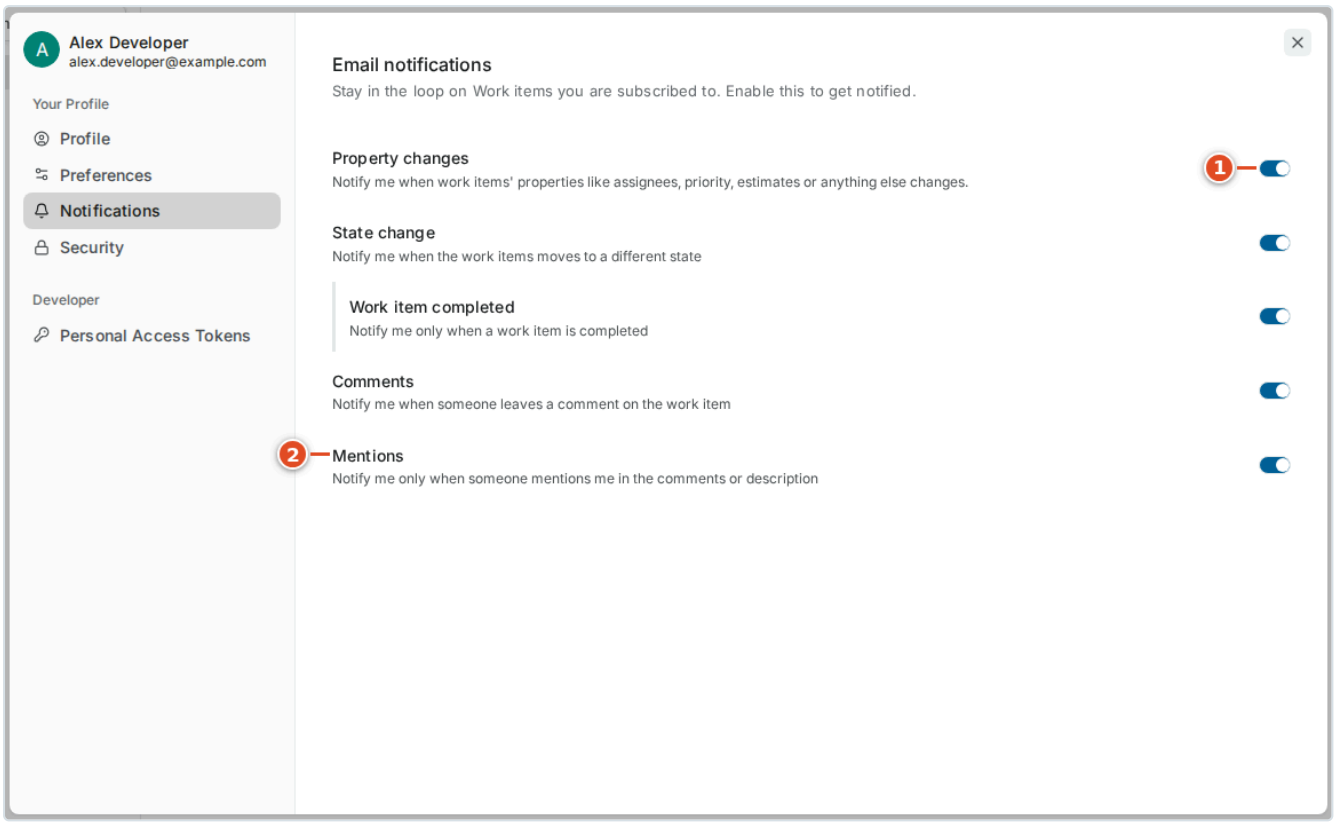
Assignment describes responsibility. Subscription describes interest in updates. You may subscribe to a dependency, a review task or a cross-team change even when you are not implementing it. The control on an unsubscribed item reads **Subscribe**; after subscribing it reads **Unsubscribe**.

1. Open the item you need to follow.
2. Select **Subscribe** near its header.
3. Confirm the control changes to **Unsubscribe**.

If the item stops being relevant, unsubscribe using the same control. Removing a subscription does not unassign the item or remove your comments. When tracking several dependencies, a saved view can complement notifications by giving you a deliberate place to review their current state together.

7.4 Understand email notifications

Personal Settings → Notifications offers email categories for property changes, state changes, work-item completion, comments and mentions. Review those descriptions before changing switches. If comments are useful but every property edit is noisy, adjust the categories separately.



Email notifications: (1) property-change switch; (2) Mentions category and its switch.

Email additionally depends on the server’s mail configuration. The handbook instance has SMTP disabled, so actual delivery to a mailbox was not tested. This does not prevent the book from showing the real preference controls and in-app notifications. Your administrator should verify outgoing email for your team’s instance.

When an expected message is missing, first check whether the underlying event appears on the work item. Then inspect your subscription, account preferences and mailbox filtering. If the event is present but mail is consistently absent, give an administrator one concrete example with its item identifier, approximate time and expected category. Include no password or token.

7.5 Keep notification habits sustainable

As a suggested team habit, check direct questions before starting a long coding session and again before finishing work for the day. Use comments to leave a clear next action when handing something over. A teammate in another timezone should be able to identify what you checked, what remains uncertain and who needs to decide.

An inbox is a prompt to return to the shared record. Keep the decision on the work item, the technical code discussion on the PR, and a useful link between them. That way the next developer can reconstruct the outcome even after the notification has been marked read.

08 · Connect your personal GitHub account

Your Plane account tells the team who owns work. Your GitHub account tells the team who changes code. Connecting them lets git-sync recognize the same person on both sides. You still sign in to Plane and GitHub separately. The connection is an authorization for the integration to use your GitHub identity where the repository and App permissions allow it.

Before you start, an administrator should already have installed the GitHub App, connected its installation to your Plane workspace, mapped a repository to a project, and chosen the team's automation rules. Those are shared settings. Their installation and configuration instructions are in the administrator user guide. This chapter covers your personal connection after that setup is complete.

Three connections that are easy to confuse

A **workspace** is the place containing your team's Plane projects. A **repository** is a GitHub code project. A **GitHub App installation** gives git-sync access to a selected GitHub account's repositories. Your **personal account link** identifies you within that arrangement. None of these names mean that all repositories on your GitHub account automatically become connected.

Connection	Who manages it	What it provides
GitHub App installed on an organization or personal account	GitHub organization owner or account owner	Repository access for the App
Installation connected to a Plane workspace	Workspace administrator, sometimes with an owner's handoff	Approval for that workspace to use the installation
Repository mapped to a Plane project	Workspace administrator	The location and policies for sync
Your Plane user linked to your GitHub account	You, as a workspace member	Your identity for attribution and user mapping

The last row is your day-one task. You do not need to create a GitHub App or copy an App private key. If someone asks you to paste the organization's webhook secret into a work item, stop and ask your administrator for the intended setup procedure. Those credentials belong in administrator configuration, not in task descriptions or screenshots.

Find the member account screen

The actual screen is called **My GitHub account**. Its card heading repeats that name. The source is `src/ui/client/Connect.tsx`, and its member route is registered in `src/ui/client/router.ts` and `src/ui/api.ts`.

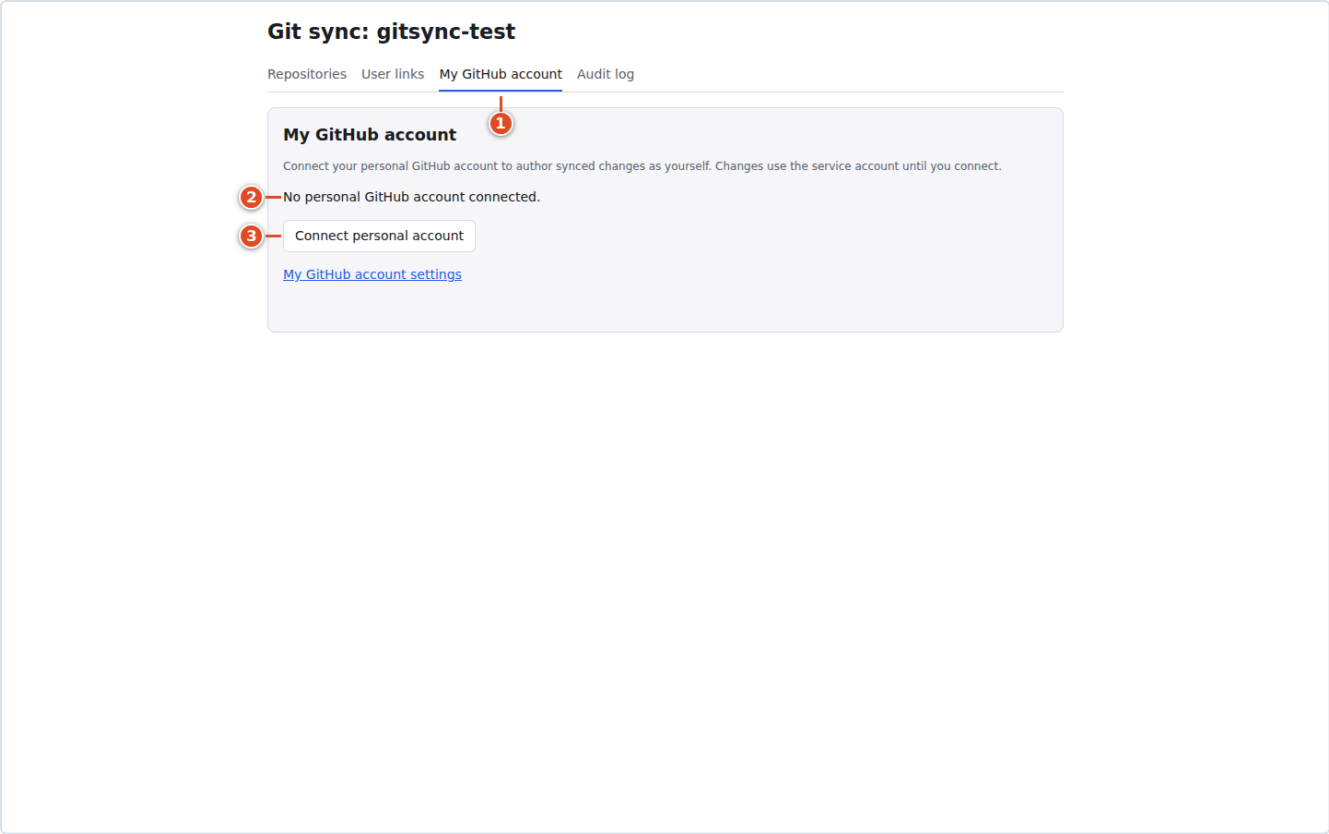
Open it from Plane itself: **Workspace settings → Developer → GitHub Sync**. Workspace administrators see the full integration there; ordinary workspace **members** see only their own **My GitHub account** page. Guests do not get the page. You can also open the page directly. For the fictional workspace in this handbook the direct link is:

```
https://plane.example.com/git-sync/ui/workspaces/acme/account
```

Replace `plane.example.com` with your team's Plane hostname and `acme` with the workspace slug supplied by your team. A slug is the short workspace name used in URLs. This direct link uses your existing Plane session; it is not a separate registration site.

If you are an administrator, GitHub Sync opens with the integration's tabs: **Repositories**, **User links**, **My GitHub account**, and **Audit log**. Choose **My GitHub account**. Members see the account page straight away, without the management tabs.

> **Tip:** Bookmark your member account page. It is useful later if a token expires or you change which GitHub account you use for work.



Personal GitHub account: (1) member account tab; (2) current connection status; (3) Connect personal account.

Connect for the first time

Start while signed in to the intended Plane workspace and to the GitHub account your team expects. Some developers have both personal and employer accounts. The username shown by GitHub during authorization is the useful check; your local git author email does not choose the linked account.

1. Open your team's **My GitHub account** link.
2. Read the connection status above the button.
3. Select **Connect personal account**.
4. Check the account shown on GitHub's authorization page.
5. Approve the GitHub App authorization.
6. Read the result on the returned Plane integration page.
7. Confirm that **GitHub account:** shows your expected username.

The successful return message is **Your personal GitHub account is connected.** If the page says **No personal GitHub account connected**, the flow has not finished. The application leaves its embedded frame for authorization, so a full-page navigation to GitHub is expected.

GitHub may show organization approval requirements according to your organization's policies. A personal link cannot grant App access to a repository that the organization's installation does not cover. If authorization completes but your repository is still unavailable, give the administrator the repository name and the exact visible error. Repeating the same authorization cannot add a missing repository to an installation.

The service stores the GitHub account's stable numeric ID as well as its current login. That distinction matters when a username changes: the link represents the account, rather than simply trusting whoever later owns an old username. You do not need to know or enter that number.

What your connection changes

For supported changes originating in Plane, git-sync can use your GitHub user authorization to attribute activity to you and the App. Automated creation and cross-link steps can still appear under the App or service identity. If your token is absent, expired beyond recovery, or refused for the repository, the supported GitHub write path falls back to the installation identity. A bot name on a synced comment is therefore not proof that synchronization failed.

Your connection also helps the integration translate assigned people and mentions. For example, a mapped GitHub mention of `@alex-dev` can become a Plane user mention for Alex. An unmapped GitHub login becomes a profile link instead. Linking does not give Alex membership in a private Plane project or make an inaccessible GitHub repository readable in the browser.

There is a second credential on the Plane side: a personal access token, often shortened to **PAT**. A GitHub OAuth connection does not create a Plane PAT. GitHub-to-Plane automation can write as your Plane user when the integration has a stored Plane token. With no stored Plane PAT, it uses the service identity. If a stored PAT is present but Plane rejects it, the write is not automatically retried with higher service privileges. Follow your team's administrator procedure if they support personal Plane attribution; the member connection screen described here does not contain a Plane PAT entry field.

> **Warning:** A Plane PAT carries its owner's Plane permissions. Do not paste one into a commit, comment, chat screenshot, or this handbook's example commands. You never need a real token to write `WEB-42 #comment ...`.

A practical way to read attribution is to separate **who caused the event** from **which credential performed the write**. A push caused by your authenticated GitHub account can be recognized as yours, while the resulting Plane comment uses the service credential. Smart Commit comments include commit provenance so readers can still understand the source.

Reconnect when authorization needs attention

The service attempts to refresh expiring GitHub authorization. When it cannot obtain a usable token, the account screen can show **Needs re-authorization. Reconnect your GitHub account to author changes as yourself.** It also explains that changes use the service account until you connect.

1. Open **My GitHub account**.
2. Select **Reconnect**.
3. Check the account on GitHub’s authorization page.
4. Approve the authorization.
5. Confirm your username on the returned account card.

Reconnect is also the button shown for an already linked account. A stored identity row and a usable token are separate things; seeing your username alone does not guarantee the token is currently usable. Read the alert as well as the username.

Disconnect your personal account

Disconnecting your account is a personal action. The workspace’s separate **Disconnect** button on an organization card disables that installation’s mappings; that is an administrator action with a different effect.

1. Open **My GitHub account**.
2. Select **Disconnect personal account**.
3. Read the confirmation about service-account writes.
4. Confirm the disconnect.
5. Check that the personal account card is no longer connected.

The service attempts to revoke the GitHub authorization and removes its locally stored GitHub token even if remote revocation fails. It removes your user link in this workspace and records your choice so automatic email matching does not immediately link you again. The current implementation stores GitHub tokens per Plane user, so removing that token also stops personal GitHub attribution in other workspaces until you authorize again. Your stored Plane PAT, if your team configured one, is separate and is retained by this personal GitHub disconnect.

Existing branches, commits, pull requests, and comments are not deleted. Repository mappings also remain. Subsequent supported writes can use the service account. If your intention is to stop the team’s entire repository integration, ask a workspace administrator; disconnecting your personal account does not accomplish that.

Read connection errors without guessing

Message or symptom	Meaning and next step
Button disabled; App OAuth credentials are missing	An instance administrator must configure the App client ID and secret
Connection link expired, already used, or belongs to another Plane account	Start from **Connect personal account** again in the intended Plane session
GitHub authorization was declined	Start again when ready to approve
GitHub account already linked to another member	Ask an administrator to investigate the existing identity link
Authorization exchange could not complete	Restart once; report repeated failures with the visible message
Workspace membership required	Sign in to the intended workspace or ask for membership
Personal link succeeds but repository access fails	Ask the administrator to check installation coverage and mapping health

An administrator's **User links** page can also hold manually configured identity links. That is not a reason to try random accounts until one works. Give the administrator your intended GitHub username and workspace name. They can verify the mapping without receiving your GitHub password.

A first connection example

Alex joins the `acme` workspace and receives the member account URL. The card initially says no personal account is connected. Alex selects **Connect personal account**, checks that GitHub is authorizing `alex-dev`, and approves. Back in Plane, the card displays `alex-dev`. Later, Sam reviews Alex's PR. Sam needs their own link; Alex's connection does not map the entire team.

Alex then writes a new comment on a mirrored Plane work item. If the repository and authorization allow it, the GitHub copy can be attributed through Alex's user authorization. A separately generated linkback comment may still come from the App. This is expected. The daily check is whether the correct work item, repository, and content are connected, with attribution interpreted according to the available credentials.

Implementation references for this chapter: `src/ui/client/Connect.tsx`, `src/ui/client/Workspace.tsx`, `src/ui/oauth-flow.ts`, `src/ui/install-flow.ts`, `src/auth/identity.ts`, `src/ui/api.ts`, and `apps/web/git-sync.patch`.

The member route, disconnected card and connection controls were verified in the running interface. Successful interactive GitHub OAuth authorization and subsequent disconnect were not exercised during capture; their steps above follow the current client and server implementation.

09 · Link branches, commits, and pull requests

A work item's identifier is its short address in code discussions. `WEB-42` means work item 42 in the project whose identifier is `WEB`. Putting that address in a branch or pull request lets git-sync connect the code to the task. The connection can also permit automation, so the wording deserves the same care as choosing the task's state by hand.

This chapter describes the current repository implementation. Your administrator chooses which repository is mapped, which rules are active, and which magic words the instance recognizes. The examples use the shipped default word lists. A link alone does not guarantee a state change when no matching rule is configured.

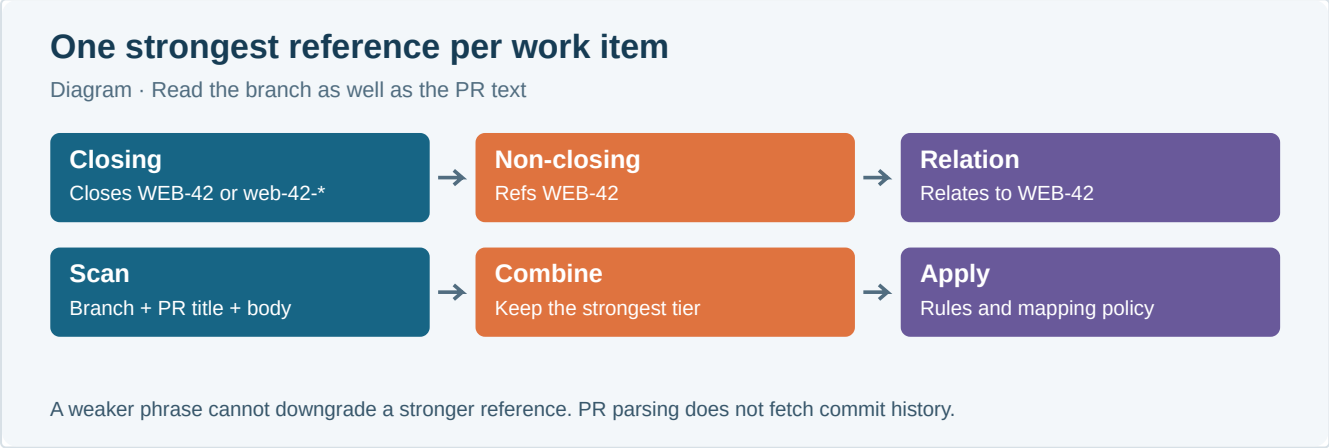


Diagram: branch, PR title and body contribute references; the strongest tier wins for each work item.

Choose the right kind of reference

There are three strengths of reference, called **tiers** in the implementation. A tier says how much automation the connection permits. It is separate from the work item's current state.

Tier	Reader-friendly meaning	Automation allowed
Closing	This code is intended to finish the item	Configured state changes, including the merge state when the multi-PR condition allows it
Non-closing	This code contributes part of the work	Configured state changes before merge; merge-state changes are skipped

Tier	Reader-friendly meaning	Automation allowed
Relation	This code is useful context	Link information; automatic state changes for that reference are skipped

For example, `Closes WEB-42` is a closing reference. `Refs WEB-42` is non-closing. `Relates to WEB-42` is a relation. These phrases do not themselves name a Plane state. A team may choose In Progress for an opened PR and Done for a merged PR, while another team chooses different states or different rules based on the destination branch.

These tiers gate state changes in the current executor. Other actions in an administrator rule, such as posting a comment or adding a label, can still run for a relation or non-closing reference. “Relation” is therefore safest read as “do not change the workflow state,” rather than a promise that every configured automation action is disabled.

A common surprise is that a bare identifier in a branch or PR title is already **closing**. If a PR should only contribute part of an item, write a non-closing phrase and check the branch too. A closing branch reference can outrank a carefully worded non-closing body.

Exact identifier rules

The parser in `src/parse/references.ts` recognizes a project prefix beginning with a letter followed by 1–11 letters or digits, then a hyphen, then decimal digits. Thus the prefix is 2–12 characters long. It resolves the resulting identifier through Plane; recognizing the text does not prove that the work item exists or that the service can read it.

Text	Recognition rule
`WEB-42`	Valid identifier shape
`API2-103`	Valid identifier shape
`X-42`	Ignored: one-character project prefix
`42-WEB`	Ignored: wrong order
`WEB_42`	Ignored: underscore instead of hyphen
`WEB-42_extra`	Ignored: an underscore immediately after the digits prevents a match
`WEB-42more`	Ignored: a letter immediately after the digits prevents a match

Text	Recognition rule
`feature/WEB-42-login`	Recognizes `WEB-42`; slash and the following hyphen are acceptable boundaries
`xWEB-42`	Can represent prefix `xWEB` in a case-insensitive context; it does not identify `WEB-42`
`WEB-0042`	Normalized to `WEB-42`

An immediately preceding letter or digit prevents a match from starting at that position. A following letter, digit, or underscore prevents it from ending there. Do not rely on an identifier buried inside a longer technical token. Use a slash, space, parenthesis, or hyphen boundary and an ordinary key copied from Plane.

The parser has a small fixed deny-list for technical tokens: `UTF`, `SHA`, `ISO`, `AES`, `RSA`, `CVE`, `CWE`, `RFC`, `TLS`, `SSL`, `MD`, `ES`, `PEP`, `USB`, `HTTP`, `IPV`, `SSH`, `GPT`, `RS`, `CRC`, `UTC`, and `WIN`. These prefixes are ignored even after a magic word in the current implementation. If your real project uses one, ask the administrator for a compatible project identifier. Do not assume `Fixes RFC-42` bypasses the deny-list.

Where identifiers are found

The general parser accepts branch, title, body, and commit-message inputs. Different event handlers provide different inputs. That distinction is useful when a commit seems related but the PR has no connection.

Location	Bare key case	Bare tier	Current use
PR head branch name	Case-insensitive	Closing	PR linkage and relevant push processing
PR title	Uppercase prefix only	Closing	PR linkage
PR body	Uppercase prefix only	Relation	PR linkage
Commit message	Uppercase prefix only	Relation	Push rules and Smart Commit key discovery

Location	Bare key case	Bare tier	Current use
Key after a configured magic phrase	Case-insensitive	Phrase’s tier	Supported parser inputs supplied by the relevant handler

The head branch is the branch containing your changes. The base branch is the branch receiving them. A branch-aware rule can inspect the **base** branch, such as `main` or `staging`, but the PR’s identifier scan uses its **head** branch.

The current PR handler scans the head branch, title, and body. It does not fetch the PR’s commit history and feed those messages into the PR reference parser. Commit text is processed on push paths, including Smart Commits. Put the work-item key in the PR itself or its head branch whenever you want a durable PR connection. A key that exists only in an old commit is a poor substitute.

Case rules explain several apparently inconsistent examples:

```
Branch: feature/web-42-login       -> closing WEB-42
PR title: WEB-42 Improve login     -> closing WEB-42
PR title: web-42 Improve login     -> no bare match
PR body: Related discussion WEB-42 -> relation WEB-42
PR body: Related discussion web-42 -> no bare match
PR body: Refs web-42               -> non-closing WEB-42
```

Magic phrases are case-insensitive. `REFS web-42`, `Refs WEB-42`, and `refs Web-42` all carry the same default non-closing meaning. A bare lowercase key in a body does not.

Name a branch deliberately

A useful team convention is a key followed by a short explanation: `web-42-fix-login-redirect`. That is a suggestion, not a mandatory global naming format. The Development panel offers the administrator’s actual configured format.

The branch helper’s default template is `{key}-{title-slug}`. Supported placeholders are `{key}`, `{title-slug}`, and `{username}`. A configured template must include `{key}` in a position the parser can recover. A title slug is a shortened, lowercase version of the work-item title with words separated by hyphens. A missing username is omitted with its nearby separator.

1. Open the intended Plane work item.
2. Find **Development** in its detail sidebar.
3. Locate the intended repository under **Start work**.

4. Select **Copy branch name**.
5. Create that branch in your local checkout.
6. Push the branch to the mapped repository.

For example, after copying a suggested name you might run:

```
git switch -c WEB-42-fix-login-redirect
git push -u origin WEB-42-fix-login-redirect
```

Check the repository before running those commands. A copied name does not choose a local checkout or an `origin` remote for you. The panel's copy button copies text only; its implementation does not change the work-item state. Branch creation or first-push rules may change state when configured, but copying alone is not a promise that work has started.

Project members with sufficient permission may also see **Create branch**. That creates a remote branch from the repository's current default branch using the GitHub App. It does not create a local checkout. If the name already exists, the panel reports **Branch already exists**. An empty repository needs a first commit before a branch can be created. Missing App Contents write permission is an administrator issue.

For partial work, a branch with no work-item key, such as `login-layout-part-one`, plus `Refs WEB-42` in the PR title or body avoids an accidental closing branch reference. `refs/web-42/login-layout` still contains a bare closing key: a slash is not the whitespace required after a magic phrase. Agree on the partial-work convention with the team before relying on it.

Write a PR that says what it finishes

1. Open the pull request in the mapped GitHub repository.
2. Give it a descriptive title.
3. Add `Closes WEB-42` to the body when it completes that item.
4. Check the head branch for other work-item keys.
5. Check the body for unintended references.
6. Save the pull request.
7. Open the Plane work item to inspect **Development**.

The parser scans the whole supplied text; the identifier need not be at the beginning of the title or on the first line of the body. It is not a Markdown-aware parser. Quoted text and code blocks are still text for reference discovery. A copied example containing `WEB-99` can therefore create an unintended relation. Keep unrelated examples generic, or use the body opt-out for a key before the initial link forms.

A single magic phrase can apply to several keys. Commas, whitespace, `and`, and `&` are supported separators:

```
Fixes WEB-42, WEB-43 and API-9
Refs WEB-42 WEB-43
Relates to WEB-42 & API-9
Closes: WEB-42
```

The colon is optional. There must be whitespace between the phrase and the key; use `Closes: WEB-42`, not `Closes:WEB-42`. Parentheses between the phrase and key break that phrase's match: `Refs (WEB-42)` leaves a bare key, whose meaning depends on its location.

Resolve competing references

Each identifier receives the strongest tier found anywhere in the supplied inputs: closing outranks non-closing, which outranks relation. When tiers are equal, the recorded source preference is branch, title, body, then commit. This source preference is bookkeeping; it does not make a relation beat a closing phrase.

Suppose a branch is `web-42-login`, the title is `Refs WEB-42`, and the body says `Relates to WEB-42`. The result is closing because of the branch. If your intention is partial work, remove or change the closing occurrence too. An occurrence already classified by a magic phrase is not separately reclassified as a bare key, so a title containing only `Refs WEB-42` remains non-closing.

In a PR **body**, `skip WEB-42` or `ignore WEB-42` suppresses that key in the current parser result, regardless of where a competing occurrence was found. The words are case-insensitive. This exception is body-only; `skip WEB-42` in a title is not an opt-out. Each key needs its own skip phrase; do not assume `skip WEB-42 WEB-43` suppresses both.

> **Warning:** The running PR handler also loads existing active links from its database. Adding `skip WEB-42` after a link already exists, or deleting the visible key from a PR, does not remove that stored association. Ask an administrator to repair an unintended existing link and check its tier before merging.

Create a work item from a PR

`WEB-NEW` requests a new item in the WEB project. The current parser recognizes the marker in the PR title **or** body; the suffix `NEW` must be uppercase. `web-NEW` is recognized, while `WEB-new` is not. A branch marker or commit marker does not make this request.

The PR handler processes create requests on opened, edited, and reopened events. It uses the PR title as the new item’s name, chooses a started state when one can be resolved, and creates a closing PR association. It does not promise to copy the complete PR body into the new item’s description. Repeated delivery of the same request is guarded so ordinary retries do not create a fresh item each time.

Use this shortcut when the team explicitly accepts PR-first work. If the task already exists, reference its real number instead. The marker identifies a project, not the next number you would like Plane to assign.

Read the Development panel

The panel combines categorized links maintained by git-sync. It can show **Pull requests**, **Builds**, **Deployments**, **Previews**, **Branches**, and **Start work**. A category without links is omitted. Its data reflects recorded events and links; it is not a continuously live GitHub browser.

This version has no separate **Commits** list in the Development panel. Commit connections are visible through Smart Commit provenance, GitHub’s PR commits view, and supported build/status links that can point to a commit. A push containing a bare key can supply a target to configured push rules, but it does not by itself promise a new standalone commit row in Plane. Open the linked PR on GitHub when you need its complete commit history.

Label	Meaning
`OPEN`	PR remains open
`MERGED`	PR merged
`DECLINED`	PR closed without merging
`REVIEW`	No current decisive approval/change request is represented, or reviews could not be read completely
`APPROVAL`	A current approval exists and no reviewer’s standing change request takes precedence
`REJECTED`	At least one reviewer has a standing change request
Build `pass`	All represented runs passed; neutral/skipped check conclusions count as passed

Label	Meaning
Build `fail`	A represented run failed or ended with another failure conclusion
Build `pending`	Runs are incomplete, inconclusive, or absent

For each reviewer, the latest decisive review counts; dismissing it removes that reviewer’s decision. A casual review comment does not erase a previous request for changes. The overall PR badge is `OPEN` if any linked PR is open, otherwise `MERGED` if any merged, otherwise `DECLINED`. That badge summarizes PRs, not the Plane state or production deployment.

A preview is a shortcut to a deployed version for inspection. Supported provider detection includes Vercel, Netlify, Cloudflare Pages, and AWS Amplify environment URLs. A Markdown link whose label ends in `Preview`, such as `[Website Preview](https://preview.example.com)`, can also be detected in supported body inputs. The integration does not deploy the site itself.

Several PRs can link to one work item. At merge automation time, at least one must be merged and none may remain open before merge-state changes proceed. A closed unmerged PR does not block and does not count as a merge. If another PR’s current status cannot be read, the implementation conservatively treats it as open. Inspect the whole linked set before deciding an item is stuck.

Implementation references: `src/parse/references.ts`, `src/parse/magicWords.ts`, `src/app/pr.ts`, `src/app/push.ts`, `src/app/rules.ts`, `src/sync/dev-info.ts`, `src/ui/dev-panel.ts`, and `src/ui/client/DevPanel.tsx`.

Completion is evaluated on a qualifying merge event. If one linked PR merges while another remains open, later closing the second PR unmerged does not rerun the first merge under the shipping defaults. Close obsolete PRs before the final completing merge, or agree an explicit state correction with the team.

10 · Magic words and Smart Commits

Magic words describe a code change's relationship to a task. Smart Commits request an immediate action on a task when a commit is pushed. Both put useful intent next to the code, but they run through different paths. A sentence saying `Closes WEB-42` does not behave like the command `WEB-42 #done`.

A **magic word** qualifies an identifier: "this finishes it," "this contributes to it," or "this is related context." A **Smart Commit** contains a hash command, such as `#comment`, on a line that also names its target work item. Magic words support the PR lifecycle; Smart Commits can add comments, record a textual time note, and select a Plane workflow state at push time.

Default magic-word reference

Administrators can replace any of the three instance-wide word lists. The following is the exact shipped vocabulary in `src/parse/magicWords.ts`. Capitalization does not matter for phrases or the identifier immediately following them.

Tier	All default phrases	Meaning
Closing	`close`, `closes`, `closed`, `closing`	Permit completion automation
Closing	`fix`, `fixes`, `fixed`, `fixing`	Permit completion automation
Closing	`resolve`, `resolves`, `resolved`, `resolving`	Permit completion automation
Closing	`complete`, `completes`, `completed`, `completing`	Permit completion automation
Closing	`implement`, `implements`, `implemented`, `implementing`	Permit completion automation
Non-closing	`ref`, `refs`, `references`	Contribute without merge-state automation
Non-closing	`part of`, `contributes to`, `toward`, `towards`	Contribute without merge-state automation

Tier	All default phrases	Meaning
Relation	`relates to`, `related to`	Link context without automatic state changes for that reference

Multiword phrases allow whitespace between their words. The parser prefers the stronger tier if the same configured phrase appears in more than one list. A phrase must be a separate token: letters, digits, or underscores immediately alongside it can prevent a match. A colon is allowed after the phrase, followed by whitespace before the key.

Example supplied to the parser	Default interpretation
`Fixes WEB-42`	Closing
`FIXES web-42`	Closing, normalized to `WEB-42`
`Completing: WEB-42`	Closing
`Part of WEB-42`	Non-closing
`Contributes to WEB-42 and API-9`	Non-closing for both
`Related to WEB-42, WEB-43`	Relation for both
`Closes:WEB-42`	Phrase does not match; a bare key can still match
`Refs (WEB-42)`	Phrase does not reach the key; bare-key rules apply
`unfixes WEB-42`	`fixes` is not a separate word; bare-key rules apply
`See WEB-42` in a PR body	Bare relation; `see` is not a default magic word
`Addresses WEB-42` in a PR title	Bare closing key; `addresses` is not a default magic word

The last two examples matter because unrecognized wording does not automatically mean “ignored.” A valid bare key remains meaningful. In a PR title a bare uppercase key is closing; in a body or commit message it is a relation. A branch key is closing regardless of letter case. Always inspect all occurrences when the outcome seems stronger than intended.

> **Example:** A PR titled `Refs WEB-42` on branch `login-layout-part-one` can remain non-closing. The same title on branch `web-42-login-layout` has a closing reference because the branch contributes the stronger tier.

What a magic word actually changes

The word establishes the reference tier. The configured rule set supplies the action. A closing reference may let an opened PR move the item to In Progress and a merged PR move it to Done. It does not require those exact state names, and it does not close the item merely because the body was saved.

At merge, closing references still obey the all-linked-PR condition: at least one linked PR must have merged and none can remain open. Closed unmerged PRs neither block nor satisfy the merge requirement. Non-closing references skip merge-state changes. Relation references skip automatic state changes. The current executor gates the `set_state` action this way; configured comments, labels, assignments, links, and time-note actions can still run. An event-level create-item action is also separate. Development links can still show the PR for all three tiers.

For two items on one PR, you can state different intentions on separate lines:

```
Closes WEB-42
Refs WEB-43
Relates to API-9
```

This allows WEB-42 to participate in merge completion, WEB-43 to represent continuing work, and API-9 to provide context. Before saving, ensure the head branch and title do not independently contain a closing occurrence for the partial or related items.

Smart Commit command reference

Smart Commits are enabled per repository mapping. An administrator can require signature-verified commits. If either policy prevents execution, valid syntax alone will not perform an action. These commands are evaluated from pushed commit messages; typing them into a PR body or a Plane comment does not invoke the Smart Commit processor.

Command	Example	Plane effect
`#comment`	`WEB-42 #comment Added redirect tests`	Adds a plain-text comment with commit provenance

Command	Example	Plane effect
`#time`	`WEB-42 #time 1h 30m`	Adds a time-note comment; no CE worklog entry
A workflow-state token	`WEB-42 #in-review`	Requests the matching state in that item's project
Several commands	`WEB-42 #in-review #time 30m #comment Ready for review`	State change, time-note comment, and text comment
Several keys	`WEB-42 API-9 #comment Shared client updated`	Applies the comment to each recognized target

A command starts with `#` at the beginning of a line or after whitespace. Its name begins with a letter and can contain letters, digits, underscores, hyphens, or apostrophes. The reserved commands `comment` and `time` are case-insensitive. Other names are interpreted as workflow-state tokens, not as an unlimited library of git-sync commands.

The general shape of a useful line is:

```
Optional text WEB-42 optional text #command arguments
```

The target keys must appear **before the first command on the same line**. Text before and between them can be ordinary prose. Keys inside the comment text do not become extra targets. Bare keys should use uppercase prefixes; an explicit magic phrase can recognize a lowercase key through the shared reference parser, but uppercase keys are clearer for everyday commands.

Write and push a Smart Commit

1. Confirm the intended work-item identifier in Plane.
2. Make your code changes on the appropriate branch.
3. Stage the intended files.
4. Create a commit with the command on a line naming the target.
5. Push the commit to the mapped repository.
6. Open the Plane item's activity or comments.
7. Check the expected result.

For example:

```
git add src/login.ts
git commit -m "Handle expired login sessions" \
  -m "WEB-42 #in-review #time 45m #comment Ready for review"
git push
```

`src/login.ts` is an illustrative path; stage the real file you changed. The first `-m` supplies the subject and the second supplies a body paragraph. There is no requirement that commands appear in the subject. A separate line in the body is often easier to read and review.

Do not select Done merely to show that one commit is finished. A Smart Commit transition can move an item at push time, before review or merge. If the team closes work only after merge, use a comment or a suitable intermediate state in the commit and a closing reference on the PR.

One line is one command context

A command does not span lines. Every line that performs an action must include its own key before the first hash command. This message has two valid independent command lines:

```
Improve the checkout flow

WEB-42 #in-progress #comment Started redirect handling
API-9 #comment Client needs the new error shape
```

This message does not carry the key down to the second line:

```
WEB-42
#comment Started redirect handling
```

The second line has no key before its first command, so it supplies no Smart Commit entry. Similarly, `#comment WEB-42 added tests` has no target key before its first command. Move the key to the front.

`#comment` consumes the rest of its line. That lets a comment contain issue numbers or text resembling commands without unexpectedly executing them:

```
WEB-42 #comment We discussed #done but need another review
```

The comment text includes `#done`; no Done transition is requested. If you want a transition, write it before `#comment`. A command-looking token before `#comment` is treated as a command, so keep ordinary hashtags inside the comment section.

Choose an exact workflow-state token

States belong to individual Plane projects. `#done` works only if the target project's states resolve that token. There is no global promise that every project calls its completion state Done. The resolver checks three levels in order, stopping at the first level containing matches:

Matching level	Example
Full state name, lowercase, spaces replaced with hyphens	`In Review` matches `#in-review`
Same comparison ignoring hyphens and apostrophes	`Won't Do` can match `#wontdo`
Prefix ending at a hyphen-separated word boundary	`#in` can match `In Review` only when it is the sole match at that level

If one level yields several states, the token is ambiguous and is refused. The resolver does not guess the first item in the state list. For a project containing both In Progress and In Review, `#in` is ambiguous. `#in-review` names the intended state. If the project has a state named In and another named In Review, the exact In match wins before prefix matching is attempted.

Names are case-insensitive after normalization. `#IN-REVIEW` and `#in-review` resolve the same way. Use hyphens for spaces: `#in review` supplies token `in` with trailing text, not the two-word state name. Automation cannot move items into the triage state group. Triage is intake, so an administrator must choose an ordinary workflow destination.

A multi-key line resolves each key against its own project's workflow. `WEB-42 API-9 #in-review` can succeed for WEB-42 and fail for API-9 if only the WEB project has that state. Partial success is possible. Check every target instead of assuming one successful comment proves the entire line worked.

Record time accurately

Plane Community Edition has no worklog API for git-sync to call here. `#time` creates a comment reading **Time logged:** with the supplied duration and its computed minute count. It does not update an estimate, cycle capacity, billed hours, or a dedicated time-tracking report.

Duration	Conversion used by git-sync
`1m`	1 minute
`1h`	60 minutes
`1d`	8 hours, or 480 minutes
`1w`	5 working days, or 2,400 minutes
`1.5h`	90 minutes
`1w 2d 4h 30m`	3,630 minutes
`0.5m`	Rounded to 1 minute

Units are `w`, `d`, `h`, and `m`, case-insensitively. Decimal numbers are accepted and the total is rounded to minutes. Mixed units can be separated by whitespace. Colon notation such as `1:30`, a negative duration, or words such as `two hours` are not the documented duration form.

The command accepts a leading sequence of duration parts. In `#time 1h 30m investigated redirects`, the recognized time is `1h 30m`; the trailing words do not become a separate free-text comment. Add `#comment investigated redirects` when that explanation should be posted. Several `#time` commands on one line are combined into that line's time note.

Time notes are additive comments. Repeating a duration in a new commit records another note, rather than replacing the old one. To correct a mistake, leave a clear corrective comment according to team practice; do not assume editing git history edits a previous Plane note.

More worked examples

Commit message line	Expected result when enabled and authorized
`WEB-42 #comment Added a regression test`	One text comment
`WEB-42 #time 20m #comment Investigated redirect loop`	Time note and separate text comment
`WEB-42 #in-review #comment Ready for Sam`	State transition followed by comment
`WEB-42 WEB-43 #time 30m`	A 30-minute note on each item; it is not divided between them
`WEB-42 #time 15m #time 20m`	One 35-minute time note for that line
`WEB-42 #done #comment Fixed today`	Immediate Done request, then comment
`WEB-42 #comment Looks good #time 1h`	One literal comment including `#time 1h`
`WEB-42 #time`	Malformed time command is reported
`WEB-42 #comment`	Missing comment text is reported
`WEB-42 #assign alex-dev`	Attempts to resolve state token `assign`; no built-in assignment command
`WEB-42 #close`	Attempts a state named or matching `close`; it is not the magic phrase `Closes`
`WEB-42 #in-review` in a PR body	Ordinary PR text; does not invoke Smart Commits

Malformed commands are not an all-or-nothing transaction. Other valid commands and other valid targets can still complete. A missing state is recorded as a failure, while a valid comment later in the same entry can still be applied. Permanent failures and network retries are treated differently so temporary outages do not require you to create duplicate commits.

Attribution, signatures, and repetition

The authenticated GitHub webhook `sender` determines the actor. The processor does not trust the commit author, committer, or pusher metadata as permission proof. Changing `git config user.email` or using `git commit --author` cannot make a Smart Commit act with another person’s Plane permissions.

Bots, GitHub's ghost account, and unmapped accounts use service attribution. A recognized user without a stored Plane PAT also falls back to the service identity. This explains why a correctly parsed command may be visible under a service user even after you connected your personal GitHub account.

The administrator may enable **Require signed commits for Smart Commits**. In that case the integration checks GitHub's `verification.verified` result and ignores commands in commits that do not pass. The author line in `git log` is not that verification result. If GitHub cannot be reached to check verification, the operation is retried rather than assuming the commit is verified.

Commands are guarded by commit SHA within the workspace. Pushing the same SHA again, including through another branch or fork, does not repeat completed commands. A retry resumes unfinished steps. A rebase, amend, cherry-pick, or squash can produce a different SHA; a new SHA can execute commands again. Remove or deliberately review old commands when rewriting a message. "Already pushed once" is not a reliable guard after history changes.

If the repository's **Report problems** setting is enabled, failed Smart Commits can receive a commit comment explaining the problem, and known work items can receive a Plane notice. The setting defaults off, so absence of a public warning does not establish success. Give an administrator the commit URL, target keys, expected action, and visible result; they can inspect the audit log without needing your tokens.

Implementation references: `src/parse/magicWords.ts`, `src/parse/references.ts`, `src/github/smart-commits.ts`, `src/app/push.ts`, `src/auth/identity.ts`, and `src/github/problems.ts`.

11 · What syncs, in which direction, and when

Git-sync connects several kinds of information. A PR can **reference** a Plane work item without copying its fields. An ordinary GitHub issue can instead become a **mirror partner**, where selected fields are copied between the two systems. A Smart Commit performs explicit commands from a pushed message. Keeping these three paths distinct makes most sync behavior easier to predict.

For example, `Closes WEB-42` on a PR permits the team's PR automation for WEB-42. It does not make the PR title replace the Plane item's title. Adding the `plane` label to an ordinary GitHub issue can create a Plane mirror partner whose title and description are synchronized. These are deliberately different relationships.

The behavior in this chapter comes from the implementation, especially `src/sync/mirror.ts`, `src/app/issues.ts`, `src/app/pr.ts`, `src/app/push.ts`, `src/app/rules.ts`, and the design and amendment in `docs/superpowers/specs/2026-10-10-plane-github-sync-design.md`. An administrator's mapping and rules can narrow the behavior described here.

The quick direction map

Starting event	Destination behavior	Main condition
New GitHub issue with `plane` label	Creates a Plane work item	Repository mapping and App access
`plane` label added to an unmirrored GitHub issue	Creates a Plane work item	Label-gated opt-in
Plane work item gains `github` label	Creates a GitHub issue	Two-way mapping
Edited mirrored GitHub issue	Copies eligible fields into Plane	Field policy
Edited mirrored Plane item	Copies eligible fields into GitHub	Two-way mapping and field policy
PR opened, reviewed, merged, or closed	Updates links and runs configured rules	Recognized/stored reference and tier
First push of a keyed branch	Adds branch link; may start work	Matching branch rule for state changes
Pushed Smart Commit	Applies explicit commands in Plane	Enabled mapping and optional verification
Checks or deployment status	Updates recorded development information	Linked work and relevant event handling

“Two-way” means both sources are eligible under the mapping. It does not mean every field has identical ownership, or that every change visible in one product is copied. Some fields do not exist on both sides.

Opt in an ordinary issue

The default gate labels are `plane` on GitHub and `github` on Plane. Label comparisons for these gates are case-insensitive. Their purpose is to request a mirror pair; adding a work-item identifier to prose is a reference, not an equivalent mirror request.

To bring an ordinary GitHub issue into Plane:

1. Open the issue in the mapped repository.
2. Add the `plane` label.

3. Wait for the create operation to finish.
4. Find the new item in the mapped Plane project.
5. Inspect its title, description, and mirror confirmation.

To send a Plane item to GitHub under a two-way mapping:

1. Open the work item in the mapped Plane project.
2. Add the `github` label.
3. Wait for the create operation to finish.
4. Open the resulting GitHub issue.
5. Check its title and description.

If the gate label does not exist, ask the administrator or someone with label-management permission. The integration creates required target-side labels during supported writes; that does not mean every developer can create source labels in every project.

The mirror keeps a durable association between the two item IDs. Removing a gate label later does not delete the pair or turn synchronization off. Likewise, a linked PR is not treated as an ordinary issue mirror. The mirror code explicitly excludes pull requests, preventing a PR edit from unexpectedly renaming its task.

On GitHub-to-Plane creation, title, converted description, mapped assignees, ordinary labels, and a suitable initial state can be supplied. On Plane-to-GitHub creation, the current create path supplies title, converted description with a Plane-origin footer, and labels; it does not pass the Plane assignee list or terminal state as create fields. Later supported assignment/state changes can synchronize them. Check a newly created partner instead of assuming it began with every source property.

The six mirrored fields

Field	GitHub → Plane	Plane → GitHub
Title	Issue title becomes work-item name	Work-item name becomes issue title
Description	Markdown becomes Plane HTML	Plane HTML becomes GitHub Markdown
State	Closed/reopened issue selects mapped state	State group closes or reopens issue
Assignees	GitHub accounts mapped to Plane members	Plane members mapped to GitHub accounts
Labels	Names resolved or created in Plane	Added/removed ordinary labels applied to GitHub
New comments	Added as new Plane comments with provenance	Added as new GitHub comments with provenance

These fields belong to **mirror partners**. A manually added URL, branch reference, or magic-word PR association does not automatically opt into six-field copying. A work item's Development section and its ordinary mirror relationship may coexist, but they answer different questions.

Understand field ownership before editing

The mapping has one conflict policy per field. "Conflict" means both sides could change a value; the policy says which changes are eligible to cross over. The shipped defaults are:

Field	Default policy	Practical effect
Title	GitHub is source of truth	Later GitHub title edits can update Plane; Plane title edits do not update GitHub
Description	GitHub is source of truth	Edit the mirrored description on GitHub by default
State	Last write wins	Supported changes from either side may cross

Field	Default policy	Practical effect
Assignees	Last write wins	Supported mapped assignments may cross
Labels	Last write wins	Supported ordinary label changes may cross
Comments	Last write wins	Eligible new comments may cross in both directions

The available policy values in source are `last_write_wins`, `source_of_truth:github`, and `source_of_truth:plane`. Administrators may change defaults. Ask where your team wants edits made if the two screens disagree.

A source-of-truth policy is an outbound eligibility rule, not an editing lock. Plane may let you edit a title even while its GitHub partner owns title synchronization. That local edit can remain visible until a later eligible GitHub title change arrives. The integration does not prevent you from typing it, and it does not promise to immediately reverse every local edit.

The mapping direction has priority. **GitHub to Plane** (`github_to_plane`) blocks Plane-origin mirror writes entirely. Selecting Plane as source of truth for a field in such a mapping does not create a reverse channel; it can leave neither direction eligible for that field. An administrator should resolve that policy combination.

“Last write wins” also needs care. Delivery order is not a dependable clock. The implementation reads the source object’s current value and compares its `updated_at` timestamp with the delivered snapshot for full-value fields. It avoids applying an older snapshot when a newer current value is available. This is not a collaborative text merge, and it does not provide a human conflict-resolution dialog. Make substantial description edits on the agreed source side and communicate concurrent changes.

Closing and reopening ordinary issues

A mapping may choose explicit Plane **open** and **closed** states. The names are project-specific. GitHub has a simpler open/closed distinction, so several Plane workflow states can correspond to one GitHub state.

Event or state	Mirror result when state policy permits
GitHub issue closed	Chosen closed state; otherwise a completed-group state

Event or state	Mirror result when state policy permits
GitHub issue reopened	Chosen open state; otherwise an unstarted-group state
Open GitHub issue first mirrored	Chosen open state if valid; otherwise Plane's project default
Closed GitHub issue first mirrored	Chosen closed state or completed-group fallback
Plane item enters completed group	GitHub issue closed with reason `completed`
Plane item enters cancelled group	GitHub issue closed with reason `not_planned`
Plane item enters another group	GitHub issue opened with reason `reopened` when the state changes

The fallback resolver prefers a suitable default state in the requested group. If an explicit chosen state was deleted or became unsuitable, the service records a fallback in the audit log. If no appropriate state can be resolved, it does not invent one. Automation refuses to move work into triage.

Moving a task from In Progress to In Review may cause no visible GitHub issue state change because both are normally open groups. That is expected: GitHub's binary issue state cannot display every Plane workflow step. A PR review badge carries different information and should not be used as a substitute for the work item's state.

Closing a GitHub issue as "not planned" does not necessarily choose a cancelled Plane state. The incoming issue path uses the mapping's configured closed state or completed fallback. If your team distinguishes cancelled work, agree on the chosen closed state and the side where that decision should be made.

Assigned people and mentions

People are translated through the workspace user map. A GitHub assignee's stable account ID identifies the linked Plane user. A person without a mapping cannot be assigned by guessing their display name or git email. The result can therefore contain fewer assignees than the source.

For Plane-to-GitHub updates, the service reconciles the mapped part of GitHub's assignee list. It removes mapped accounts that Plane no longer wants, adds mapped accounts Plane requests, and leaves GitHub assignees that Plane cannot represent alone. Clearing the last mapped Plane assignee is a real change, even though the desired set is empty.

A personal account link helps establish this relationship, but it does not override repository eligibility. GitHub can still refuse an assignment or a user-token write when the account lacks appropriate access. Give the administrator both the account name and repository name when assignment differs.

Mentions inside descriptions and comments are converted too. A mapped GitHub `@login` can become a native Plane mention. An unmapped GitHub login becomes a profile link. A Plane mention of a mapped member becomes `@login` on GitHub; an unmapped member is represented by their display name where available. These conversions preserve readable text without manufacturing an identity match.

GitHub issue references such as `#123`, `owner/repo#123`, and full issue URLs can become explicit GitHub links in the converted Plane content. That is prose conversion, separate from recognizing `WEB-42` for automation. Inline code and code blocks are not rewritten by the prose converter, even though the separate work-item reference parser can scan textual keys in supplied PR text.

Labels and comments are not full object copies

Ordinary label names are synchronized, with missing target labels created where supported. The gate labels have special handling to avoid recursively creating mirror pairs. Names, rather than internal label IDs, connect the two systems. Colors and label descriptions are not promised as mirrored fields.

GitHub label events use the source's current label set for the Plane update, excluding the GitHub `plane` gate through the helper logic. Plane label events use the reported additions and removals, resolving label IDs back to project label names. This means label synchronization is not simply copying a screenshot of one side's entire settings page.

Only **new** ordinary comments are mirrored by these handlers. Editing or deleting a comment does not edit or delete its existing counterpart. A correction should be a new comment if both sides need to receive it. Historical comments are not automatically copied as a complete conversation when a new mirror pair is created.

A mirrored comment identifies its source author and side. A GitHub comment copied into Plane includes who commented and, when available, a link to the original. A Plane comment copied into GitHub has a "commented in Plane" introduction. Where usable personal authorization exists, supported GitHub writes can use it; otherwise the App identity performs the write. The provenance remains useful in either case.

PR events and configured automation

PR lifecycle automation is driven by ordered rules. The first matching rule supplies its actions. The simple lifecycle-to-state settings are another way to configure this same engine. There is no second independent state engine running alongside those rules.

Lifecycle point	Meaning to a developer
Drafted	PR is a draft
Opened	PR is open for normal work/review
Review requested	Someone was asked to review
Changes requested	A review requests changes
Approved	An approval review event occurred
Ready for merge	Integration checked readiness from GitHub
Merged	PR merged into its base branch
Closed unmerged	PR closed without a merge
Reopened	A closed PR was reopened

Rules can inspect the base branch, head branch, labels, author, repository, and target project. For example, a team can map merge into `staging` to In QA and merge into `main` to Done. Those are examples of configured behavior, not an assertion that every project has those states or ships those rules.

Actions can select a state, add or remove a label, assign a mapped person, post a comment, add a link, record time as a comment, or create another work item. Your administrator owns the actual rule set. Unexpected labels or comments after a PR event may therefore be deliberate rules rather than part of ordinary issue mirroring.

Readiness is checked from current GitHub data for an open, non-draft PR. It needs at least one standing approval and no standing request for changes. Relevant check suites must be completed with success, neutral, or skipped conclusions; stale suites and suites with zero latest runs are non-blocking. Combined commit statuses must succeed, while a reported zero-status set is allowed. A visible approval badge alone does not establish readiness. Missing permissions or incomplete reads cannot safely be treated as green. The ready-for-merge event is guarded per PR head commit so several simultaneous approval/check events do not repeatedly apply the same readiness action. A new head commit can become ready again.

Closing without merge does not inherently finish or cancel a task. It has its own lifecycle point, and a team may configure no action. Merging a closing PR still waits for the linked-PR condition described in Chapter 09. Review and check information can update even when the Plane state stays unchanged. In the current rule executor, reference tiers and the multiple-PR merge guard gate state changes specifically. Other configured actions, such as labels or comments, can still run even when a state change is skipped.

Development information and linkbacks

The integration stores categorized Plane links for PRs, branches, builds, deployments, and previews. It updates a stored rollup link in place when a status changes. The Development panel groups those links for convenient reading. Native Plane links can also show reserved prefixes such as `[PR]`, `[Branch]`, `[CI]`, `[Deploy]`, or `[Preview]`.

Checks and deployment statuses report work done by GitHub workflows or external deployment systems. Git-sync does not run the build or deploy your application. A merged PR can have a failed build or a pending deployment; verify the relevant category before claiming a release is live.

The mapping's **linkback** option controls extra cross-links. For a newly created mirror pair, it permits a Plane link to the GitHub issue and a GitHub comment linking the Plane item. A Plane-side "Mirrored from/to" confirmation is separate and remains part of creation. For a newly linked PR, linkback can post a GitHub comment beginning "Linked Plane work item" or "Linked Plane work items." Linkback off suppresses those extra cross-links, not the underlying mirror or reference.

Report problems is a separate option and defaults off. When enabled, the service can place durable diagnostic notices on a PR, commit, or known Plane item. Some unresolved-key notices are restricted to linked workspace members and true missing-item results. A lack of a public notice is therefore not proof that every supplied key resolved.

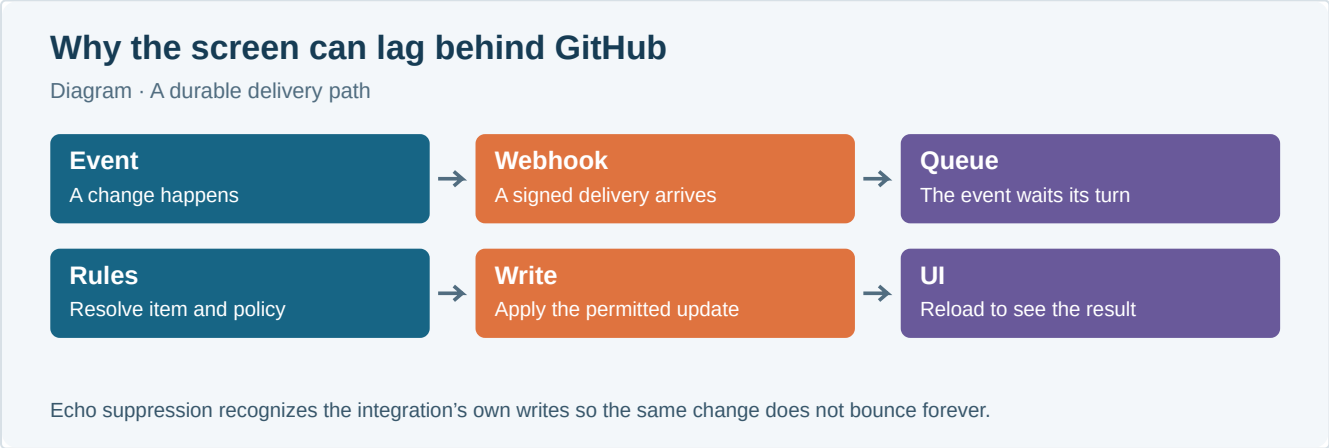


Diagram: events travel through signed webhooks, queues, rules and durable writes before a refreshed UI reflects the result.

Queues, delays, and echoes

A webhook is a short notification that something changed. The incoming HTTP request is validated and queued; workers then interpret it and register outbound writes. GitHub-bound and Plane-bound writes have separate queues so one side's outage need not block all work on the other side.

The service records multi-step operations durably. For example, creating a mirror partner, remembering its ID, and posting a cross-link are related steps that may finish at different times. A create can be accepted while its linkback is still pending. Retries resume eligible unfinished work rather than asking you to create the issue again.

There is no guaranteed instant update or fixed number of seconds promised to developers. Rate limits, outages, queued work, and authorization failures affect completion. Avoid editing both sides repeatedly to “wake up” the integration. Inspect the source, wait for processing, and give the administrator a concrete event URL and expected result if the delay persists.

Two-way sync must also avoid an endless bounce: Plane changes GitHub, GitHub reports that change, and the integration would otherwise write it back to Plane. The service keeps a recent ledger of fields it wrote and recognizes matching return notifications as **echoes**. It also filters Plane events authored by the configured Plane service user. GitHub bots are not universally ignored. An echo is a receipt for work already performed, so ignoring it is the correct result.

Echo suppression differs from deduplication. A repeated webhook delivery should not repeat its effects; a repeated Smart Commit SHA should not repeat completed commands; an echo should not start a reverse write. These are separate safeguards. A changed SHA after history rewriting is still a new Smart Commit identity, as Chapter 10 explains.

Plane can emit one notification per changed field, so one edit may create several queue entries without representing several user actions. The service compares full-value fields with newer source snapshots where possible. An old PR lifecycle event that contradicts the current PR state is skipped rather than moving work backward after a merge or reopen.

What is never assumed to sync

The six-field mirror is deliberately narrower than the whole Plane data model. Priority, start and due dates, estimates, cycles, modules, parent/sub-work-item structure, relations, subscribers, reactions, attachments as independent objects, saved views, pages, notification preferences, and project settings are not ordinary mirrored fields. A description can contain links or images after conversion without that implying an independent attachment-sync feature.

Deleting a Plane item is not a closing instruction. Plane v1.4.2 does not provide the necessary issue-deletion webhook for immediate deletion mirroring. A periodic reconciliation sweep examines linked items. It distinguishes archived items and archived projects where it can, and marks a confirmed or repeatedly missing partner as orphaned. It does not close a GitHub issue merely because Plane returned a 404.

Archiving is also different from Done. Archived items can be hidden by Plane's detail API while still existing. The reconciliation logic records archived links without treating that as deletion. Do not delete or archive an item as a way to force a GitHub close; use the agreed workflow state.

GitHub issue deletion and comment deletion do not trigger destructive counterpart deletion in the ordinary mirror handlers. Reopening a task does not automatically revert code, reopen every PR, or undo a deployment. A reverted commit does not magically reverse old comments or time notes. Those are separate team decisions requiring explicit work and, when appropriate, new links or state changes.

A reliable way to investigate a mismatch

1. Identify whether the relationship is a mirror, a PR reference, or a Smart Commit.
2. Open the original event or source object.
3. Check the exact identifier and repository.
4. Check all linked PRs when completion is expected.
5. Read any visible connection or problem notice.
6. Record the expected result and the observed result.
7. Send the evidence to your workspace administrator through the team's normal channel.

The administrator can check mapping health, direction, field ownership, rule order, App access, signature policy, queue status, and audit entries. A screenshot of the result and a source URL are more useful than repeated edits. Include a rough time and timezone so the administrator can find the relevant delivery. Share no tokens or webhook secrets.

Implementation references: `src/sync/mirror.ts`, `src/sync/reconcile.ts`, `src/sync/dev-info.ts`, `src/app/issues.ts`, `src/app/pr.ts`, `src/app/push.ts`, `src/app/rules.ts`, `src/github/pr-automation.ts`, `src/queue/index.ts`, and `src/ui/validate.ts`.

A completion check runs at a qualifying merge event. Closing another linked PR unmerged later does not automatically revisit an earlier merge that was blocked by that PR. Reconciliation does not imply continuous re-evaluation of this merge rule.

12. A day in the life

This chapter follows Alex Developer through one small change: **WEB-42 — Keep the checkout button visible on a narrow screen**. Sam Reviewer reviews the code. The names and checkout scenario are teaching examples. Use your team's actual repository and assigned work item when doing real work. Captures for this handbook use a dedicated demonstration project and an authorized GitHub test repository. Captions and the image notes distinguish real captured events from illustrations or prepared examples.

A work item describes the outcome the team wants. A branch holds a developer's changes. A commit records a set of changes. A pull request asks the team to review and merge those changes. git-sync recognizes references to the work item's identifier, such as `WEB-42`, in the relevant branch, PR or Smart Commit processing. The identifier travels with the work, so another developer can discover why a change exists without remembering who wrote it.

The worked result assumes a mapped repository, enabled Smart Commits, the shipping automation defaults, a project with suitable In Progress, In Review and Done states, and a pull request merged into `main`. Administrators can change these policies. Treat each expected result below as a checkpoint against your team's configuration. The final state also depends on other linked pull requests: an open linked PR prevents the merge completion transition.

> Tip: Keep Plane in one browser tab and the repository in another. Use your terminal for git commands. Return to the work item at each checkpoint so a missing connection is discovered early.

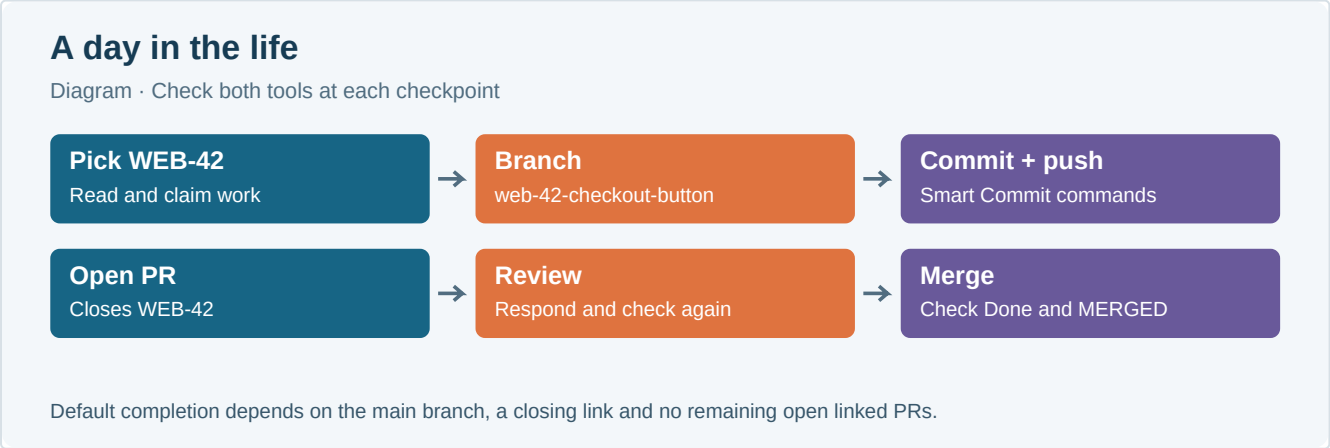


Diagram: the daily loop from picking WEB-42 to verifying Done and MERGED.

> Warning: The capture environment’s GitHub connection was unavailable during this edition. The Plane edits and GitHub PR #117 are real, but automated Smart Commit comments, linked Development entries and the final Done/MERGED transition have not yet been verified live. The remaining automation checkpoints describe the behavior checked in this repository’s code. The image notes record the blocker and the open PR for completion after the test connection is restored.

12.1 Choose a task with a clear finish line

1. Open the Website project in Plane.
2. Open the view your team uses to choose its next work item.
3. Open `WEB-42`.
4. Read its description.
5. Read its comments.
6. Check its assignees.
7. Check its priority.
8. Check any linked blocking work.

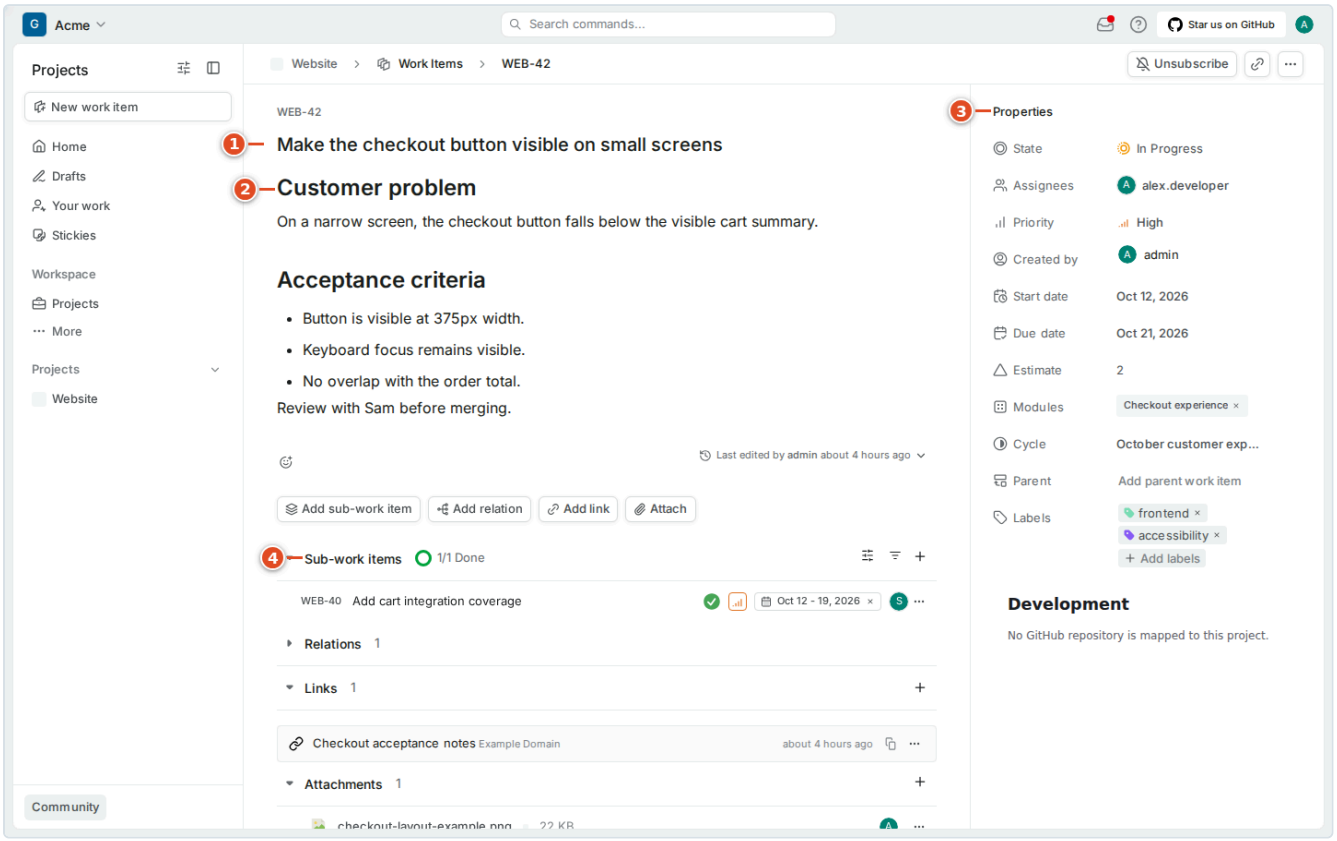
Alex learns that the checkout button disappears below a width of 400 pixels. The description asks for a visible button at 320, 375 and 390 pixels, no overlapping text, and unchanged keyboard access. A comment says that Sam can review the visual change. Those details provide a finish line: a small CSS adjustment plus checks at the listed sizes. “Make checkout better” would be too vague to implement confidently.

An assignee identifies who is responsible for moving work forward. It does not prove that person is currently writing code. Alex checks ownership before starting so two people do not independently implement the same fix. A blocking relation means another outcome is needed first. Reading the work item can reveal a dependency that a short board card does not show.

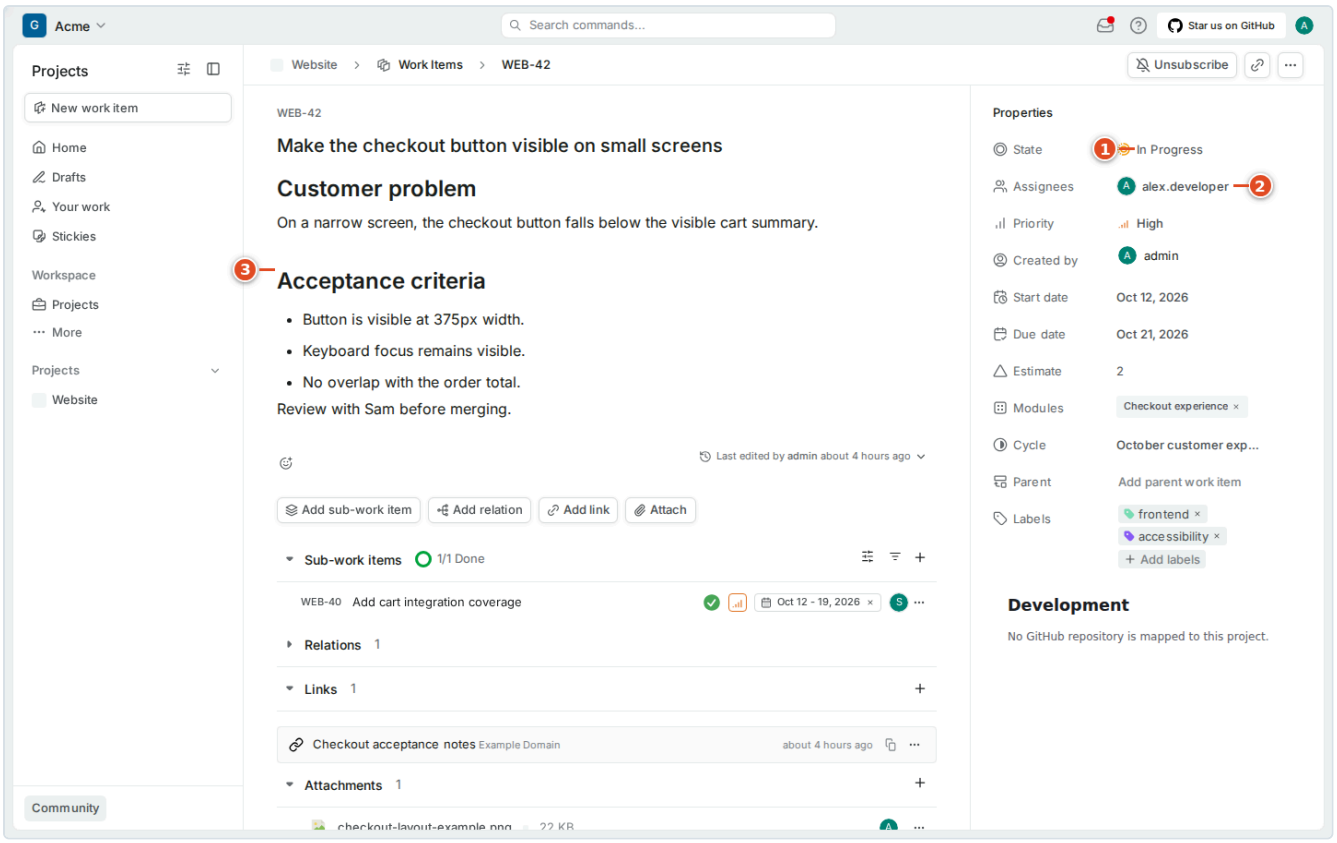
9. Assign the work item to yourself if your team expects developers to claim work.
10. Move the work item to In Progress.
11. Add a short comment describing your intended approach.

> Example: “Starting the narrow-screen checkout fix. I will preserve keyboard focus order and check the three widths listed above.” This tells the team what is happening without repeating the entire description.

Manually setting In Progress is a normal way to declare that work has begun. The default branch and PR automation can also move a linked item into a started state. A later automatic transition does not make this first update unnecessary: your teammates need to know that you are investigating before the first push.



WEB-42: (1) title and identifier; (2) description; (3) properties; (4) sub-work items.



Starting work: (1) In Progress state; (2) assigned developer; (3) acceptance criteria.

12.2 Prepare a clean local branch

The following terminal steps assume you already have a local clone and a remote called `origin`. They do not install anything. Substitute the directory containing your team's checkout. A clean starting point matters because unrelated edits can accidentally become part of the same pull request.

1. Open a terminal in your local repository.
2. Inspect your current changes.

```
git status
```

If you have unfinished work, preserve it according to your team's git practice before switching branches. Do not discard unfamiliar files just to make this example easier to follow. Ask a teammate to help identify changes you do not recognize.

3. Switch to the base branch.

```
git switch main
```

4. Update the base branch without creating an unexpected merge commit.

```
git pull --ff-only
```

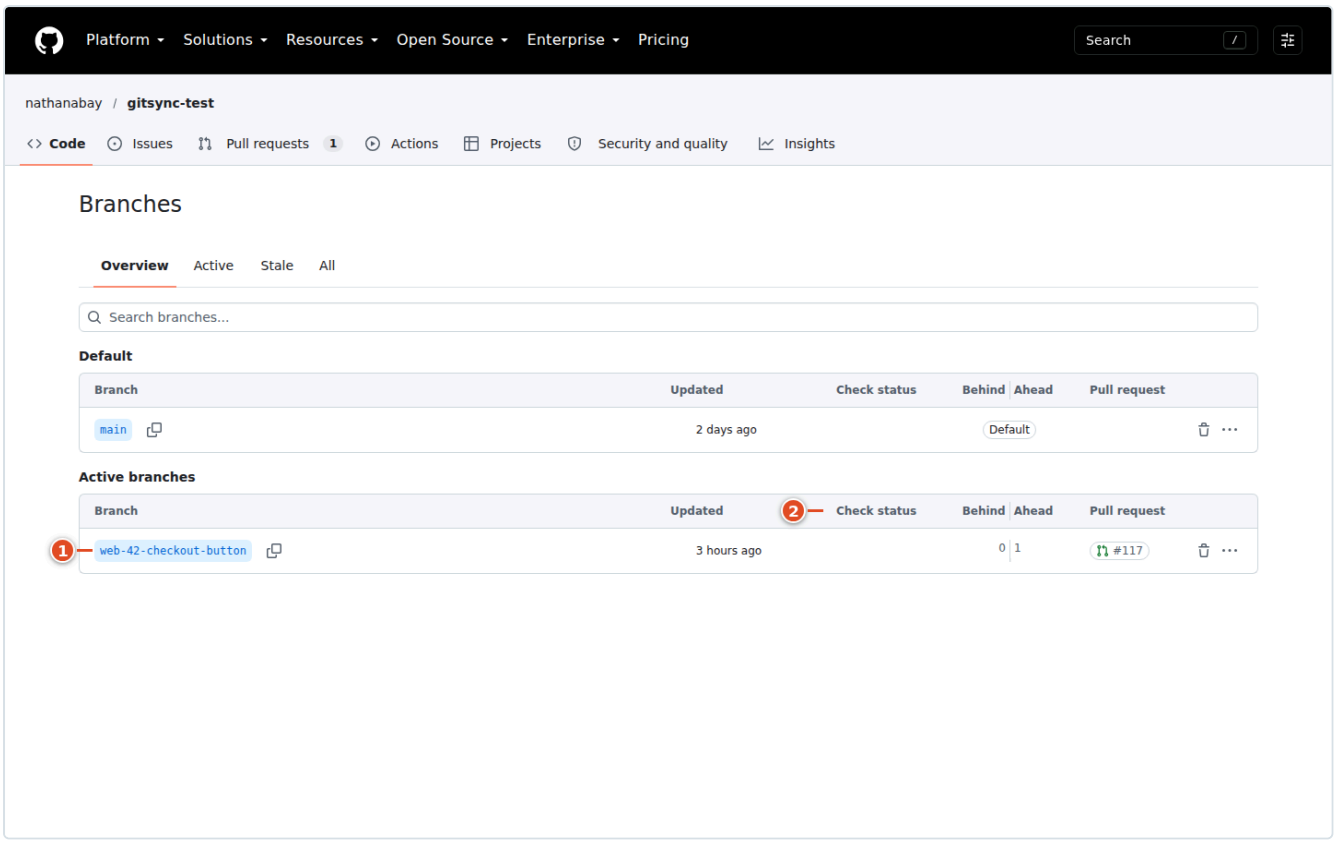
5. Create the task branch.

```
git switch -c web-42-checkout-button
```

The lower-case `web-42` in this branch is recognized. Branch references are case-insensitive. Keeping the key near the start makes it easy for humans to read, although the parser can find a correctly delimited key elsewhere in a branch name. Hyphens make useful separators. Avoid changing the identifier into `WEB_42`, `WEB42`, or `WEB-42_fix`: those are different strings, and the last form fails the parser's trailing boundary rule because an underscore follows the number.

The Development panel offers **Copy branch name** for each mapped repository. Its suggested name follows the administrator's template. Use that suggested name when your team has standardized it. **Create branch**, when available, creates a remote branch from the repository's default branch; it does not switch your local checkout. After using it, fetch the remote branch and switch to it instead of creating an unrelated branch with a different name. The branch controls and authorization are implemented in `src/ui/client/DevPanel.tsx` and `src/ui/dev-panel.ts`.

> Warning: A bare key in a branch is a closing reference. A later `Refs WEB-42` in the PR body cannot weaken it because the parser keeps the strongest reference found. Use the no-key branch and explicit non-closing PR wording in section 12.8 when you are contributing only part of an item.



Real GitHub branches: (1) web-42-checkout-button; (2) Check status column. The branch row also links PR #117.

12.3 Implement a focused change

1. Edit the files needed for the checkout fix.
2. Run the project's relevant checks.
3. Inspect the change before staging it.

```
git diff
```

4. Stage the intended files.

```
git add path/to/checkout.css path/to/checkout.test.ts
```

The paths above are examples. Select files that actually exist in your repository. Specific paths make it easier to avoid committing unrelated local work. Your project may use different tests or have no automated visual test; record what you actually checked.

5. Inspect the staged change.

```
git diff --cached
```

6. Create a commit with a Smart Commit line.

```
git commit -m "WEB-42 keep checkout button visible" \
  -m "WEB-42 #time 45m #comment Checked 320, 375 and 390 pixel widths."
```

The first line gives a concise description. The second message paragraph contains an identifier before commands on the same line. `#time 45m` records a comment describing 45 minutes. Community Edition does not receive a native worklog entry from this command. `#comment` adds the remaining text as a Plane comment. Put it last because everything after `#comment` is comment text, including strings that look like additional commands.

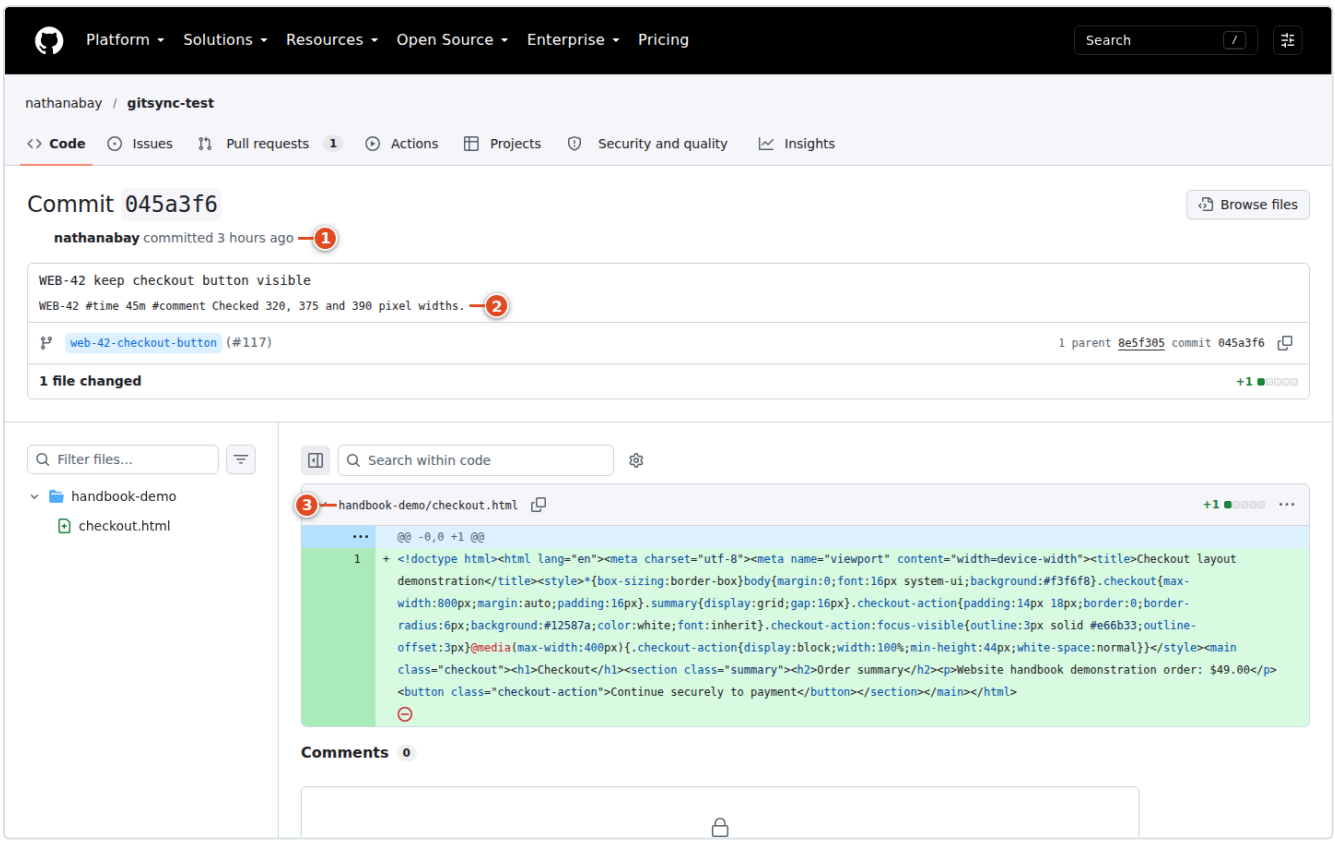
7. Push the branch.

```
git push -u origin web-42-checkout-button
```

A local commit does not contact git-sync. The push causes GitHub to deliver an event to the service. Smart Commit processing then depends on the mapping being enabled and, where configured, signature verification. The authenticated GitHub event sender determines attribution; changing the author email in a commit does not authorize acting as another Plane member.

8. Reopen the Plane work item.
9. Check its comments for the Smart Commit result.
10. Check Development for the branch link.

Do not expect a separate commit list in the current Development component: it renders PRs, builds, deployments, previews and branches. The Smart Commit comment can provide commit provenance. The initial branch link is associated with a newly pushed branch; repeatedly pushing the same branch is not a general repair command for a missing initial branch event. See `src/app/push.ts` and `src/github/smart-commits.ts` for these distinctions.



Real GitHub commit: (1) author and commit time; (2) Smart Commit message; (3) changed demonstration file.

12.4 Open the pull request with an explicit outcome

1. Open the repository on GitHub.
2. Start a pull request from `web-42-checkout-button`.
3. Check that its base branch is `main`.
4. Set the title to `WEB-42 Keep checkout button visible`.
5. Enter the pull request description.

```
Closes WEB-42

What changed
- Keep the checkout action within the narrow-screen layout.
- Preserve the existing keyboard focus order.

Checked
- 320, 375 and 390 pixel widths.
- The project's checkout tests.

Reviewer focus
- Text wrapping with the longest supported button label.
```

6. Create the pull request.

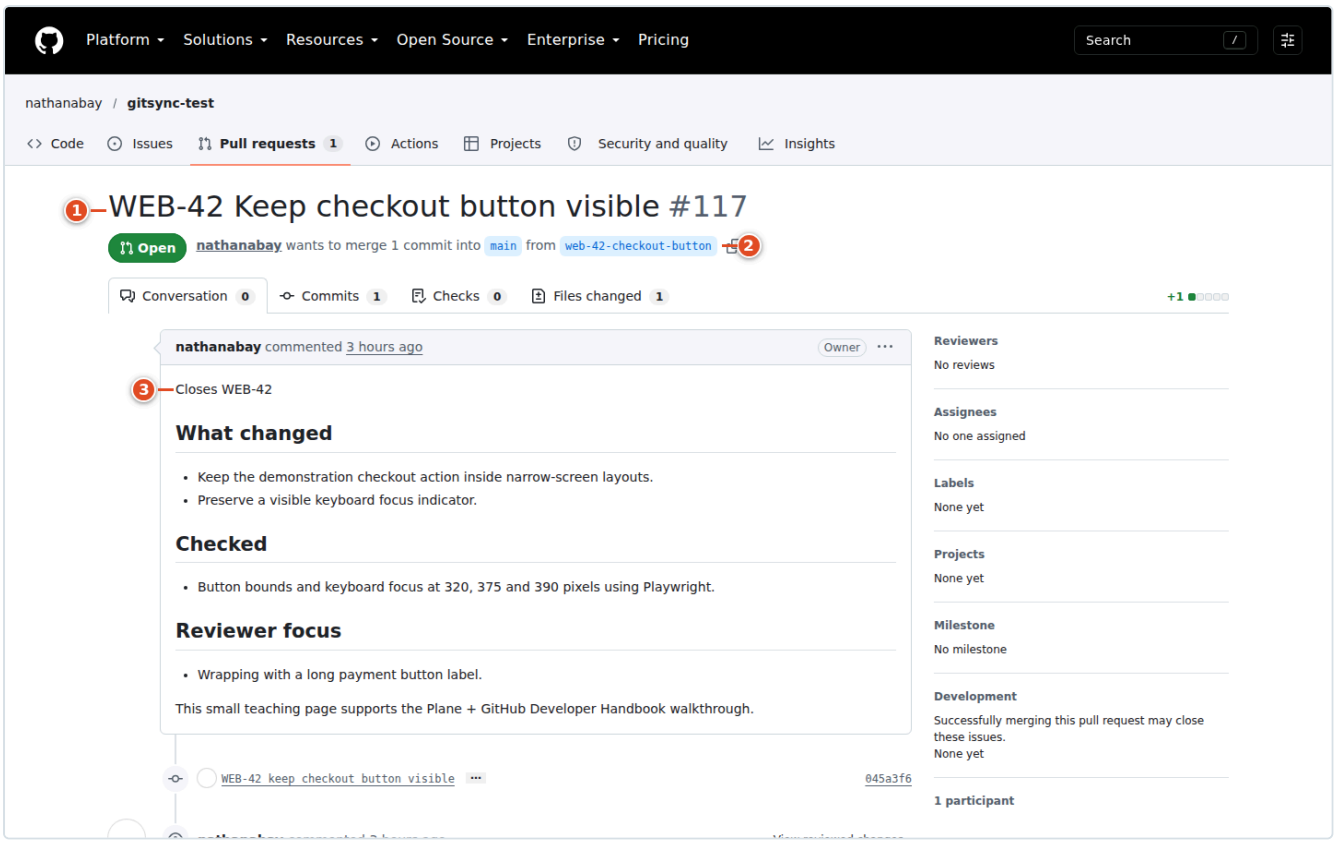
Here, `Closes WEB-42` states that this PR intends to finish the item. Under the default vocabulary, `Closes` is a closing word. A bare uppercase identifier in the title also counts as closing, as does the branch's identifier. The duplication is useful communication for people reading the PR, but the parser still produces one strongest reference per work item.

The phrase addresses git-sync's Plane reference parser. It does not claim that GitHub itself understands a Plane identifier as a native GitHub issue reference. A GitHub issue has a repository number such as `#123`; `WEB-42` belongs to Plane. When both are involved, make each relationship explicit and review its effects.

7. Return to `WEB-42` in Plane.
8. Open Development.
9. Check the linked PR title.
10. Follow the PR link to verify its destination.

A typical linked entry reads `#123 WEB-42 Keep checkout button visible · OPEN · REVIEW`. `OPEN` describes the PR lifecycle. `REVIEW` describes the aggregate review outcome while it is open. The Plane work item may still be In Progress. These are different records: linking a PR does not mean somebody has requested a review yet.

If a linkback comment is enabled, git-sync also posts a comment on the GitHub PR when new Plane links are established. Its absence is not sufficient evidence of a failed link because administrators can turn linkback off. The Plane Development entry and the admin audit log provide other evidence.



Real PR #117: (1) title; (2) feature branch targeting main; (3) Closes WEB-42.

12.5 Ask for review and respond to feedback

1. Request Sam as a reviewer on GitHub.
2. Return to the work item.
3. Check its state.

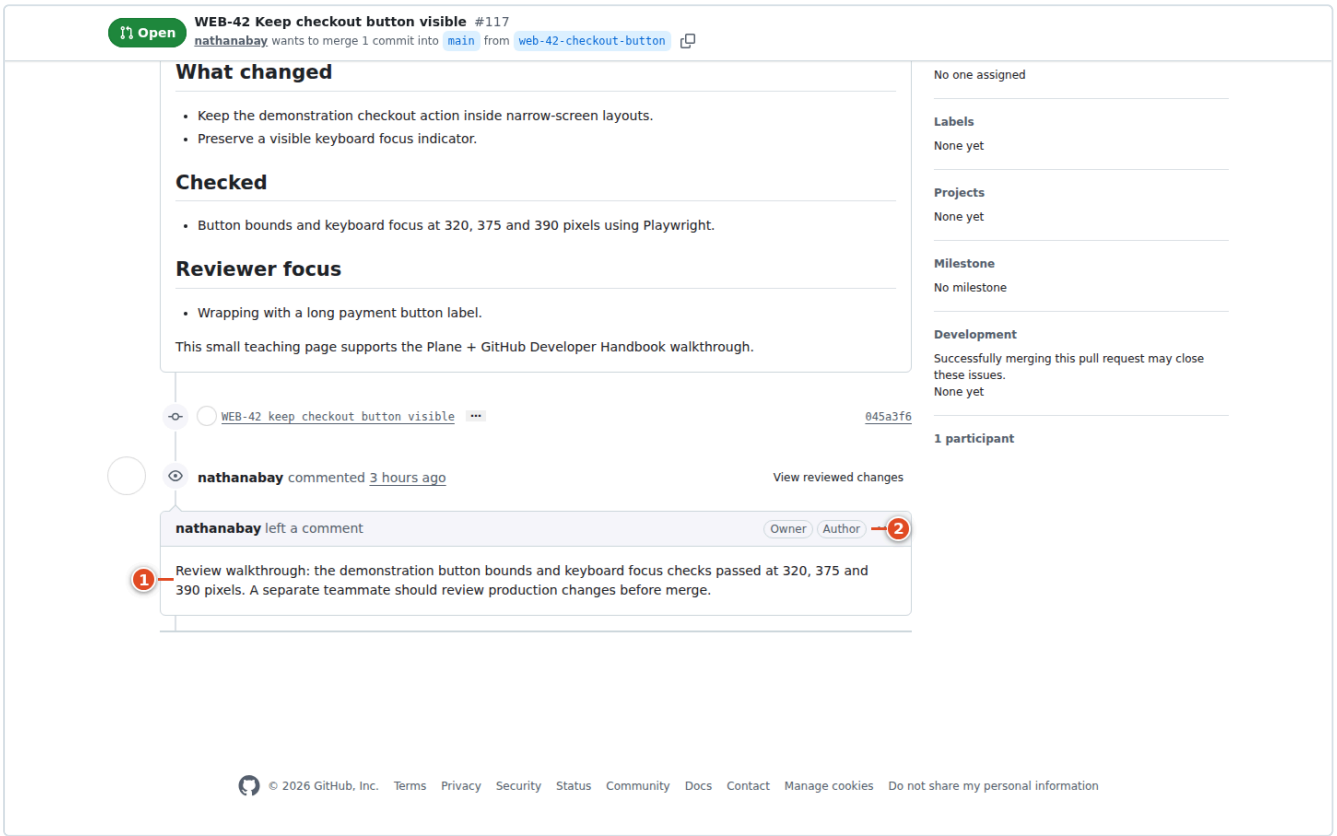
With the shipping defaults, the review-request event asks Plane to move the item to In Review in the started group. Simply publishing the PR does not use this review-request transition. If your team’s states or rules differ, use their documented review state instead of creating a new state just for this example.

4. Read Sam’s review on GitHub.
5. Implement any agreed changes.
6. Run the checks affected by those changes.
7. Commit the revision.
8. Push the revision.
9. Respond to the review discussion.

> Example: Sam spots wrapping with a translated label. Alex adjusts the layout and replies with the checked width and label. The work item comment can summarize the outcome for teammates who do not follow every code discussion.

An approval changes the review summary but does not, by default, move the work item to Done. A requested change can produce a `REJECTED` review summary while the PR remains `OPEN`. This word describes review feedback, not a deleted work item or an abandoned task. An approving review from another person does not cancel a different reviewer's standing request for changes. Latest decisive reviews are considered per reviewer.

Build links and deployment links appear only when the relevant data is available to git-sync. A green display is evidence about reported checks, not a promise that every possible test has run. Follow the project's required review and merge rules. git-sync's readiness calculation and GitHub's ability to permit a merge are separate concerns.



Real COMMENTED review: (1) walkthrough review text; (2) author identity on the review. This is not independent approval.

12.6 Merge and verify the completed work

1. Confirm that the pull request meets the team’s merge requirements.
2. Merge it into `main` using the repository’s permitted method.
3. Open `WEB-42` in Plane.
4. Check that the state reaches Done.
5. Check that Development reports the PR as `MERGED`.
6. Add any useful handover or release note to the work item.

Under the defaults, a closing-linked PR merged into `main` triggers the completed Done target. Completion waits until no linked PR remains open, and at least one linked PR has merged. A closed, unmerged PR does not itself prove completion and does not block a different merged PR. If another linked PR cannot be read, the service conservatively treats that unknown PR as open. These safeguards live in `src/app/pr.ts` and `src/github/pr-automation.ts`.

Completion is checked at a qualifying merge event. If PR A merges while PR B remains open, the item stays started. Closing B later without merging does not, under the default rules, revisit A’s completion check. Close obsolete PRs before the final completing merge, or make an explicit team-approved state correction after a later abandonment.

Updates travel through webhooks and queues, so the browser can lag behind the completed merge. Reload once after allowing processing time. If the state remains unchanged, check the merge branch, all linked PRs, link tier and mapping policy before manually fighting the automation. Chapter 14 provides a troubleshooting order.

7. Switch your local checkout to `main`.
8. Update your local base branch.
9. Delete the completed local task branch if your team permits it.

Deleting a local branch is optional cleanup after the merge is confirmed. Preserve unmerged work. The work item remains useful after coding ends: it holds the reason for the change, discussion, and links to implementation. A merged PR is also different from a production deployment. Record a deployment or release separately when that distinction matters to your team.

12.7 Variant: a bug begins as a GitHub issue

A user reports a checkout bug as GitHub issue `#140`. The team wants to plan it in Plane while keeping the discussion accessible on GitHub. Issue mirroring is an opt-in relationship, separate from a PR reference.

1. Open the existing GitHub issue.

2. Check whether it already links to a Plane work item.
3. Add the `plane` label if your team's mapping permits this workflow.
4. Wait for the mirror to create its Plane partner.
5. Open the linked Plane work item.
6. Confirm the title and description describe the same bug.
7. Triage the work item with your team.
8. Follow the normal branch and PR loop using the new Plane identifier.

The label can be present when an issue opens or added later. With an eligible mapping, that event requests creation. Do not create a second Plane item while the first request is pending. When linkback is enabled, the cross-link helps you find the new record. If it is disabled, an administrator can identify the mapping and created partner.

Only a mirror pair receives ordinary issue field synchronization. Mentioning `WEB-42` on an unrelated GitHub issue does not create the same field mirror. Allowed directions and per-field policies still apply. New comments can mirror; edits to an existing comment are not a reliable way to send a correction to the other side. Add a follow-up comment when both audiences need the correction.

12.8 Variant: split work across sub-work items and PRs

Suppose `WEB-42` is too large for one review. The team creates `WEB-43` for layout and `WEB-44` for keyboard behavior as sub-work items. A parent expresses the shared outcome; the children make ownership and acceptance checks smaller. Parentage does not automatically make every child PR a closing PR for the parent.

1. Open the parent work item.
2. Create a sub-work item for one independently testable outcome.
3. Create a second sub-work item for the remaining outcome.
4. Write acceptance checks on each child.
5. Assign each child to its owner.
6. Create a branch for the child you are implementing.
7. Reference that child with a closing phrase in its PR.
8. Reference the parent with an explicit non-closing phrase.

```
Closes WEB-43
Refs WEB-42
```

Use branch `web-43-checkout-layout` and a title centered on `WEB-43`. Keep a bare `WEB-42` out of the branch and title; either would promote the parent to a closing reference. The PR handler compares the branch, title and body and keeps their strongest tier. Commit messages are handled separately by push processing; inspect their Smart Commit commands so they do not move the parent directly.

For a single item implemented across several partial PRs, use a branch without the key, such as `checkout-layout-part-one`. Use `Refs WEB-42` in the PR title and body. A branch such as `refs/web-42/layout` still contains a bare key and is closing: the slash does not satisfy the magic-word parser's required whitespace. Non-closing references can participate in earlier lifecycle state changes, but their merge does not complete the item. `Relates to WEB-42` is weaker: it links without lifecycle state changes. Make the final completion decision explicit when the whole outcome is ready.

12.9 Variant: one PR completes two work items

1. Check that both work items have clear acceptance criteria.
2. Describe both outcomes in the PR.
3. Add an explicit closing line covering both identifiers.

```
Closes WEB-42, WEB-45
```

4. Check Development on each work item.
5. Review the acceptance checks for both items.
6. Merge after both outcomes are ready.
7. Verify the state of each item separately.

One PR can link several identifiers. The parser accepts separators such as a comma or `and` after a magic word. Each item's other linked PRs and policies still matter. One item may complete while another remains started because it has another open PR. Avoid bundling unrelated changes simply to reduce the number of pull requests: the review should remain understandable.

12.10 Variant: revert a merged change

A revert is a new change that undoes earlier code. git-sync does not have a special "undo every Plane effect of the original PR" operation. The branch lifecycle rule avoids starting old work solely from branch names beginning `revert-`, but ordinary PR references and Smart Commit processing still apply. A title or quoted message in the revert can still be parsed, which makes careful wording important.

1. Open a work item describing why the revert is needed.

2. Decide with the team whether the original item should reopen.
3. Create the revert using the repository’s established git process.
4. Inspect the revert PR’s branch, title, body and commit messages for identifiers.
5. Use the new corrective item’s identifier for its intended completion.
6. Describe the relationship to the original change deliberately.
7. Merge after the required review.
8. Verify both work items’ states.

For example, close a new corrective item with `Closes WEB-51` and describe the original with `Relates to WEB-42`. If a generated revert PR contains a bare original key in its branch or title, or a closing phrase in its body, the relation alone will not weaken that stronger reference. A deliberate `skip WEB-42` in the **PR body** suppresses that identifier for PR reference parsing. Use this exclusion before the PR first establishes that link: existing stored PR links remain active, so adding `skip` later is not an unlink operation. It also does not cancel Smart Commit commands already executed by an earlier push. Review the generated text before pushing, especially if it copied `#done` or `#time` from an earlier message.

Reopen the original work item explicitly if its promised outcome is no longer delivered and your team’s policy calls for reopening. The absence of an automatic reopen is expected: merge history is not an inverse workflow engine. Retain the explanation so the next developer understands whether the task is unfinished, replaced, or intentionally withdrawn.

13. Team conventions and good habits

Everything in this chapter is a **suggested team convention**. These practices are useful starting points, not additional parser rules or guarantees enforced by git-sync. Agree on a small set with your team, publish the decision in a team page, and adjust it when real work reveals a problem. Administrator settings and repository protections still govern what the software actually allows.

13.1 Give the work item an outcome

A useful title tells a teammate what changes for a user or operator. “Keep the checkout button visible on narrow screens” is easier to finish and review than “Checkout improvements.” Put the evidence needed to prove success in the description. Include the affected audience, the problem, acceptance checks and constraints that cannot be inferred from code.

For a bug, write reproduction steps, actual behavior and expected behavior. For a new feature, explain the decision the user will make or the action they will complete. A small screenshot can clarify a visual problem, but write enough text that the work item still makes sense to someone reading a notification without the picture.

> Example: “At 320 pixels wide, the checkout action remains visible, its label wraps without covering the total, and keyboard focus reaches it after the address fields.” This is a testable outcome. “Responsive checkout” is a topic.

Use comments for conversation and the description for the current agreement. When the team changes the plan, update the description and leave a comment explaining the change. A newcomer should not have to infer the present requirement from ten contradictory comments.

13.2 Make responsibility easy to find

Choose one person to move each item forward, even if several developers collaborate. Plane can represent multiple assignees, but your team should agree who arranges review and resolves questions. A reviewer is not automatically a co-owner of implementation. Mention the person whose decision is needed and state the question clearly.

Before starting, check the work item’s state, owner and blocking relations. When blocked, record what is needed and who can provide it. Avoid repeatedly changing priority to attract attention; priority should describe the team’s relative urgency. A clear comment with a concrete dependency is easier to act on.

At the end of the day, leave one sentence about the next step if somebody else may pick up the work. “Layout fixed; waiting for confirmation of the translated label” is more useful than “Still working.” Keep passwords, access tokens, customer records and private incident details out of ordinary comments and screenshots.

13.3 Use identifiers consistently

A simple default branch convention is `web-42-short-description`. Prefer the identifier exactly as Plane displays it when writing prose: `WEB-42`. The branch parser tolerates lower case, but bare prose references are generally uppercase-only. Consistent spelling makes both human scanning and automation more reliable.

Distinguish three intentions before writing a PR: finish the item, contribute part of it, or mention related context. Use `Closes`, `Refs`, or `Relates to` accordingly, after checking the administrator’s configured vocabulary. Review the branch and title as well as the body. A weaker body phrase does not override a stronger reference elsewhere.

> Warning: `Refs WEB-42` in the body plus branch `web-42-layout` still yields a closing reference. For a partial contribution, use a branch without the key, such as `checkout-layout-part-one`, and an explicit `Refs WEB-42` PR title. `refs/web-42/layout` still parses as closing because the slash does not satisfy the magic-word whitespace rule.

Do not scatter identifiers into copied templates without understanding why they are there. A PR description saying “Closes WEB-42” is an instruction under the default vocabulary, even when the author meant it only as an example. Use `skip WEB-42` in the PR body before first linking when deliberately excluding a copied reference, and document why the exclusion is needed for reviewers. Adding it later does not remove a stored PR link.

13.4 Keep commits useful and deliberate

Write a commit summary that explains the change. Smart Commit commands should add useful history, not replace a readable message. Consider placing the commands on their own message paragraph, with a key before commands on each line. Put `#comment` last. Use full hyphenated state names instead of ambiguous abbreviations.

Log time only when the team wants this information and with a shared understanding of its meaning. In this integration, `#time` produces a comment in Plane CE. It does not feed a native timesheet. The parser treats a day as eight hours and a week as five such days; do not assume elapsed calendar time or your personal work schedule is used.

Avoid `#done` in intermediate commits unless you intend a state transition when the push is processed. Closing words in PR references express completion intent for lifecycle automation, while a state command requests a transition during Smart Commit processing. They have different timing. A branch can still contain unfinished work when a pushed command moves the work item.

13.5 Write a PR that a teammate can review

Use a short structure: purpose, changes, validation, and reviewer focus. Include the intended Plane references near the beginning. Describe the checks you actually ran and any meaningful gaps. “Tests passed” is weak evidence when a visual issue was never opened at the affected viewport width.

Keep the change small enough that a reviewer can understand it as one decision. A multi-item PR can be appropriate when the outcomes depend on the same change, but each linked item still needs its acceptance checks reviewed. Ask for review explicitly; publishing an open PR is not the same event as requesting a reviewer.

Treat review feedback as part of the work. Reply with the fix or the reason for a different approach. Update the work item when the outcome or delivery plan changes, so teammates who do not read code discussions can still understand its status.

13.6 Separate code completion from delivery

Agree on what Done means in your project. The shipping automation can move an item to Done after a qualifying merge. Some teams need a deployment, a manual verification or release communication before considering a user-facing outcome delivered. If that is your team, ask the administrator to align rules with the agreed workflow.

Do not use a green build or a `MERGED` badge as a substitute for release evidence. Build, deployment and preview links describe different events. Preview environments are useful for review, but a preview URL is not proof of a production deployment. Put the release decision somewhere a teammate can find it.

When one item has several PRs, list the intended parts in its description. Keep outdated PRs clearly closed if they were abandoned. An open linked PR intentionally blocks automatic completion on another merge. Closing an obsolete PR is a workflow decision; deleting the link merely to force Done can hide unfinished scope.

13.7 Make troubleshooting collaborative

When something looks wrong, record the identifier, PR URL or commit SHA, approximate time, expected result and actual result. Include the smallest relevant screenshot after removing sensitive details. This gives an administrator enough evidence to find an event without asking you to reproduce a potentially harmful write.

Prefer one deliberate correction over repeated edits intended to “wake up sync.” Retries can be legitimate, but repeated Smart Commit messages with new SHAs can repeat time comments or other side effects. Reconnect your personal account when asked by the UI; repository access problems usually need an administrator or organization owner.

A team page should contain the chosen branch convention, review state, merge base, meaning of Done, time-recording practice, and support contact. Review it during onboarding. The purpose is to reduce decisions a new developer must guess, while keeping project policy visible and changeable.

14. Troubleshooting

Most apparent sync failures come from one of four places: the identifier was not recognized, the repository or project is not mapped, policy does not allow the expected change, or an event has not finished processing. Start with the visible record and work outward. A work item does not need to be recreated just because a link or state update is delayed.

The following tables describe member-level checks first. Administrator checks are identified explicitly. They refer to this repository's implementation, especially `src/parse/references.ts`, `src/app/pr.ts`, `src/app/push.ts`, `src/github/smart-commits.ts`, `src/sync/mirror.ts`, `src/queue/workers.ts` and `src/ui/dev-panel.ts`. An administrator can use the operations manual for setup and operational procedures.

> Tip: Separate three questions: "Is the code linked?", "Did the event arrive?", and "Was the requested change allowed?" A visible PR link can be correct even when the workflow intentionally leaves the work item state alone.

14.1 Check the identifier and the link

Symptom	Likely cause	What to do
No PR appears on the work item	The key is absent, misspelled, suppressed, or unresolved	Copy the key from Plane; inspect the branch, title and body; check for `skip` or `ignore` in the body.
`web-42` in a prose paragraph does not link	Bare free-text references are uppercase-only	Use `WEB-42`, or an explicit configured magic phrase such as `Refs web-42`.
A branch ending `WEB-42_fix` does not link	An underscore immediately after the sequence violates the key boundary	Use a delimiter such as `web-42-fix`.
`X-1` does not link	Project identifiers in this parser need 2–12 characters	Ask the project administrator about a supported project identifier.
A key links the wrong work item	The number belongs to a different task or project	Compare the Plane identifier and title before correcting the PR text.
A valid-looking key remains unknown	The sequence does not exist or the service cannot see its project	Confirm you can open the item; ask an admin to check service membership and the unresolved-key audit entry.
A branch exists only on your laptop	No remote push has occurred	Push the intended branch to the mapped repository.
An old remote branch has no new branch link	Initial branch creation delivery was missing or occurred before mapping	Ask an admin to inspect that event; another ordinary push is not guaranteed to recreate the initial branch record.
Adding `skip` does not remove an old PR link	Existing stored links are retained	Ask an admin to inspect the association; `skip` controls new parsing, not unlinking.

Symptom	Likely cause	What to do
The PR has no git-sync comment	Linkback is disabled or its write is pending	Check Plane Development first; ask an admin about linkback policy if that evidence is needed.

The key parser checks boundaries to avoid treating part of a larger technical token as a work item. It accepts a letter followed by letters or digits for the project portion, with a hyphen and digits for the sequence. It normalizes recognized project letters to uppercase. Ordinary words such as `step-2` are not automatically Plane references in prose. Some technical prefixes, including `UTF` and `SHA`, are explicitly excluded by this implementation; an administrator should investigate unusual project naming instead of asking members to guess a workaround.

A work item that you can see may still be invisible to the integration’s service account. Conversely, GitHub repository access does not prove Plane project access. A `403` or `404` during identifier resolution is reported as unknown rather than exposing inaccessible work. Share the key and repository with your administrator; do not change privacy settings as a troubleshooting experiment.

14.2 Check a state transition

Symptom	Likely cause	What to do
PR is linked but opening it does not change state	The rule is disabled, customized, or uses a state missing from the project	Ask an admin to inspect the applicable lifecycle rule and target state.
A PR contribution completes an item unexpectedly	A bare branch/title key or another closing phrase won over `Refs`	Inspect every source of references; use explicit non-closing wording throughout future partial PRs.
Approval does not mark the item Done	Approval has no shipping state transition	Continue the team’s merge process; verify the configured completion trigger.
Review request does not produce In Review	No review was requested, or policy/state names differ	Confirm a reviewer was requested on GitHub; check the team’s configured review rule.
A merged PR leaves the item started	Base branch does not match, link is non-closing, or another linked PR is open	Check the merge base, reference tier, and every linked PR.
A merged PR on `develop` leaves the item started	Shipping merge rule matches `main`	Ask about the configured merge-base expression; do not assume every base branch completes work.
Closing a PR leaves the work item open	Closed without merge has no shipping state transition	Decide whether the work is abandoned, replaced or still needed; update Plane deliberately.
A revert does not reopen the old item	There is no automatic inverse of the old merge	Follow the team’s reopen or corrective-item policy.
A relation link never changes state	`Relates to` is intentionally relation-only	Use the tier that matches the intended outcome, after checking all other reference sources.

An open linked PR blocks the merge completion transition even when it is only a draft or belongs to another repository. An inaccessible linked PR is treated conservatively as open. This prevents the service from declaring work complete because it could not read part of the evidence. Ask the owner whether an obsolete PR should be closed; ask an administrator to investigate an inaccessible one.

The Development summary and the work item state are different signals. `PRs: OPEN` means at least one linked PR is open. A `MERGED` summary means there is a merged PR and no open PR in the displayed aggregate. It does not prove a particular Plane state rule ran or a production deployment completed. Compare the individual PR rows before diagnosing the aggregate label.

14.3 Check Smart Commits

Symptom	Likely cause	What to do
Nothing happened after a local commit	Commands run after a push event	Push the branch when ready.
Pushed commands have no effect	Smart Commits are disabled, or signature policy rejected the commit	Ask the administrator about the mapping and verified-commit requirement.
The identifier is ignored on a command line	It appears after the first command or on another line	Put `WEB-42` before commands on the same line.
`#done` appears inside a Plane comment	`#comment` consumes everything after it	Put state and time commands before `#comment`.
`#review` fails	Several state names share the prefix, or none matches	Use the complete hyphenated state name, such as `#in-review`.
`#time` fails	It has no supported duration	Use a value such as `45m`, `1h 30m`, or `0.5d`.
Time appears only as a comment	CE has no worklog API used by this integration	Read the recorded minutes in the comment; do not expect a native timesheet.
Pushing the same commit again does not repeat the command	Completed commands are recorded by SHA and workspace	This is expected; ask about recovery if the original result was wrong.
Rebasing produced a second time comment	The rewritten commit has a new SHA	Avoid replaying operational commands in rewritten history; explain any duplicated record.
A state changed before PR merge	A pushed state command requested that transition	Review command timing; use PR lifecycle completion when that matches team policy.

Commands are processed independently. A malformed time command does not prove that a valid comment or transition on the same commit was ignored. Read the actual result and any problem report before trying again. Retrying with a new SHA can duplicate actions that already succeeded. An administrator can distinguish a permanent rejected step from a transient failure awaiting retry in the audit log.

> Example: `WEB-42 #comment Tested locally #done` adds comment text containing `#done`. To request both actions, use `WEB-42 #done #comment Tested locally`. Only use a state token if that exact intended state exists in your project.

14.4 Check account and repository access

Symptom	Likely cause	What to do
“No personal GitHub account connected”	Your workspace identity has not been linked	Open **My GitHub account** and use **Connect personal account**.
“Needs re-authorization”	Stored authorization can no longer be used	Use **Reconnect** and complete authorization with the intended GitHub account.
“Already linked” during connection	This GitHub account belongs to another Plane member in this workspace	Ask the workspace admin to resolve the identity link.
Connection link expired or was already used	Authorization state is single-use and time-limited	Start a fresh connection from the settings page.
Changes appear as a service account	No usable personal credential or identity mapping was available	Check your personal connection; ask about service fallback if it persists.
Private repository is absent	The App was not granted it, or the workspace mapping is unavailable	Ask the organization owner or workspace admin to check installation repository access.
You can read Development but cannot create a branch	Creating needs project member/admin role and a mapped repository	Ask the project admin about your role and mapping.
Development says no repository is mapped	No eligible mapping exists for this project	Ask a workspace admin to configure the project-to-repository mapping.
Development asks you to sign in	Your Plane session expired	Sign in to Plane again and reopen the work item.

Your personal GitHub connection identifies you and supports attribution. The workspace installation gives the service repository access. These are separate pieces. Reconnecting your personal account does not grant the App access to a private repository. Disconnecting your personal account does not intentionally disconnect the whole team’s repository integration; subsequent eligible writes can use the service identity.

The UI names above come from `src/ui/client/Connect.tsx`. Members should use personal account controls. Workspace installation disconnect controls disable mappings and affect the team; do not use them as a personal reset button.

14.5 Check mirrored issues, delays and echo suppression

Symptom	Likely cause	What to do
GitHub issue creates no Plane work item	It was not opted in with `plane`, or no enabled repository mapping exists	Confirm the gate label and repository with the admin.
Plane item creates no GitHub issue	It lacks the `github` opt-in label, or direction prevents creation	Confirm your team's mirroring workflow before adding the label.
A field updates only in one direction	Direction or field policy restricts it	Ask which system owns that field.
A PR discussion comment does not mirror to Plane	Issue comment mirroring is for mirrored issue partners	Summarize the review outcome on the work item when needed.
Editing a mirrored comment does not update the copy	The mirror handles new comments, not edits/deletes as a shared document	Add a corrective follow-up where both audiences need it.
Bot-originated Plane edits do not trigger another round	Service-user writes and recognized echoes are suppressed	This is expected loop prevention; make the intended human edit in the owning system.
Sync is slow after a valid action	Queue backlog, API rate limit, or transient network failure	Allow processing time; ask an admin for queue/audit evidence if it persists.
An entire event is absent after an outage	GitHub did not successfully deliver it	Ask an admin to inspect webhook deliveries and redeliver the specific failed event.
A deleted item leaves a GitHub issue	Plane deletions do not provide the normal deletion webhook	Ask an admin about orphan reconciliation; do not recreate the item blindly.

Echo suppression prevents an edit sent from Plane to GitHub from bouncing back as a fresh edit forever. The service remembers its own writes and consumes matching echoes. Plane events attributed to the configured service user are also ignored at ingress. This does not mean all GitHub bots are globally ignored: bot senders can use service identity for applicable actions. The distinction matters when diagnosing an automation account’s work.

Internal queue retries and GitHub delivery are also different. Once accepted, transient internal failures can be retried. A webhook rejected before acceptance is not guaranteed to arrive again automatically; this repository explicitly documents operator redelivery for failed GitHub webhooks. Repeated browser reloads cannot supply a missing delivery.

14.6 Give an administrator a useful report

1. Copy the Plane work item identifier.
2. Copy the relevant GitHub PR or issue URL.
3. Record the commit SHA when a Smart Commit is involved.
4. Record the approximate action time and timezone.
5. Write the result you expected.
6. Write the result you observed.
7. Attach a screenshot with sensitive details removed.
8. Send the report through your team’s support channel.

> Example: “WEB-42, PR #123, merged into main at about 14:20 Europe/Berlin. Development shows MERGED, but the item is still In Review. Another linked PR #121 is open. Should it still block completion?” This gives the administrator a specific policy question and usable evidence.

Do not send access tokens, browser cookies, webhook secrets or unfiltered configuration files. Administrators can investigate signatures, installation permissions, service membership, mapping rules, audit outcomes, queue state and reconciliation without your secret credentials. Tell them if you already tried a correction, especially a new commit or a manual state change, so the timeline stays understandable.

15. FAQ and glossary

These answers bring together the daily decisions a developer makes while using Plane and GitHub. They describe the current integration and distinguish software behavior from team policy. If your workspace behaves differently, start with its configured mappings and rules, then use chapter 14 to collect evidence.

15.1 Getting started

Do I have to understand every Plane feature on my first day?

No. Start by finding your assigned project, opening a work item, reading its description and comments, and understanding its current state. Connect your personal GitHub account before relying on personal attribution. Then follow the daily loop in chapter 12. Cycles, modules, views and pages become useful as you join planning and handover conversations. Learn the team's actual practices before reorganizing shared work.

What is the difference between a workspace and a project?

A workspace groups people and projects. A project groups related work and gives its items an identifier prefix, such as `WEB`. The item `WEB-42` is sequence 42 in that project. Check the workspace and project when a familiar-looking title appears twice. Workspace membership and project access are related but separate access decisions; you may need an invitation or a project administrator's help.

Which system should I open first each morning?

A useful suggested habit is to check Plane for priorities, mentions and blockers, then GitHub for requested reviews and the status of your open PRs. Your team may choose another order. Plane gives planning context; GitHub gives code context. Neither automatically replaces the conversations and decisions that make work ready to start.

Can I use a saved view as the team's source of truth?

A view presents a filtered selection of work. It can hide items because of its filters, grouping or sub-work-item setting. If an item disappears, search for its identifier and check the view configuration before assuming it was deleted. Shared views are helpful for common planning routines, but changing shared filters can affect what teammates see.

15.2 Accounts and permissions

Why connect my personal GitHub account when the repository already syncs?

The repository installation permits the service to access selected repository data. Your personal connection links your GitHub identity to your Plane membership and supports writing eligible changes as you. The settings UI explains that eligible changes use service identity until you connect. A missing personal connection is therefore different from a broken repository mapping.

Does connecting grant access to every private repository I can see?

No. Installation access and project mappings decide which repository the integration uses. Your own GitHub access does not expand an installation's permissions automatically. Ask the workspace administrator or organization owner if an expected repository is missing. Do not request broader access than your work requires.

Can I disconnect without breaking everyone's sync?

The personal control is **Disconnect personal account**. It removes your personal connection; the UI warns that synced writes will use service identity until you reconnect. Workspace installation controls have a wider effect and can disable mappings. Check the label carefully and use the personal account page for a personal disconnect.

Why is a change credited to the service account?

The sender may be unlinked, a bot, a deleted-user identity, or missing a usable credential. Some integration-created records intentionally use service identity. The authenticated webhook sender determines Smart Commit attribution, not the forgeable author or committer fields in a git commit. Reconnecting helps eligible future personal actions; it does not rewrite historical attribution.

15.3 Linking and completion

Do I need the key in the branch, commit and PR?

A valid reference in an applicable scanned source can be enough. Repeating the key thoughtfully improves readability, but it also creates opportunities for conflicting intent. For PR linkage, the handler scans the branch, title and body. Under the defaults, a bare branch key is closing, a bare uppercase PR title key is closing, and a bare body key is relation-only. Explicit configured magic words assign a tier. The parser keeps the strongest tier it finds for each identifier. Commit messages are processed separately on push for Smart Commits and applicable push rules; do not rely on a commit message alone to establish the PR link.

Why did `Refs WEB-42` still allow completion?

Look for a stronger source. Branch `web-42-fix`, title `WEB-42 Fix layout`, or a closing phrase in another scanned source can make the final tier closing. A non-closing phrase does not veto a closing one. For a partial contribution, omit the key from the branch and use explicit non-closing wording in the PR title and body. A branch such as `refs/web-42/layout` still has a bare closing key. Verify the intended links before merge. Existing stored links persist even if the text is later removed or suppressed with `skip`; ask an administrator about an association established in error.

Is `Closes WEB-42` the same as `#done`?

No. `Closes WEB-42` is a reference classified for PR lifecycle automation. A qualifying merge can complete the work item under the applicable rules. `WEB-42 #done` is a Smart Commit state command interpreted when the commit is processed from a push, assuming `Done` resolves to a state. It can change the work item before code review or merge.

Does GitHub understand Plane keys itself?

The integration reads the Plane identifier. GitHub issue numbers use their own repository context. A Plane work item and a GitHub issue can be mirrored, but that is a separate opt-in relationship. Do not assume a native GitHub closing expression and a Plane closing expression refer to the same record.

What happens if one PR addresses two items?

Use an explicit phrase such as `Closes WEB-42, WEB-45` if the PR really finishes both. The parser supports multiple identifiers after one magic word. Each work item’s other linked PRs still matter. Verify both records after merge because one can remain open while the other completes.

What happens if one item needs two PRs?

The completion check requires at least one merged linked PR and no open linked PRs. A closed, unmerged PR neither establishes completion nor blocks a merge transition. The service treats a PR it cannot read as open rather than guessing it is finished. Use non-closing references for contributions when merge must not mean the entire item is complete.

Will a parent item close when all its children close?

Do not rely on git-sync to provide a child-to-parent completion rule. It evaluates the linked work items and PRs described in its reference and lifecycle logic. Parent/sub-work-item planning in Plane does not automatically create a PR reference for the parent. Follow the team’s explicit parent acceptance and completion process.

Does reverting a PR reopen the work item?

There is no dedicated inverse operation that undoes the old PR’s state, comments or time records. A revert is new code and new events. Review generated revert text because copied keys and closing phrases can still be recognized. Reopen the old item deliberately when appropriate, or track the corrective work with a new item and explain the relationship.

15.4 Comments, time and issue mirroring

Where does `#time` go?

It becomes a Plane comment containing the duration and calculated minutes, with provenance. The CE integration does not write a native worklog. Duration units are weeks, days, hours and minutes; a day is eight hours and a week is five days. Decimals are supported. Agree with the team before treating these comments as reportable time records.

Can a command span two commit-message lines?

No. Each parsed line needs its identifiers before the first command on that same line. Put `WEB-42` again on a new line that contains another command. A `#comment` takes the rest of its line as text. Put it after state and time commands to avoid accidentally turning commands into prose.

If a push failed halfway through, should I make another commit with the commands?

First inspect the actual result and ask an administrator about pending steps. The integration records completed steps and can retry transient failures without repeating completed actions for the same commit identity. A new commit has a new SHA and can repeat effects. This is especially relevant for time comments and messages that were already written successfully.

Will every GitHub issue become a Plane work item?

No. The ordinary mirror uses opt-in labels: `plane` on the GitHub issue or `github` on the Plane work item. The mapping and allowed direction still govern creation. A PR reference is not the same as a mirrored issue pair. Check for an existing partner before creating a manual duplicate.

Will an edited comment update on both sides?

The mirror handles newly created comments. It is not a shared live document that synchronizes comment edits or deletions. Add a clear correction when both audiences need the new information. Markdown and Plane’s rich text are converted representations; complicated formatting may not survive identically, even when the meaning remains readable.

Why is a label, assignee or state different on each side?

Your administrator can configure direction and field policy. Identity mapping also affects which assignees can be represented. Some labels are integration opt-in controls rather than normal mirrored categorization. Read the team’s policy before editing both systems to force agreement; competing edits can obscure the real ownership decision.

15.5 Trusting what you see

Does `MERGED` mean the feature is live?

It means the PR merged. A deployment and a release are later or separate events in many teams. Development can also show deployment or preview links when reported data is available. Check the intended environment and team release process before telling a user a fix is live.

Why can the panel show `OPEN · APPROVAL` while the item is In Review?

The first labels describe the PR and its review summary. In Review is a Plane work item state chosen by rules or people. Approval has no shipping transition to Done. A team can configure different rules, but the displayed labels are still distinct concepts.

How long should synchronization take?

This handbook does not promise a fixed number of seconds. Delivery, queue load, API limits and retries affect timing. Check once after allowing processing time. If a result stays missing, collect the identifier, URL, action time, expected behavior and actual behavior. An administrator can tell whether the event is missing, waiting, rejected or intentionally ignored.

Why are some bot edits ignored?

The service filters its own Plane service-user events and recognized echoes to prevent endless copying. That is expected. It does not mean every action by every GitHub bot is discarded. Applicable bot actions can run under service identity. Ask about the specific event rather than relying only on the displayed account name.

Can I fix a missing link by deleting and recreating the item?

That usually loses context and can create duplicates. First verify the key, mapping, permissions and event. Deletion is a separate lifecycle with reconciliation limits. Preserve the original evidence while an administrator investigates. Recreate only after the team understands what happened and has decided which record should remain authoritative.

15.6 Glossary

Term	Meaning in this handbook
Acceptance criteria	Concrete checks that show the requested outcome has been delivered.
Activity history	The record of changes and discussion associated with a work item.
Administrator	A person authorized to manage workspace, project or instance configuration; the scope matters.
Assignee	A person responsible for moving a work item forward.
Base branch	The branch a PR proposes to merge into, such as `main`.
Blocking relation	A statement that one work item depends on another outcome first.
Branch	A named line of development in git. A local branch reaches GitHub only after a push or remote creation.
Build	A reported automated check or collection of checks; its scope depends on the repository.
Closing reference	A link tier eligible for merge completion, subject to rules and all linked PRs.
Commit	A recorded git change with a message and an identifying SHA.
Cycle	A planning period used to organize a time-bounded set of project work.
Deployment	An event placing a version in an environment; it is separate from merging code.
Development panel	Work-item view of linked PRs, branches and available build, deployment and preview information.

Term	Meaning in this handbook
Direction	The configured allowance for changes to travel from GitHub, Plane or both.
Draft	Saved incomplete work; a Plane draft and a GitHub draft PR are different kinds of record.
Echo	An event produced when the service's own write is reported back by the other system.
Field policy	A per-field decision about which changes the integration may apply.
Gate label	An opt-in label used to request ordinary issue mirroring: `plane` on GitHub or `github` in Plane.
GitHub App	The installed integration identity with repository permissions granted by its owner.
Identifier	A project's prefix and work-item sequence, such as `WEB-42`.
Intake	A place to receive and triage incoming work before normal project handling.
Linkback	A cross-link or comment that helps a reader reach the related record; it can be configured.
Magic word	Configurable prose such as `Closes` or `Refs` that classifies a following reference.
Mapping	The configuration joining a Plane project and a GitHub repository with sync policy.
Mention	A reference to a person in discussion, often used to bring a question to their attention.
Mirror pair	An opted-in GitHub issue and Plane work item whose allowed fields synchronize.
Module	A planning container for a related body of work within a project.

Term	Meaning in this handbook
Non-closing reference	A contribution link that can take part in earlier lifecycle changes but does not complete on merge.
Orphan	A link whose partner has been confirmed missing by the integration’s reconciliation process.
Personal connection	Your linked GitHub account and associated authorization for attribution.
Preview	A reported review environment link, often temporary; it is not a production release guarantee.
Priority	The relative urgency assigned to a work item.
Project	A grouping of related work with its own identifier prefix, members and workflow settings.
Pull request / PR	A proposal to review and merge changes from one branch into another.
Push	Sending local git commits and branch updates to a remote repository.
Queue	Durable pending work processed by the service after an event arrives.
Reconciliation	A background check that repairs or classifies differences missed by normal event handling.
Relation reference	A contextual link that does not make lifecycle state changes.
Review summary	The aggregate review label, such as REVIEW, APPROVAL or REJECTED, shown for an open PR.
Rule	A configured trigger, conditions and actions that describe automation behavior.

Term	Meaning in this handbook
Saved view	A reusable way to filter and display work items; it does not create another copy of each item.
Service account	The integration’s non-personal identity used for system writes or permitted fallback.
SHA	The git commit identifier used to distinguish recorded commit actions.
Smart Commit	A commit-message line containing work-item keys and commands interpreted from a push.
State	A named point in the project’s workflow, such as In Review.
State group	A broader workflow category, such as started or completed, containing named states.
Sub-work item	A smaller work item connected to a parent to break down a larger outcome.
Subscription	Following a work item to stay informed about its activity.
Webhook	A signed event notification sent by one system to the integration.
Work item	The Plane record describing an outcome, its ownership, workflow and discussion.
Workspace	A shared Plane area containing members, projects and workspace-level settings.

The source references for the integration vocabulary are `src/parse/magicWords.ts`, `src/parse/references.ts`, `src/github/smart-commits.ts`, `src/github/pr-automation.ts`, `src/sync/dev-info.ts` and `src/sync/mirror.ts`. The complete field and event reference is in chapter 11. Keep the printable page that follows nearby while you learn the daily flow.

16. Printable cheat sheet

Plane + GitHub daily loop · Example project `WEB` · Check your team's rules.

1. Open the work item.
2. Read its acceptance checks and blockers.
3. Confirm ownership.
4. Move it to In Progress.
5. Create a feature branch.
6. Implement the change.
7. Run relevant checks.
8. Commit the intended files.
9. Push the branch.
10. Open the PR with a Plane reference.
11. Request review.
12. Resolve feedback.
13. Merge after required checks and review.
14. Verify Plane's state and Development.

```
git switch -c web-42-checkout-button
# Edit, check and stage the intended files.
git commit -m "WEB-42 improve checkout" \
  -m "WEB-42 #time 45m #comment Checked narrow screens."
git push -u origin web-42-checkout-button
```

Choose the reference

Intent	PR text
Finish work	`Closes WEB-42`
Contribute part	`Refs WEB-42`
Context only	`Relates to WEB-42`

Strongest wins. Bare branch keys and uppercase PR title keys close. For partial work, use a branch without the key and explicit `Refs WEB-42`. A slash-prefixed key still closes. `skip WEB-42` in the body excludes before the first link; it does not remove an existing link.

```
WEB-42 #in-review #time 30m #comment Ready for review.
```

Keep keys **before commands, on the same line**. Put `#comment` last. Commands run after **push**. `#time` writes a CE comment; `1d = 8h`, `1w = 5d`.

Defaults: branch/PR → In Progress; review request → In Review; qualifying merge into `main` → Done. Approval alone changes no state. Other open linked PRs block completion at that merge event.

Development: **OPEN** = open PR; **MERGED** = merged; **DECLINED** = closed unmerged. Review labels are separate. Merge does not prove deployment.

> Tip: If stuck, report the key, event URL/SHA, time, expected result and actual result. Never share tokens or cookies.